

В.Ю. Пирогов



**АСЕМБЛЕР GAS
В ОПЕРАЦИОННОЙ СИСТЕМЕ
LINUX
НА ПЛАТФОРМЕ X86-64**

ФЛИНТА

В.Ю. Пирогов

**АССЕМБЛЕР GAS
В ОПЕРАЦИОННОЙ СИСТЕМЕ LINUX
НА ПЛАТФОРМЕ X86-64**

Монография

2-е издание, стереотипное

Москва
Издательство «ФЛИНТА»
2024

Шадринск
ШГПУ
2024

УДК 004.431.4
ББК 32.973.21
ПЗЗ

Рецензенты:

Баландин А.А. – канд. пед. наук,
доцент кафедры программирования и автоматизации бизнес-процессов
ФГБОУ ВО «Шадринский государственный педагогический университет»
Семахина А.М. – канд. технич. наук, доцента кафедры программного
обеспечения автоматизированных систем
ФГБОУ ВО «Курганский государственный университет»

Пирогов В.Ю.

ПЗЗ Ассемблер GAS в операционной системе Linux на платформе x86-64 : монография / В.Ю. Пирогов. — 2-е изд., стер. — Москва : ФЛИНТА, 2024. — 176 с. — ISBN 978-5-9765-5586-0 (ФЛИНТА) ; ISBN 978-5-87818-702-2 (ШГПУ). — Текст : электронный.

В монографии рассматриваются различные аспекты низкоуровневого программирования для 64-битовых операционных систем Linux. На основе многочисленных примеров проводится анализ 64-битового программирования. Значительная часть монографии посвящена программно-архитектурным особенностям систем x86-64, в частности рассмотрению команд микропроцессора. В монографии рассматриваются также особенности интеграции языка ассемблер с языками высокого уровня. Часть монографии посвящена особенностям ассемблера GAS, его 64-битовой подсистеме.

Монография предназначена программистам, разрабатывающим приложения для Linux. Она также может быть использована как учебное пособие в высших и средних учебных заведениях на инженерных специальностях по таким дисциплинам как низкоуровневое программирование, системное программирование, программирование на языке ассемблера и др.

УДК 004.431.4
ББК 32.973.21

ISBN 978-5-9765-5586-0 (ФЛИНТА)
ISBN 978-5-87818-702-2 (ШГПУ)

© Пирогов В.Ю., 2024
© ШГПУ, 2024

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	5
ГЛАВА 1. АССЕМБЛЕРЫ И ПЛАТФОРМА X86-64	9
1.1. Платформа x86-64.....	9
1.2. Ассемблеры для платформы x86-64	11
1.2.1. Masm.....	12
1.2.2. Tasm	13
1.2.3. Nasm	13
1.2.4. Yasm	14
1.2.5. Fasm	15
1.2.6. Gas	15
1.3. Аппаратная архитектура	16
1.3.1 Архитектура процессора x86-64.....	16
1.3.2. Регистр флагов.....	19
ГЛАВА 2. АССЕМБЛЕР В ОПЕРАЦИОННОЙ СИСТЕМЕ LINUX	23
2.1. Основы программирования на языке ассемблера в операционной системе Linux	23
2.1.1. Языки высокого уровня и ассемблер	23
2.1.2. Ассемблер GAS	36
2.1.3. Об основах программирования на платформе x86-64 на ассемблере GAS	42
2.1.3.1. Адресация.....	42
2.1.3.2. Условные и безусловные переходы	44
2.1.3.3. Системные вызовы	50
2.2. Стек и функции	55
2.2.1. Структура стека.....	55
2.2.2. Вызов функций.....	58
2.2.3. Передача параметров в функцию и локальные переменные	63
2.2.4. Многомодульное программирование на ассемблере GAS	71
2.3. Интеграция ассемблера и языков высокого уровня	77
2.3.1. Использование программы gcc для компилирования ассемблерных модулей	78
2.3.2. Параметры командной строки	80
2.3.3. Использование ассемблерных модулей на языках высокого уровня.....	87
2.3.4. Статические библиотеки	90
2.3.5. Использование языков высокого уровня на языке ассемблера	95
2.3.6. Динамические библиотеки.....	100

2.4. Основы системного программирования в операционной системе Linux.....	102
2.4.1. Файловая система.....	102
2.4.2. Доступ к содержимому файлов	103
2.4.3. Управление файловой системой.....	110
2.4.4. Управление памятью.....	121
2.4.4.1. Виды памяти	121
2.4.4.2. Динамическая память и файлы отображаемые в памяти .	131
2.4.5. Управление процессами	139
2.4.5.1. Запуск процессов и создание процессов	139
2.4.5.2. Взаимодействие процессов.....	152
ЗАКЛЮЧЕНИЕ	159
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	160
Приложение 1. Список системных функций Linux, используемый в работе с кратким описанием в нотации языка C	167
Приложение 2. Система команд процессора X86-64	169

ВВЕДЕНИЕ

Приступая к книге об языке ассемблера, следует прежде всего ответить на следующие вопросы:

1. Что такое язык ассемблера?
2. О какой аппаратной платформе пойдет речь?
3. Какая программная платформа (операционная система) будет использована?
4. Какой ассемблер для выбранных платформ будет рассматриваться?

Приступим к ответам на поставленные вопросы.

Языком ассемблера будем называть язык программирования, в основе которого лежат команды процессора, обозначаемые удобным для восприятия человека способом. Обозначения обычно представляют собой слова или сокращения на одном из естественных языков, чаще всего английском языке. Основная идея заключается в том, что в таком виде язык становится более понятным и более удобным для восприятия, чем набор двоичных кодов. Например, команда `mov`, сокращение от английского `move`, т.е. перемещать. Используется для обозначения команды перемещения данных. Кроме символьного обозначения команда языка ассемблера содержит в себе как правило следующие элементы [12, 13]:

- возможность использовать символьные метки, для обозначения адреса команды или адреса области памяти (переменной);
- именованные константы, которые в процессе трансляции переводятся в конкретные значения;
- макросы, позволяют обозначить определенную последовательность кода именем, что дает возможность заменять подобный код в программе именем макроса с возможностью параметрической настройки его;
- директивы ассемблера, позволяющие влиять на процесс трансляции программы.

Процесс трансляции программы на языке ассемблера обычно называют ассемблированием. В результате ассемблирования создается двоичный файл, который может исполняться в той или иной среде (операционной системе) или служить частью другого исполняемого файла. Следует особо отметить, что в литературе термин ассемблер используется и как синоним термину «язык ассемблера» и для обозначения программы для трансляции программ, написанных на языке ассемблера.

В качестве аппаратной платформы нами взята платформа, обычно

обозначаемая как x86–64. Термин x86–64 имеет несколько синонимов, таких, например, как Intel 64, AMD64 и др. Это вносит определенную нечеткость в изложение, хотя, по сути, имеется в виду одно и то же – 64-битовое расширение архитектуры x86. Архитектуры x86-64, разрабатываемые корпорациями Intel и AMD [23-28, 33-38], отличаются друг от друга, но это сказывается в основном при разработке компиляторов и операционных систем [62]. На разработку прикладного программного обеспечения это не влияет. Так что мы в дальнейшем не будем акцентировать внимание на эти различия.

За основу взята программная платформа, основанная на Unix-подобных операционных системах [17, 19-20], учитывая, что популярность этих операционных систем с каждым годом в нашей стране увеличивается.

В качестве основного инструмента взят ассемблер GAS (GNU assembler), входящий в пакет GNU Binutils¹ и входящий в тот же пакет.

Остановимся теперь подробнее на содержательной стороне представленной ниже монографии. Прежде всего следует отметить, что литературы по программированию на языке ассемблера для операционной системы Linux на русском языке практически нет, за исключением авторского учебного пособия [18]. Англоязычная литература представлена в следующих книгах [57-62]. Это в значительной степени определило содержание монографии. В определенной степени она дополняет книгу [18], пропуская некоторые уже изложенные там вопросы, либо излагая их конспективным образом.

Структура книги разбита на две главы. Первая глава посвящена двум вопросам: краткому обзору существующих ассемблеров для платформы x86-64 и изложению программной архитектуры x86-64. Особый акцент сделан на особенностях архитектуры x86-64 в сравнении с архитектурой x86 (32). Дана также аргументация выбора ассемблера GAS в сравнении с другими ассемблерами.

Вторая глава посвящена программированию с использованием ассемблера GAS в операционной системе Linux. Рассмотрены следующие вопросы:

1. Особенности компиляции программ на языках высокого уровня.
2. Особенности структуры программ на языке ассемблера GAS.
3. Адресация в x86-64 и GAS.
4. Условные конструкции.

¹ Набор утилит в Unix-подобных системах для работы с двоичными исполняемыми файлами. В частности, в набор входят утилиты `as`, `ld`, `ar`, которые будут использованы в нашей книге.

5. Понятие системного вызова.
6. Различные аспекты стековой памяти при программировании на ассемблере x86-64.
7. Технология интеграции языка ассемблера с языками высокого уровня.
8. Многомодульное программирование на языке ассемблера.
9. Динамические и статические библиотеки.
10. Основы программного управления файловой системой.
11. Основы доступа к динамической памяти.
12. Основы многозадачного программирования.

В конце монографии представлены два приложения: обзор используемых в книге системных вызовов, полный обзор команд процессора x86-64.

ГЛАВА 1. АССЕМБЛЕРЫ И ПЛАТФОРМА X86-64

1.1. Платформа x86-64

Следует отметить, что элементы 64-ого представления данных зародилось еще в 1960-е годы [31]. Еще в 1961 году IBM выпускает компьютер IBM 7030, в котором можно использовать 64-битовые операнды. Т.е. истории с 64-битовой компьютерной архитектурой уже как минимум 60 лет. В 1994 году корпорация Intel объявила о планах разработки 64-битовой архитектуры IA-642, которая основывалась на процессоре IA-323. Следует отметить, что планируемая архитектура не имела ничего общего с уже широко используемой в персональных компьютерах архитектурой x86, которая производилась Intel. Однако в 1999 году AMD разрабатывает 64-битовое расширение для x86, тем самым опередив корпорацию Intel. Расширение было названо x86-64, а в последствие AMD64. Данная разработка предполагала возможность выполнения программ в 64-разрядном режиме. Корпорация Intel вынуждена была опираться уже на существующие разработки AMD, в частности на определенный уже набор команд, и выпустила свой вариант архитектуры, называемый Intel 64 (также EMT64).

Процессоры данной архитектуры могут работать в двух режимах: длинный режим (long mode) и устаревший наследуемый режим (legacy mode). Первый режим является основным с 64-битовыми базовыми регистрами и возможностью адресовать память с помощью 64-битовых адресов. Этот режим позволяет использовать 64-битовые операционные системы. Важно отметить, что в этом режиме имеется поддержка запуска 32-битовых приложений с поддержкой как в самой архитектуре x86-64, так и в 64-х битовых операционных системах. Следует отметить, что в 64-битовом режиме можно использовать и 64-битовые и 32-битовые регистры. Кроме того, сам набор основных регистров процессора был увеличен с восьми до шестнадцати.

Следует также отметить, что в x86-64 были удалены некоторые устаревшие возможности процессора x86. В частности, была удалена возможность сегментной адресации, сегментные регистры fs и gs остались. Был убран также режим virtual 8086 и механизм переключения задач. Впрочем, все эти рудименты остались в наследуемом режиме.

-
- 2 IA-64 является другим обозначением процессора Intel Itanium (также используется название IPF), который несовместим с платформой x86-64.
 - 3 Совместимость между IA-32 и IA-64 весьма условна. В частности, речь идет о производительности кода для IA-32, исполняемого в системе на основе IA-64.

В наследуемом режиме предполагается возможность выполнять команды процессора x86. Соответственно в нем можно запустить 32-битовую операционную систему в полной совместимости с 32-битовыми приложениями. В этом режиме нельзя использовать 64-битовые регистры.

Основные операционные системы поддерживают длинный режим x86-64: семейство ОС BSD (FreeBSD, DragonFly BSD, NetBSD, OpenBSD), операционные системы Linux, операционные системы macOS, операционная система Solaris, операционные системы Windows (Windows XP Professional x64, Windows Server 2003 x64, Windows Server 2008, Windows 7, Windows 8, Windows 10 и т.д.).

Следует отметить, что между реализациями AMD64 и Intel 64 есть определенные различия, в частности на уровне набора инструкций [23-28, 32-38]. Однако эти различия важны только для разработчиков операционных систем и компиляторов. Компилятор, установленный в операционной системе, производит код, который будет корректно выполняться в данной операционной системе.

Говоря о микропроцессоре, следует выделить два важных момента: 1. Размеры числовых данных, над которыми может осуществлять операции процессор. 2. Размер физической и виртуальной памяти, которую может адресовать процессор. Процессоры x86-64 в своем базовом наборе регистров могут выполнять операции над 64-битовыми операндами. Однако для данного процессора разработаны расширения (см. Приложение 2), позволяющие манипулировать и большими операндами, с размерами в 128 битов (SSE24 расширение). Разработано также расширение AVX5 с возможностью оперировать 256 битами. Что касается адресации, в настоящее время процессоры поддерживают 48-битовую адресацию для виртуальной памяти и 40-битовую адресацию для физической памяти.

1.2. Ассемблеры для платформы x86-64

Имеется несколько ассемблеров, которые могут быть использованы для платформы x86-64. Следует отметить, что ассемблеры могут отличаться не только по аппаратной платформе, но и по отношению к операционным системам. Программа, написанная для использования в одной операционной системе на кроссплатформенном ассемблере, часто с большим трудом может быть перенесена в другую операционную систему

4 SSE - Streaming SIMD Extensions (потокковые расширения SIMD), SIMD - Single instruction, multiple data — одна инструкция, много данных.

5 AVX - Advanced Vector Extensions (продвинутые векторные расширения).

той же аппаратной платформы. Кроме того, ассемблеры могут отличаться программным синтаксисом, особенно в части макросредств.

1.2.1. Masm

Название MASM раскрывается как Microsoft Macro Assembler. Изначально был предназначен для программирования в операционной системе MS DOS. В дальнейшем он бы адаптирован для программирования в 32-битовой операционной системе Windows [12]. Позднее Стивен Хатчессон разработал пакет для программирования на 32-битовом ассемблере для Windows – MASM326. Корпорация Microsoft перестала работать на MASM, как над коммерческим проектом, оставив его для внутреннего пользования. Начиная с выхода пакета Visual Studio 2005 ассемблер стал 64-битовым. Теперь компилятор называется ml64.exe и автоматически устанавливается при установке Visual Studio. Конечно, существуют извлечения ассемблера из основного пакета Visual Studio, но это уже неофициальное программистское творчество, результаты которого можно найти в сети Интернет.

Основным минусом ассемблера MASM является его ориентация только на семейство операционных систем Windows. С другой стороны, кроссплатформенность ассемблера, не является значительным плюсом в использовании, если речь идет о таких разных операционных системах, как Windows и Linux. Перенос ассемблерной программы между такими системами фактически сводится к переписыванию их заново. Причиной этому служит и разный интерфейс с операционной системой, и разные библиотеки и, даже, разный протокол вызова функций.

6 См. сайт The MASM32 SDK — <https://www.masm32.com/>.

1.2.2. Tasm

TASM или Turbo Assembler был разработан корпорацией Borland в 1989 году. Был ориентирован на создание 16- и 32-битовых приложений для операционной системы MS DOS, а затем и для операционной системы Windows. Транслятор использовался в других языках программирования, компиляторы для которых разрабатывала Borland. Кроме самого ассемблера в пакет выходил также компоновщик (tlink.exe), отладчик и библиотекарь (tlib.exe). TASM мог компилировать в режиме, совместимом с MASM, имелся также расширенный режим IDEAL, улучшающий его функциональные возможности. Последняя версия TASM датируется 1996 годом (версия 5.4), поддерживалась до 2010 года. Для TASM была написана среда разработки TASM Visual7. 64-битовой версии TASM (для разработки 64-битовых приложений) не было разработано. В настоящее время данный ассемблер интересен скорее с исторической точки зрения⁸.

1.2.3. Nasm

NASM или Netwide assembler (расширенный ассемблер) – ассемблер, ориентированный на архитектуру x86. Особенностью ассемблера является его кроссплатформенность. Ассемблер может быть использован для Unix-подобных операционных систем, операционных систем семейства Windows, MacOS, MS DOS. Кроме того, NASM поддерживает создание простых бинарных файлов, например для создания загрузочных программ. Имеется сайт поддержки⁹ с весьма обширной документацией и возможностью скачать последнюю версию ассемблера. Распространяется NASM под так называемой упрощенной (simplified) BSD лицензии. В пакете NASM отсутствует свой компоновщик и для получения исполняемых файлов используются компоновщики, ориентированные на соответствующую операционную систему (link.exe, ld и т.п.). Первая версия ассемблера появилась в 1996 году (версия 0.9). Последняя версия датируется 2020 годом (2.15). В варианте языка, поддерживаемого NASM, используется разновидность основанная на Intel-синтаксисе. Кроссплатформенность NASM позволяет разрабатывать исполняемые модули для одной операционной системы, находясь в другой. Особенностью ассемблера является также его многопроходность, которая

⁷ Сайт поддержки программы располагается по адресу <http://gri-software.ru/tasmvisual.html>.

⁸ Полное описание одной из последних версий TASM можно найти, например, здесь <http://old-dos.ru/books/2/3/2/TASm5.pdf>.

⁹ См. <https://nasm.us/>, а также <http://www.opennet.ru/docs/RUS/nasm/>.

позволяет использовать макросредства на каждом из проходов.

1.2.4. Yasm

При разработке ассемблера YASM¹⁰ за основу был взят ассемблер NASM¹¹, благо это позволяла лицензия BSD. Также как и NASM это кроссплатформенный продукт, позволяющий использовать его в разных операционных системах (Unix-подобных систем, MacOS, операционных систем семейства Windows). При написании программ YASM допускает использование и синтаксиса Intel и синтаксиса AT&T, берущего свое начало в историческом развитии Unix. Особенностью YASM также является то, что он легко интегрируется в Microsoft Visual Studio. Также в YASM добавлена возможность генерации отладочной информации в формате CodeView. Последняя версия YASM датируется 2019 годом.

1.2.5. Fasm

FASM или Flat Assembler – ассемблер, разработанный Томашем Грыштаром (Tomasz Grysztar)¹². Многопроходность (три прохода) ассемблера, как и в случае с NASM, позволяет, широко использовать макросредства. Многопроходность позволяет также использовать выражения до их использования. Другой особенностью FASM является то, что ассемблер может сам проводить компоновку, без использования компоновщиков. Хотя FASM и придерживается в общем случае синтаксиса Intel, в нем введено ряд дополнительных правил, позволяющих упростить синтаксис, по сравнению, скажем с синтаксисом MASM. Для установки в Linux достаточно просто скачать и развернуть архив с транслятором и примерами. Последнее обновление ассемблера датируется 2022 годом (версия 1.73.30).

1.2.6. Gas

GAS – GNU¹³ Assembler, ассемблер, разработанный для поддержки компиляторов в операционной системе Linux. Входит в пакет GNU Binutils [57-62]. Первая версия ассемблера была разработана в 1986 году Динем

¹⁰ Одна из расшифровок названия YASM - Yet another assembler, т. е. Еще один ассемблер.

¹¹ На сайте YASM (<http://yasm.tortall.net/>) написано буквально следующее: «Yasm is a complete rewrite of the NASM assembler under the “new” BSD License», т. е. «Yasm это полностью переписанный ассемблер NASM по лицензии new BSD».

¹² Сайт разработчика располагается по адресу <https://flatassembler.net/>.

¹³ GNU — проект по разработке свободного программного обеспечения. В частности, в рамках этого проекта разрабатываются операционные системы Linux.

Элснером (Dean Elsner) и первоначально был предназначен для VAX-архитектуры. В настоящее время GAS поддерживает различную аппаратную архитектуру. За основу взят синтаксис AT&T. В последних версиях появилась поддержка синтаксиса Intel. GAS ориентирован в основном на операционные системы Linux, но поддерживает также MAC OS.

1.3. Аппаратная архитектура

1.3.1 Архитектура процессора x86-64

Прежде всего, что будем понимать под архитектурой микропроцессора, в данном случае микропроцессора x86-64. Прежде всего следует выделить ту часть архитектуры, которая имеет прямое отношение к программированию. Я бы разделил этот вопрос на две части: 1. Система регистров микропроцессора. 2. Система команд микропроцессора. Ниже мы остановимся на регистрах микропроцессора, рассмотрение же системы команд отложим до второй главы, поскольку этот вопрос корректнее излагать вместе с языком ассемблера.

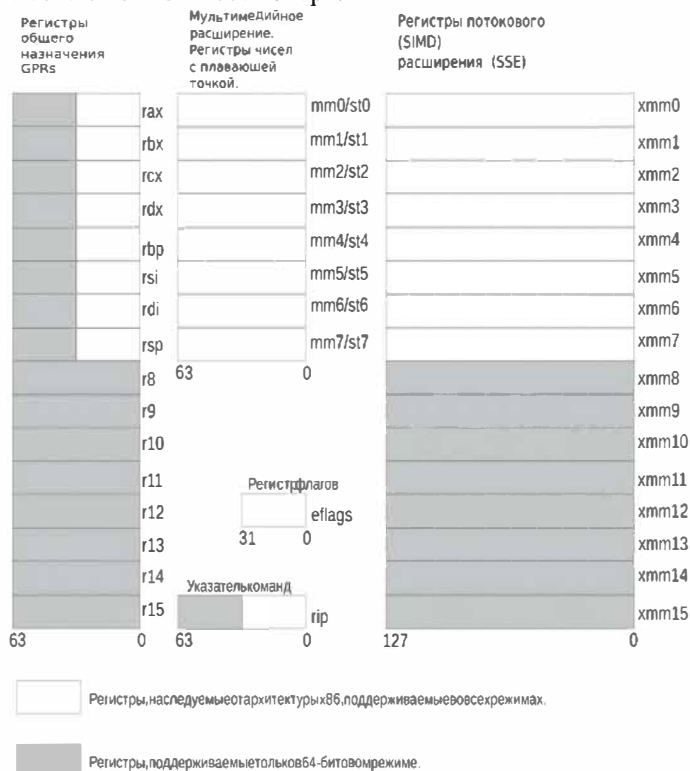


Рис.1. Регистровая архитектура процессора x86-64

Регистры процессора можно назвать собственной памятью процессора. Эта память не велика, по сравнению с оперативной памятью, но имеет важное преимущество перед ней – быстродействие. Поэтому один из важных принципов программирования на языке ассемблера – часть данных переносится из оперативной памяти в регистры и там производятся основные операции. Следует также отметить, что количество команд,

выполняемых непосредственно над регистрами процессора больше, чем команд, выполняемых только над ячейками памяти. Обратимся к рисунку 1, где в графическом виде представлены регистры, непосредственно используемые в программировании. Прежде всего обратим внимание на группу регистров общего назначения (GPR - General Purpose Register). Они принимают на себя большую часть нагрузки в прикладных программах. Серая часть прямоугольника, обозначающего регистр, обозначает нововведение в x86-64. В предыдущей версии процессора существовало 8 регистров: *eax*, *ebx*, *ecx*, *edx*, *ebp*, *esi*, *edi*, *esp* с размерностью 32 бита. В новой версии они дополнены еще 32 битами и получили новые наименования. При этом доступ к отдельным младшим частям регистров остался (см. ниже) и в новом режиме, и в наследуемом. Регистры *r8* – *r15* это новые регистры, которых не было в предыдущей версии процессора. Важно отметить, по доступу к отдельным частям эти регистры полностью аналогичны «старым» регистрам (см. Таблица 1).

При написании программ на языке ассемблера следует понимать возможности машинного языка по доступу к отдельным частям регистров общего назначения. В Таблице 1 представлены возможности такого доступа. Рассмотрим, например, регистр *rax*. Можно использовать инструкции (например, арифметические операции или передачу данных) для полного 64-битового регистра (*rax*), для младшей 32-й части (*eax*), для младшей 16-битовой части (*ax*), для младшей 8-битовой части (*al*), для старшей 8-битовой части регистра *ax* (*ah*).

Таблица 1

Доступ к элементам регистров общего назначения

64-битовый регистр	Младшая 32-битовая часть	Младшая 16-битовая часть	Младшая 8- битовая часть	Старшая 8- битовая часть в младшем слове
<i>rax</i>	<i>eax</i>	<i>ax</i>	<i>al</i>	<i>ah</i>
<i>rbx</i>	<i>ebx</i>	<i>bx</i>	<i>bl</i>	<i>bh</i>
<i>rcx</i>	<i>ecx</i>	<i>cx</i>	<i>cl</i>	<i>ch</i>
<i>rdx</i>	<i>edx</i>	<i>dx</i>	<i>dl</i>	<i>dh</i>
<i>rsi</i>	<i>esi</i>	<i>si</i>	<i>sil</i>	
<i>rdi</i>	<i>edi</i>	<i>di</i>	<i>dil</i>	
<i>rbp</i>	<i>ebp</i>	<i>bp</i>	<i>bpl</i>	
<i>rsp</i>	<i>esp</i>	<i>sp</i>	<i>spl</i>	
<i>r8</i>	<i>r8d</i>	<i>r8w</i>	<i>r8b</i>	

<i>r9</i>	<i>r9d</i>	<i>r9w</i>	<i>r9b</i>	
<i>r10</i>	<i>r10d</i>	<i>r10w</i>	<i>r10b</i>	
<i>r11</i>	<i>r11d</i>	<i>r11w</i>	<i>r11b</i>	
<i>r12</i>	<i>r12d</i>	<i>r12w</i>	<i>r12b</i>	
<i>r13</i>	<i>r13d</i>	<i>r13w</i>	<i>r13b</i>	
<i>r14</i>	<i>r14d</i>	<i>r14w</i>	<i>r14b</i>	
<i>r15</i>	<i>r15d</i>	<i>r15w</i>	<i>r15b</i>	

Следующая группа из восьми регистров – 64-битовые регистры мультимедийного расширения. Этот набор регистров, вместе с набором обслуживающих их команд, существовали и в старой модели. Особенностью данного набора регистров является то, что они совмещаются с регистрами мантисс арифметического сопроцессора. Такое совмещение не позволяет одновременно использовать обе группы регистров. Инструкции процессора поддержки этой группы регистров обрабатывает данные, состоящие 64-битовых групп целых чисел или отдельных 64-битовых целых чисел, находящихся в данных регистрах или в памяти.

Регистры потокового расширения (SSE) также существовали 32-битовых процессорах x86, теперь их количество удвоилось, т.е. На платформе x86-64 их теперь 16. Особенностью этих регистров является то, что они 128-битовые. В регистр можно упаковать 4 32-битовых вещественных или целых числа или два 64-битовых числа. В определенном смысле расширение SSE является заменой системе арифметического сопроцессора. Хотя здесь и отсутствует возможность вычисления некоторых математических функций (например, тригонометрических). При необходимости придется реализовывать свои алгоритмы приближенного вычисления таких функций.

Кроме перечисленных выше регистров есть еще два, непосредственно участвующих в выполнении команд. Это регистр флагов и указатель команд. Биты регистра флагов меняются в зависимости от результата выполнения некоторых команд, например инструкции вычитания. В свою очередь те или иные биты влияют на результат выполнения других команд. Мы разберем содержимое данного регистра в следующем разделе. Указатель команд *rip* всегда содержит адрес следующей выполняемой команды. В x86-64 появилась возможность использовать *rip* при формировании адреса в той или иной команде.

выполнения). Этот флаг был в свое время предназначен для отладчиков. Если флаг установлен в 1, то после выполнения каждой инструкции микропроцессора управление через прерывание 1 передается отладчику.

9 – IF – Interrupt Enable Flag, флаг разрешения прерываний. Сброс этого флага в 0 приводит к тому, что микропроцессор перестает воспринимать прерывания от внешних устройств.

10 – DF – Direction Flag, флаг направления. Данный флаг учитывается в строковых операциях. Если флаг равен 1, то в строковых операциях адрес автоматически уменьшается.

11 – OF – Overflow Flag, флаг переполнения. Флаг устанавливается в 1, если при выполнении инструкции над числами со знаком, результат превосходит выход за допустимые значения.

12 – 13 – IOPL, I/O Privilege Level, уровень приоритета ввода-вывода. Определяет, какой привилегией должен обладать код, чтобы ему было разрешено выполнить команды ввода-вывода, а также других привилегированных команд.

14 – NT – Nested Task, флаг вложенности задач.

15 – 0, зарезервированный флаг всегда равный 0.

16 – RF – Resume Flag. Флаг возобновления. Используется в обработке прерываний от регистров отладки.

17 – VM – Virtual Mode. Признак работы процесса в режиме виртуального 8086. Этот режим был удален в x86-64.

18 – AC – Alignment Check. Проверка выравнивания. При равенстве этого флага 1 и при обращении к невыровненному операнду вызывает исключение.

19 – VIF – Virtual Interrupt Flag. Виртуальный флаг разрешения прерываний. Виртуальная версия флага IF.

20 – VIP – Virtual Interrupt Pending. Флаг виртуального запроса прерывания.

21 – ID – ID Flag. Проверка на доступность инструкции CPUID. Данная инструкция используется для выдачи технической информации о процессоре.

22 – 31 – 0 – зарезервированные биты.

Использование флагов подробно рассматривается в Приложении 2.

Закljučая главу, отметим:

1. Выбор в качестве основного ассемблера для изложения не случаен, а обусловлен прежде всего тем, что ассемблер GAS фактически является частью программного обеспечения операционной системы Linux.

2. Кроссплатформенность других ассемблеров в данном случае не даёт каких-либо преимуществ, поскольку и структура программы в разных операционных системах и системные вызовы настолько несовместимы, что о переносимости программ на ассемблер с этой точки зрения не может быть и речи. Переносимость же чисто алгоритмических элементов программы легко реализовать из программ на GAS в программу на другом ассемблере для платформы x86-64.

3. Программная архитектура x86-64 значительно расширена по сравнению с 32-битовой архитектурой. Кроме расширения длины самих регистров общего назначения само их количество увеличилось с восьми до шестнадцати. Важной особенностью является возможность доступа к 32-битовой составляющей регистров. Что касается регистра флагов, то здесь изменения минимальны.

ГЛАВА 2. АССЕМБЛЕР В ОПЕРАЦИОННОЙ СИСТЕМЕ LINUX

2.1. Основы программирования на языке ассемблера в операционной системе Linux

2.1.1. Языки высокого уровня и ассемблер

Рассмотрим место ассемблера в разработке программного обеспечения на компилируемых языках высокого уровня. В качестве инструмента будем использовать набор компиляторов gcc (GNU Compiler Collection)¹⁴. Данный набор компиляторов охватывает следующие языки программирования: C, C++, Objective-C, Fortran, Ada, Go, D, а также библиотеки для этих языков. В данный набор не входит компилятор языка программирования Pascal, мы будем использовать компилятор fpc, который может быть установлен из репозитория Linux отдельно.

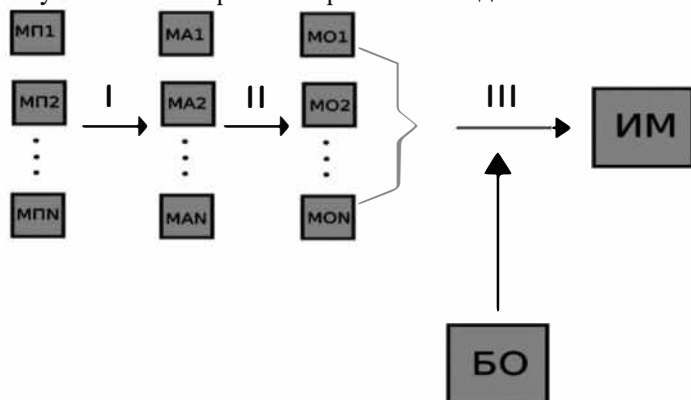


Рис. 3. Схема трансляции программы с языка высокого уровня

Общая схема компиляции программы на языке высокого уровня представлена на рисунке 3. В общем случае можно выделить три этапа компиляции (см. рисунок)

1. Исходные программные модули на языке высокого уровня (МП1...МРН) преобразуются в программные модули на языке ассемблера GAS.
2. Программные модули на языке ассемблера (МА1...МАН) преобразуются в так называемый объектный код на машинном языке с неразрешенными ссылками на внешние функции и переменные.
3. Происходит объединение объектных модулей (МО1...МОН) в

¹⁴ Информацию об языке и компиляторах gcc можно найти на сайте <https://gcc.gnu.org/>.

исполняемый модуль с разрешением внешних ссылок, при котором указываются правильные адреса вызываемых внешних функций и переменных. На этом же этапе подключаются статические библиотеки (БО – библиотека объектная), а также возможно библиотечные модули, с процедурами запускаемыми на начальном этапе выполнения программы. Процесс разрешения ссылок на внешние функции и переменные, называется также *ранним связыванием*, т.е. связыванием на стадии компиляции. Также последний этап компиляции называется этапом *компоновки*.

Перечисленные этапы компиляции не обязательно должны отражаться внешними проявлениями – промежуточными файлами. По умолчанию в случае отсутствия ошибок мы сразу получаем результат в виде исполняемого модуля. Например:

```
gcc prog.c -o prog
```

или

```
cc prog.c -o prog
```

В первом случае мы запустили программу-интерфейс, которая по расширению исходного файла определила нужный компилятор и запустила его. Во втором случае мы запустили сам компилятор.

Описанный выше процесс компиляции, состоящий из нескольких этапов, обычно называют *раздельной компиляцией*. Раздельная компиляция обладает рядом положительных сторон. Перечислим их:

1. Раздельная компиляция позволяет воспользоваться так называемым многомодульным подходом к программированию, когда вся программа разбивается на несколько файлов, что позволяет легче ориентироваться в ее содержании.

2. Раздельная компиляция позволяет в многих случаях сократить время повторных компиляций, поскольку можно использовать готовые промежуточные файлы.

3. Раздельная компиляция дает возможность командной разработки программы, распределяя отдельные модули между членами команды разработчиков.

4. Одним из серьезных преимуществ раздельной компиляции – возможность использования объектных статических библиотек, содержащих готовый код для реализации того или иного алгоритма.

Компиляторы, входящие в коллекцию GCC, имеют ключи, позволяющие выделить ту или иную стадию компиляции. Рассмотрим, например, простую программу на языке fortran (см. Рисунок 4).

```

program good_morning ! начало программы
implicit none ! запрет predefined типов
! вывод строки на экран
print *, "Доброе утро, друзья!"
! конец программы
end program good_morning

```

Рис. 4. Программа на языке fortran, выводящая строку на консоль

Программа из рисунка 4 может откомпилирована командой `gfortran prog.f90 -o prog` в результате выполнения которой будет получен исполняемый модуль `prog`. Если опустить последние два параметра строки, то по умолчанию исполняемый модуль получит наименование `a.out`. При этом других файлов не создается.

Для получения результата первого этапа компиляции программы используем ключ `-S`

```
gfortran -S prog.f90
```

В результате получим файл `prog.s` – результат выполнения первого этапа трансляции программы (см. Рисунок 5).

```

.text
.LC0:
.string "prog.f90"
.align 8
.LC1:
.ascii "\320\224\320\276\320\261\321\200\320\276\320\265 \321\203\321"
.ascii "\202\321\200\320\276, \320\264\321\200\321\203\320\267\321\214"
.ascii "\321\217!"
.text
MAIN__:
pushq %rbp
movq %rsp, %rbp
subq $544, %rsp
movq %fs:40, %rax
movq %rax, -8(%rbp)
xorl %eax, %eax
leaq .LC0(%rip), %rax
movq %rax, -536(%rbp)

```



```

movl $4, -528(%rbp)
movl $128, -544(%rbp)
movl $6, -540(%rbp)
leaq -544(%rbp), %rax
movq %rax, %rdi
call _gfortran_st_write@PLT
leaq -544(%rbp), %rax
movl $36, %edx
leaq .LC1(%rip), %rsi
movq %rax, %rdi
call _gfortran_transfer_character_write@PLT
leaq -544(%rbp), %rax
movq %rax, %rdi
call _gfortran_st_write_done@PLT
nop
movq -8(%rbp), %rax
subq %fs:40, %rax
je .L2
call __stack_chk_fail@PLT
.L2:
leave
ret
.size MAIN__, .-MAIN__
.globl main
main:
pushq %rbp
movq %rsp, %rbp
subq $16, %rsp
movl %edi, -4(%rbp)
movq %rsi, -16(%rbp)
movq -16(%rbp), %rdx
movl -4(%rbp), %eax
movq %rdx, %rsi
movl %eax, %edi
call _gfortran_set_args@PLT
leaq options.1.0(%rip), %rsi
movl $7, %edi

```

```
call _gfortran_set_options@PLT
call MAIN__
movl $0, %eax
leave
ret
```

Рис. 5. Ассемблерный модуль, полученный из программы на языке fortran

Программа в рисунке 5 выглядит великовата, даже после того, как из нее были удалены излишние константы и директивы. Важно иметь в виду, что программа содержит ссылки на функции из стандартной библиотеки языка fortran (имена после команды ассемблера call). Она может быть преобразована в начале в объектный модуль, а потом в исполняемый, при условии, правильно подключения всех обязательных библиотек языка fortran.

Но вернемся к программе из рисунка 4. Для того чтобы получить результат второго этапа компиляции – объектные модуль достаточно откомпилировать программу с ключом -c

```
gfortran -c prog.f90
```

В результате на диске появится файл prog.o, который в последствие может быть преобразован в исполняемый при условии правильного подключения библиотек.

Компилятор gfortran, и это относится ко всем компиляторам коллекции GCC, можно подключать на любом этапе компиляции. Например, указав команду

```
gfortran prog.s -o prog
```

или

```
gfortran prog.o -o prog
```

И мы получим исполняемый модуль. В действительности gfortran в зависимости от расширения исходного файла подключает для дальнейшей компиляции или ассемблер GAS (команда as), а потом компоновщик ld или только компоновщик ld. В действительности для файла prog.s мы можем сами выполнить команду

```
as prog.s -o prog.o
```

и получить файл prog.o. Но вот компилировать ее далее с помощью команды ld, дело весьма затратное, поскольку мы не знаем в общем случае какие библиотеки и с какими ключами нужно указывать в командной строке. По этой причине использование команды

```
gfortran prog.o -o prog
```

 является оптимальным.

Обратимся теперь к компилятору fpc – Free Pascal [53], который не

входит в коллекцию GCC, но который в принципе также позволяет проводить раздельную компиляцию.

```
var a, b, c, f: integer;
begin
  a:=100; b:=200; c:=300;.
  f:=a+b+c;
  writeln(f);
end.
```

Рис. 6. Пример простой программы на языке программирования Pascal

Для компиляции программы из рисунка 6 достаточно выполнить команду

```
fpc prog.pas
```

Особенностью данной команды является то, что будет создан не только исполняемый модуль prog, но и промежуточный объектный модуль prog.o. Можно использовать ключ -a и тогда также будет сгенерирован ассемблерный файл.

```
fpc -a prog.pas
```

не будем приводить его в данном тексте, заметим только, что он, как и в случае с языком fortran имеет ссылки на внешние функции, которые очевидно, должны быть разрешены на стадии компоновки.

Если при компилировании программы на языке Pascal можно указать ключ -Sn и тогда конечный исполняемый модуль создан не будет, процесс остановится после создания объектного модуля. Интересно, что компилятор создает скрипт на языке sh, позволяющий создать исполняемый модуль из полученного объектного модуля. В упрощенном виде он представлен в рисунке 7.

```
#!/bin/sh
ld -b elf64-x86-64 -m elf_x86_64 -s -L. -o prog -T link6288.res -e _start
```

Рис. 7. Скрипт для создания исполняемого модуля из объектного модуля для Free Pascal

В сущности, мы имеем только команду компоновать объектный файл. Ту важна, однако, опция -T. Она позволяет подключать файлы со скриптовыми командами самого компоновщика ld. В нашем случае данный сценарий представлен в рисунке 8. Не сокращаем текст, чтобы внести некоторые пояснения.

```
SEARCH_DIR("/lib64/")
```

```
SEARCH_DIR("/lib/")
SEARCH_DIR("/usr/lib64/")
SEARCH_DIR("/usr/lib/")
SEARCH_DIR("/usr/lib64/gcc/x86_64-alt-linux/10/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/httpd22/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/rtl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/odata/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/pastoids/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-async/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/hermes/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fpmkunit/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/bfd/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-net/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/openssl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/tcl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/gdbm/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-extra/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/googleapi/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/aspell/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/pthreads/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-sdo/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/zlib/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fastcgi/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/pclib/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/bzip2/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/cairo/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/openssl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/xforms/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/regexpr/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libcups/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/ggi/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fpindexer/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/chm/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libxml2/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-image/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/ldap/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libmagic/")
```

```
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/dbus/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/pasjpeg/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libffi/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fppkg/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/symbolic/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/sdl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/gnutls/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libpng/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-web/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/rtl-extra/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/a52/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/ibase/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libvlc/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/odbc/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-process/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/unzip/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/ptc/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/modplug/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-sound/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/rtl-generics/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fftw/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/vcl-compat/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/sqlite/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/uuid/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/x11/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-report/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/proj4/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/iconvenc/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/pcap/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/paszlib/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/lua/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libmicrohttpd/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/jni/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/openssl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/svgalib/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/rsvg/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-registry/")
```

```
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libgc/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-xml/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/openssl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/cdrom/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/utmp/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/webidl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/hash/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-db/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libsee/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/dts/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/mysql/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libtar/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-res/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/newt/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-stl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/rtl-console/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-passrc/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/users/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libenet/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/zorba/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/rtl-unicode/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/gtk2/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-fpcunit/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/dblib/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libfontconfig/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/ncurses/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-js/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/httpd24/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/openssl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/graph/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/syslog/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/oracle/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/oggvorbis/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-json/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/gdbint/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/imagemagick/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fv/")
```

```

SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/numlib/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-base/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libusb/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/rtl-objpas/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/gmp/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/tplylib/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/utls-pas2js/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/mad/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/fcl-pdf/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libcurl/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/libgd/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/postgres/")
SEARCH_DIR("/usr/lib64/fpc/units/x86_64-linux/")
SEARCH_DIR("/usr/lib64/fpc/")
INPUT(
/usr/lib64/fpc/units/x86_64-linux/rtl/si_prc.o
/usr/lib64/fpc/units/x86_64-linux/rtl/abitag.o
prog.o
/usr/lib64/fpc/units/x86_64-linux/rtl/system.o
)
SECTIONS
{
    .fpcdata      :
    {
        KEEP (*(.fpc .fpc.n_version .fpc.n_links))
    }
    .threadvar : { *(.threadvar .threadvar.*.gnu.linkonce.tv.*) }
}
INSERT AFTER .data;

```

Рис. 8. Сценарий выполнения команды ld

Прежде всего в рисунке 8 отметим команды SEARCH_DIR(), которые необходимы, чтобы указать каталоги для поиска библиотек. В нашем случае в библиотеках нет необходимости, так все множество строк с SEARCH_DIR() может быть удалено. Второй важной деталью сценарного файла является команда INPUT(). В частности, с помощью нее наш объектный файл объединяется с объектными файлами из пакета fpc. В

результате такого объединения и создается исполняемый модуль.

Теперь рассмотрим пример, с двумя исходными модулями на языке С (рисунок 9 и 10).

```
// модуль main.c
#include <stdio.h>
extern int add(int, int);
int main(){
    printf("%d\n", add(10, 20));
    return 0;
}
```

Рис. 9. Основной модуль программы

В программе из рисунка 9 вызывается внешняя функция add. Очевидно, что на последнем этапе трансляции (компоновке) эта внешняя ссылка должна быть разрешена.

```
// модуль main1.c
int add(int a, int b){
    return a+b;
}
```

Рис. 10. Модуль содержащий функцию add()

Два программных модуля на языке программирования С транслируются классически по схеме из рисунка 3. Однако выполнив команду

```
gcc main.c main1.c -o main
```

мы получим сразу результат – исполняемый модуль main, результат выполнения которого, естественно, равен 30. Чтобы получить развернутую картину трансляции выполним последовательность команд

```
gcc -S main.c
```

```
gcc -S main1.c
```

```
as main.s -o main.o
```

```
as main1.s -o main1.o
```

```
gcc -o main main.o main1.o
```

В результате кроме исходных модулей main.c и main1.c получим также: main.s, main1.s, main.o, main1.o, main. Т.е. мы получили развернутую картину трехуровневой трансляции много модульной программы. При этом последняя команда не только объединяет объектные модули, но также добавляет объектные модули из пакета языка С и стандартную

библиотеку языка C.

Рассмотрим для примера один из промежуточных модулей на языке ассемблера.

```
.text
.globl add
add:
pushq %rbp
movq %rsp, %rbp
movl %edi, -4(%rbp)
movl %esi, -8(%rbp)
movl -4(%rbp), %edx
movl -8(%rbp), %eax
addl %edx, %eax
popq %rbp
ret
```

Рис.11. Модуль main1.s

Не вдаваясь в несколько громоздкую реализацию ассемблерного модуля (были убраны не нужные константы), обратим внимание на строку `.globl add`, в которой объявляется имя метки (имя функции), которая может быть вызвана из другого модуля.

Мы видим, таким образом, что ассемблер для языков высокого уровня является частью процесса трансляции, ее этапом. Более того, мы вполне можем оптимизировать промежуточный модуль ассемблера, дабы получить нужный эффект в конечном исполняемом модуле.

2.1.2. Ассемблер GAS

Обратимся теперь непосредственно к программированию на языке ассемблера GAS, рассмотрев подробнее простую программу.

```
# prog1.s
.data # секция данных
/*
здесь можно зарезервировать память для использования

в программе
*/
.text # секция кода
```

```

.globl _start
_start:
/*
здесь можно разметить основной код на ассемблере

закончить работу программы
*/
mov $60, %rax # номер системного вызова exit – выход из программы
mov $0, %rdi  # код возврата (0 - выход без ошибок)
syscall      # системный вызов

```

Рис. 12. Простая программа на ассемблере

Программа из рисунка 12 требует подробных разъяснений.

1. Ассемблер GAS поддерживает два вида комментариев. Строковый комментарий, начало которого определяется знаком # и вплоть до конца строки. Кроме того, можно использовать многострочный комментарий в стиле языка C.

2. Обратим внимание, элементы программы, начинающиеся с символа «точка». Это директивы самого транслятора с языка ассемблера. Они влияют на сам процесс трансляции и на структуру будущей программы. В нашей программе директивы .data и .text это директивы, которые выделяют в программе секцию данных и секцию программного кода. В секции данных можно зарезервировать память для использования в программе. Можно это назвать секцией переменных. В нашем случае переменные отсутствуют, поэтому директиву .data можно было бы и опустить. Секция .text содержит программный код.

3. Трансляция осуществляется командами

```

as prog1.s -o prog1.o
ld prog1.o -o prog1

```

В результате получим исполняемый двоичный модель, запустить который на исполнение можно командой ./prog1. С точки зрения схемы трансляции с языка высокого уровня, которую мы подробно разбирали в предыдущем разделе, мы имеем сокращенный вариант трансляции, без первого этапа (см. Рисунок 3). Во всем остальном мы имеем тот же самый подход.

4. Кроме директив, которые начинаются с точки, в нашей программе есть элемент, заканчивающийся двоеточием. Двоеточием заканчиваются метка, которая используется для ссылки в ту или иную точку программы.

После трансляции она становится адресом указанного места программы¹⁵. В нашем случае, `_start` указывает на начало кода, точнее на точку кода, с которой начинает выполняться программа. В нашем случае начало кода и начало выполнения программы совпадает, но в общем случае это не обязательно. Что касается метки `_start`, то она упоминается в еще одной директиве `.globl _start`. Это очень важно. Мы указываем ассемблеру, что имя `_start` должно быть указано в объектном модуле. Компоновщик знает это имя (`_start`) и при создании исполняемого модуля указывает откуда должна начинаться работа программы. В результате наша программа при ее запуске выполняется корректно. При желании, имя метки, которая указывает на начало кода, с которого будет исполняться программа, можно изменить. Изменим `_start`, например, на `main`. Не забудем, что изменять нужно в двух местах программы. Для того, чтобы получить корректный исполняемый модуль команда компоновщика должна быть изменена.

```
ld -e main prog1.o -o prog1
```

т.е. мы указали компоновщику, что выполнение программы начнется с метки `main`.

5. В программе имеются три команды, которые также следует разобрать. Две из них имеют имя `mov`. Имя происходит от английского `move`, т.е. переместить. В ассемблере, в отличие от обычных алгебраических языков, нет присвоения, но есть перенос: из памяти в регистр, из регистра в память, числа в память, числа в регистр. В нашем случае две числовые константы 60 и 0 помещаются в регистры `rax`, `rdi`. Отметим, что числовые константы обязательно начинаются со знака `$`, а перед именем регистра стоит знак `%`. Процессорная команда `syscall` перенаправляет выполнение на ядро операционной системы. Это называется системным вызовом. Номер системного вызова как раз должен располагаться в регистре `rax`¹⁶. Этот номер в данном случае 60. Этот системный вызов заканчивает работу программы, освобождает выделенные для нее ресурсы и передает управление операционной системе. В регистр `rdi` мы можем поместить код возврата, который может интерпретировать скрипт, запустивший этот исполняемый модуль. Получить этот код на консоли можно командой `echo $?`¹⁷.

В секции `.data`, как уже было сказано, можно резервировать память для использования в программе. Можно говорить, что мы там

¹⁵ Значения меток может корректироваться и во время запуска программы на исполнение.

¹⁶ Следует иметь в виду, в 32-битовых системах номера системных вызовов не совпадают с номерами системных вызовов 64-битовых систем. И вызываются системные функции командой `int 0x80`.

¹⁷ Возвращение кода возврата запускающей системе является одним методов взаимодействия процессов, хотя, конечно и ограниченным.

определяем переменные. Для резервирования области памяти для хранения чисел в GAS используются следующие директивы:

- *.byte* – резервирует байт памяти;
- *.word* – резервирует слово памяти (2 байта);
- *.long* – резервирует двойное слово (4 байта) ;
- *.quad* – резервирует 8 байтов памяти.
- *.octa* – резервирование 16 байтов памяти.

Рассмотрим использования одной из подобных директив, несколько видоизменив программу из рисунка 12.

```
# prog2.s
/* секция данных */
.data # секция данных
var1:
.quad 57
/* программная секция */
.text # секция кода
.globl _start
_start:
/* выход из программы */
mov $60, %rax # номер системного вызова exit – выход из программы
mov var1, %rdi # код возврата берем из переменной
syscall      # системный вызов
```

Рис. 13. Пример использования переменной в программах на GAS

В секции *.data* зарезервировано и проинициализировано (присвоено значение 57) 8 байтов памяти (64-битовая переменная). Для того, чтобы обращаться к данной переменной, перед директивой поставлена мета *var1*. В сущности, она играет роль имени переменной. Инструкция

mov var1, %rdi

отправляет 8 байтов, расположенных в переменной *var1*, а регистр *%rdi*¹⁸. Тут важно отметить, что *var1* указывает адрес области памяти, но ассемблер заранее не знает каков размер памяти. Размер памяти определяется указанной выше инструкцией. Справа стоит операнд *%rdi* имеющий размер 64-бита, следовательно, ассемблер реализует команду передачи 8-байтового числа из памяти в регистр. Это важный момент, на который следует обращать внимание. Чтобы сделать подобные команды

¹⁸ Отметим кстати, что команда *mov \$var1, %rdi* отправляет в регистр уже значение *var1*, т. е. адрес, а не значение, расположенное по адресу.

более читаемыми в ассемблере GAS предусмотрены суффиксы, указывающие размерность используемого операнда (области памяти):

- *b* – байт;
- *w* – слово;
- *l* – двойное слово (4 байта);
- *q* – 8-байтовый операнд.

т. е. В зависимости от контекста можно использовать команды `movb`, `movw`, `movl`, `movq`. Тут важно лишний раз подчеркнуть размерность операнда, если зарезервированная область никак на это не указывает. В этом случае суффикс и размер регистра, куда отправляются данные, дают такую гарантию. Однако есть команды, в которых определенность в вопросе о том, над каким операндом производится действие, должна быть обязательно выражена командой. Примером может служить команда `inc num` (инкремент – увеличение числа на 1), где `num` указывает на некоторую область памяти. В этом случае ассемблеру не откуда взять информацию о том, какой размер операнда предполагается использовать в команде. Поэтому, при компиляции ассемблер выдаст предупреждение. Предупреждение, конечно, не останавливает процесс трансляции, но создает элемент неопределенности, который в конечном итоге может привести к программной ошибке. Поэтому в команде необходимо точно указать размер операнда. Например, `incq num`, `incl num` и т.п.

Вернемся снова к программе из рисунка 12. Откомпилируем программу

```
as prog2.s -o prog.o
ld prog2.o -o prog2
Выполним команду
./prog2
а затем
echo $?
```

в результате на консоль будет выведено число 57.

Описанные выше директивы, с помощью которых можно резервировать память имеют дополнительные возможности. Вместо одного числа, можно задать целый массив чисел

```
arr:
.quad 45, 3, 78, 2347, 1, 0, 88
```

Метка `arr`, таким образом, указывает на первый элемент массива. Особенности индексирования такого массивы буду разобраны нами ниже. Для резервирования памяти в арсенале GAS есть и другие директивы. Директива `.skip N` резервирует `N` байтов, заполняя их нулями. Если

заполнитель требуется другой, то можно указать `.skip 100, 1` – резервируем 100 байтов и заполняем их единицами. Синонимом директивы `.skip` является директива `.space`. Есть и третий синоним для данных директив это `.zero`. Впрочем, запись типа `zero 1000, 48` читается несколько странно, хотя и вполне корректна с точки зрения синтаксиса.

Для резервирования последовательности символов (строк) можно использовать директиву `.ascii`, которой можно зарезервировать память для нескольких строк. Следующая строка

```
.ascii "qwerty", "zxcvbn", "asdfg"
```

резервирует память для трех строк, идущих друг за другом.

Директива `.asciz` аналогична, ставим код равный нулю после каждой строки. Если запятых между строками нет, то они объединяются в одну и в конце ставится нулевой код.

Транслятор GAS позволяет создавать переменные времени трансляции, изменять их (ниже по тексту), использовать их в командах процессора в качестве констант, обязательно указывая перед именем знак \$, если не предполагается, что это адрес и речь идет о данном по этому адресу. Еще одним удобным макросредством ассемблер является точка, которая равна текущему адресу. Следующий фрагмент показывает удобство описанных выше средств

```
msg:
.ascii "Это строка"
len = . - msg
```

Переменной времени трансляции присваивается значение, равное длине строки.

2.1.3. Об основах программирования на платформе x86-64 на ассемблере GAS

Подробное описание всех групп команд процессора x86-64 будет дано в разделе «Система команд x86-64». В данном разделе мы остановимся на некоторых важных особенностях команд, связанных непосредственно с возможностями языка в плане программирования.

2.1.3.1. Адресация

В языках высокого уровня адресация скрыта за именами переменных и функций, и в большинстве случаев никак себя не проявляет. Не так в языке ассемблера. Здесь программа оперирует непосредственно памятью

(оперативной или регистровой), поэтому понятие адреса того или иного данного выступает на первое место. Проще всего обстоит дело с регистровой памятью. В качестве адресов данных здесь используется название регистров. Например:

```
mov %rax, %rbx
```

или

```
mov %si, %di
```

И здесь понятно, что в первом случае данное из регистра `rax` передается в регистр `rbx`, а во втором случае данное из регистра `si` передается в регистр `di`. Сложнее и многообразнее обстоит дело, когда один из операндов относится к оперативной памяти. Рассмотрим все возможные случаи.

1. Прямая адресация. Пример прямой адресации был приведен выше. Операция над данным осуществляется непосредственно по адресу данного. Например, `mov %rax, num` – 8-байтовое число помещается по адресу, на который указывает метка `num`. Или `incb number` – осуществляется инкремент однобайтового данного, которое указывает метка `number`. Адресация может указываться со смещением. Например, `movb 11+1, %al` – байт со смещением в 1 относительно адреса 11 отправляется в регистр `al`.

2. Косвенная адресаций. 1-й вариант. Предварительно необходимо получить адрес данного.

```
mov $11, %rdi
```

```
mov (%rdi), %eax
```

Адрес данного, на который указывает 11 отправляется в регистр `rdi`, а потом с помощью этого регистра данное извлекается и помещается в регистр `eax`. При этом из самой команды понятно, что данное имеет размер 4 байта.

3. Косвенная адресация. 2-й вариант. Указывая регистр, в котором хранится адрес данного, можно указывать и смещение. Например, `16(%rbx), %rdi` – к адресу, который хранится в `rbx` добавляется число 16 и данного берется из результирующего адреса. При этом смещение может быть и отрицательным, например, `-8(%rsi), %al`.

4. Косвенная адресация. 3-й вариант. Аналогично предыдущему, но смещение хранится в другом регистре. Например, `mov (%rax, %rdx), %r8` – адрес хранится в `rax`, смещение в `rdx`.

5. Косвенная адресация. 4-й вариант. Является комбинацией двух предыдущих случаев. Например, `mov %al, 8(%rdx, %rax)` – базовый адрес в `rdx`, смещение в `rax`, второе смещение равно 8.

6. Косвенная адресация. Вариант с масштабированием. Пример `mov`

(%rax, %rsi, 8), %rbx – результирующий адрес получается прибавлением к базовому адресу в rax значение в rsi, умноженное на 8 (масштабирование).

7. Способ получения адреса с помощью команды lea. Например, lea num(%rip), %rdi эквивалентна mov \$num, %rdi.

2.1.3.2. Условные и безусловные переходы

В основе программирования на любом алгоритмическом языке программирования являются ветвления и циклы. В основе организации данных программных структур на ассемблере являются команды условного и безусловного переходов. В определенном смысле это очень похоже на то, как это устроено в языках C и Pascal. Там есть проверка условия и оператор безусловного перехода. Проверка условия для языка ассемблера сводится к двум этапам:

1. Выполнения какого-либо действия (арифметического или битового), которое непосредственно влияет на некоторые биты регистра флагов.

2. Содержимое регистров флагов в свою очередь влияет на выполнение команд условного переходы – будет ли произведен переход или программа продолжит выполнение с команды, следующей за командой перехода.

Переходы могут осуществлять вперед или назад. Рассмотрим, например, две схемы с командами безусловного перехода.

фрагмент с переходом вперед

```
...  
jmp _ahead  
...  
_ahead:  
...
```

Рис.14. Фрагмент с переходом вперед

фрагмент с переходом назад (бесконечный цикл)

```
...  
_back:  
...  
jmp _back  
...
```

Рис.15. Фрагмент с переходом назад

Представленные выше фрагменты с безусловными переходами, очевидно, демонстрируют необходимость использования и условных переходов.

Поскольку ниже будет дан полный справочник команд x86-64, наша задача здесь показать, как работает сама схема с использованием условных и безусловных конструкций.

```
# prog3.s
.data # секция данных
num:
.quad 10
.text # секция кода
.globl _start
_start:
mov $0, %rdi
# 10 раз по 2
l1:
add $2, %rdi
decq num
jnz l1
mov $60,%rax # номер системного вызова exit – выход из программы
syscall      # системный вызов
```

Рис. 16. Пример циклической конструкции на языке ассемблера

Выполняем

as prog3.s -o prog3.o

ld prog3.o -o prog3

./prog3

echo \$?

Результат выполнения 20

В программе из рисунка 16 реализован цикл с проверкой условия в конце тела цикла (пост условие). Команда `jnz` осуществляет переход на указанную метку, если последняя арифметическая команда дает не нулевой результат. Попробуем реализовать теперь цикл с предусловием с тем же результатом.

```

# prog4.s
.data # секция данных
num:
.quad 10
.text # секция кода
.globl _start
_start:
mov $2, %rdi
# 10 раз по 2
l1:
decq num
jz l2
add $2, %rdi
jmp l1
l2:
mov $60,%rax # номер системного вызова exit – выход из программы
syscall      # системный вызов
Выполняем
as prog3.s -o prog3.o
ld prog3.o -o prog3
./prog3
echo $?

```

Рис. 17. Пример цикла с предусловием

Результат выполнения 20

В данном случае (рисунок 17) мы использовали инструкцию условного перехода `jz`. Осуществляется переход, если предыдущая команда дает 0.

Рисунки 16 и 17 показывают, как можно реализовать цикл на основе условных конструкций. Есть еще одна инструкция, которая непосредственно предназначена непосредственно для реализации цикла. Эта инструкция `loop`. Это тоже переход по указанной метке, но при этом осуществляется еще декремент содержимого `rcx`. Переход осуществляется пока содержимое не равно 0.

```

# prog5.s
.data # секция данных
num:

```

```

.quad 10
.text # секция кода
.globl _start
_start:
mov $0, %rdi
mov num, %rcx
# 10 раз по 2
l1:
add $2, %rdi
loop l1
mov $60,%rax # номер системного вызова exit – выход из программы
syscall      # системный вызов

```

Рис. 18. Реализация цикла с помощью инструкции loop

Конечно, сам цикл стал короче и в многих случаях циклы с пост условием удобно реализовывать с помощью инструкции loop.

Условные переходы основываются на результатах предыдущих операций. Например, в предыдущих примерах мы использовали нулевой результат, как признак того, что следует заканчивать выполнение цикла. Проверка осуществлялась командами jz и jnz. Если мы осуществляем операцию sub %rax, %rbx, т.е. вычитаем содержимое регистра rax из содержимого регистра rbx и результат помещаем rbx. Если результат равен нулю, то можно сделать вывод, что числа были равны друг другу. Но, заметим, что в результате операции мы разрушили содержимое регистра rbx. Не всегда это удобно. Чтобы не менять содержимое регистров, при этом для флагов eflags получить тот же эффект, как если бы мы произвели вычитание, используется команда cmp. По результатам воздействия данной команды на содержимое регистров флагов, можно судить не только о равенстве чисел, но и какое из них больше, в том числе с учетом знаков чисел (считаем ли мы числа знаковыми или беззнаковыми).

Конечно, условные конструкции интересны не только для реализации циклических алгоритмов, но и, собственно, для проверки всевозможного типа условий. Рассмотрим, как на ассемблере реализуются полные и не полные условные конструкции, знакомые из курса программирования на языках высокого уровня.

Схема полной условной конструкции на языке C выглядит так:

```

if(a>b){
    m=a

```

```

}else{
m=b
}
//максимальное число в m

```

На ассемблере подобная же конструкция будет выглядеть так¹⁹

```

cmp %rcx, %rbx # сравнение чисел в rcx и rbx
jb l1
mov %rbx, %rax
jmp l2
l1:
mov %rcx, %rax
l2:
# максимальное число в rax

```

Следует только отметить, что данную команду условного перехода используют при сравнении беззнаковых чисел. `jb` – перейти, если первый операнд больше.

Не полная условна конструкция на языке C имеет структуру

```

m=b
if(a>b){
m=a
}
//максимальное число в m

```

Аналогичная структура на языке ассемблера

```

mov %rbx, %rax
cmp %rcx, %rbx # сравнение чисел в rcx и rbx
ja l1
mov %rcx, %rax
l1:
# максимальное число в rax

```

Здесь мы использовали другую команду процессора `ja` – перейти если, второй операнд больше.

Следует также остановиться на безусловных переходах `jmp`. Существует пять способов использования безусловного перехода:

1. Переход на указанную метку: `jmp l1`, где `l1` – метка.
2. Переход на адрес, указанный в регистре: `jmp %rax` – адрес в `rax`.
3. Переход на косвенный адрес: `jmp *(%rdx)` – адрес берется из памяти, на которую указывает регистр `rdx`.
4. Переход на адрес, который хранится в памяти: `jmp *l1` – адрес

¹⁹ Для простоты полагаем, что сравниваемые числа не равны друг другу.

берется из памяти, на которую указывает 11.

5. Можно также указывать относительный адрес: `jmp .+2` – переход к команде, которая следует за `jmp`.

2.1.3.3. Системные вызовы

При написании программ на языке ассемблера у нас есть две возможности доступа к ресурсам компьютерной системы:

1. Использовать библиотеки языков высокого уровня (статические и динамические). Этот вопрос будет нами подробно разобран ниже.

2. Использовать возможности самой операционной системы. Интерфейс с операционной системой (ее ядром) обеспечивает команда `syscall`, которую мы уже использовали выше, для прекращения работы программы.

С помощью команды `syscall` вызывается одна из системных функций (список системных функций см. Приложение 1). Каждая системная функция имеет свой номер. Выше для выхода из программы мы использовали системную функцию с номером 60, имеющую название `sys_exit` (см. Приложение 1). Перед вызовом `syscall` регистры должны содержать номер функции. Параметры, необходимые для правильного выполнения системной функции, должны содержаться в регистрах процессора:

- параметр 1 – `rdi`;
- параметр 2 – `rsi`;
- параметр 3 – `rdx`;
- параметр 4 – `r10`;
- параметр 5 – `r8`;
- параметр 6 – `r9`.

В случае выхода из программы (системный вызов с номером 60) мы задействовали только один параметр – регистр `rdi`, поместив туда код выхода.

Ниже представлен пример программы, осуществляющей ввод и вывод со стандартного устройства. Программа ждет ввода строки и после ввода (Enter) выводит ее на стандартное устройство вывода.

```

# prog6.s
.data # секция данных
buf:
# резервируем область памяти.
.space 200, 0
.text # секция кода
.globl _start
_start:
# прочитать строку со стандартного входа
mov $0, %rdi # stdin - стандартный ввод (клавиатура)
mov $buf, %rsi # адрес буфера
mov $200, %rdx # длина буфера
mov $0, %rax # номер системной функции чтения со стандартного
устройства
syscall # системный вызов
# количество введенных символов помещается в rax
# печать строки на стандартное устройство
mov $1, %rdi # stdout - стандартный вывод (экран)
mov $buf, %rsi # адрес буфера
mov %rax, %rdx # количество выводимых символов
mov $1, %rax # номер системной функции вывода на стандартное
устройство
syscall # системный вызов
ex:
# закончить работу программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок), равносильно mov $0,
%rdi
syscall # системный вызов

```

Рис. 19. Пример использования системных функций ввода-вывода

Дадим подробный разбор программы из рисунка 19.

1. В секции .data с помощью директивы .space резервируется область памяти, определенного размера (200 байтов), заполненная байтами с нулевыми значениями (второй параметр).

2. Для ввода строки используется системная функция с номером 0

(sys_read). Параметры для системной функции передаются по указанной выше схеме через регистры rdi – номер стандартного ввода (0), rsi – адрес области памяти, куда будет скопирована введенная строка, rdx – длина буфера (максимальная длина вводимой строки). Если введенная строка будет длиннее указанного в rdi значения, то она обрезается. Тут следует отметить, что речь идет о длине в байтах, а не в символах, что логично, так как стандартная кодировка в Linux - utf-8. Функции при успешном вводе возвращает в регистре rax длину введенной строки в байтах – это важно, если мы хотим что-то с ней впоследствии делать. Если выполнение данной функции привело к ошибке, то функция возвращает отрицательное значение. Следует иметь в виду, что вводимая строка, попадает в буфер вместе с кодом перевода строки – 10. Байт перевода строки учитывается в возвращаемой длине.

3. Для вывода полученный строки используем функцию с номером 1 (sys_write). Функция также имеет три параметра: rdi – номер стандартного вывода (1), rsi – адрес буфера, где хранится введенная ранее строка, rdx – длина выводимой строки (для этого выполняем команду `mov %rax, %rdx`).

Для трансляции программы выполняем команды

```
as prog6.s -o prog6.o
```

```
ld prog6.o -o prog6
```

Выполнение

```
./prog6
```

Поскольку для ввода вывода мы используем стандартные устройства, то можно осуществлять перенаправление, например

```
./prog6 < file1.txt > file2.txt
```

На рисунке 20 представлен пример посимвольного вывода.

```

# prog7.s
.data # секция данных
buf:
.space 200, 0
.text # секция кода
.globl _start
_start:
# прочитать строку
mov $buf, %rsi # адрес буфера
mov $0, %rdi # stdin
mov $200, %rdx # длина буфера
mov $0, %rax # номер системной функции чтения
syscall # системный вызов
# по символный вывод строки на стандартное устройство
mov %rax, %r8 # в регистре будет длина буфера
mov $buf, %rsi # начало буфера в регистр rsi
loo:
# адрес выводимого байта в rsi
mov $1, %rdi # stdout
mov $1, %rdx # выводим один символ (байт)
mov $1, %rax # номер системной функции вывода
syscall # системный вызов
inc %rsi # переходим к следующему символу
dec %r8 # получить длину оставшейся части буфера
jnz loo # продолжать?
ex:
# закончить работу программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов

```

Рис. 20. Пример посимвольного вывода на стандартное устройство

В программе на рисунке 20 ввод осуществляется тем методом, что и в программе на рисунке 19. Но вывод происходит по одному символу, точнее по одному байту. Системная функция вывода, однако, осуществляет ввод с учетом кодировки utf-8, т.е., по сути, сам вывод на экран происходит

по одному символу, даже в случае символов, код которых имеет размер более одного байта.

Для трансляции программы выполняем команды

```
as prog7.s -o prog7.o
```

```
ld prog7.o -o prog7
```

Запуск

```
./prog7
```

При использовании системного вызова нужно помнить одну важную вещь. При вызове гарантированно не сохраняются два регистра `gsx` – здесь хранится адрес возврата, `r11` – в нем хранится регистр флагов. Кроме того, в регистрах `gsx` возвращается код возврата. Т.е. если вы своей программе держите в регистрах `gsx`, `r11` какие-либо важные величины, то перед системным вызовом их следует сохранить. Регистр же `gsx` вы все равно используете для указания номера системной функции, но после вызова значение `gsx` меняется.

2.2. Стек и функции

2.2.1. Структура стека

Стек является важнейшей структурой, реализованной на уровне процессора и используемый во всех языках высокого уровня. Стек позволяет реализовать функции – важнейший элемент программирования. Для управления этой структурой, которую можно также назвать особым видом памяти, имеется набор инструментов. При запуске исполняемого файла операционная система выделяет ему память и стек. Как работает стек x86-64. Вот несколько важных положений:

1. Стек состоит из элементов равной длины. В нашем случае это элементы длиной 8 байтов.

2. Есть две команды для работы со стеком: `push` и `pop`. `push` – положить в стек, `pop` – взять из стека.

3. У процессора есть специальный регистр `rsp` который всегда указывает на последний положенный в стек элемент. Не рекомендуется изменять содержимое этого регистра напрямую, а только через стековые команды.

4. При выполнении команды `push` содержимое регистра `rsp` уменьшается на 8 и по новому адресу кладется операнд команды `push`. Схема функционирования стека представлена на рисунке 8.

5. При выполнении команды `pop` операнд этой команды получает

значение по адресу, который хранится в регистре `rsp`, затем содержимое `rsp` увеличивается на 8.

6. Таким образом одна из функций стека – это хранение данных. Принцип доступа к данным стека можно выразить краткой формулой: «первым пришел, последним ушел».

7. Есть еще две команды, непосредственно обращающиеся к стеку: `call` и `ret`. С помощью этих команд можно организовывать такие структуры программы, как функции. В следующем параграфе мы подробно разберем как это работает.

8. Конечно, доступ к данным можно осуществлять и непосредственно используя регистр `rsp`, что очень часто используют современные компиляторы.

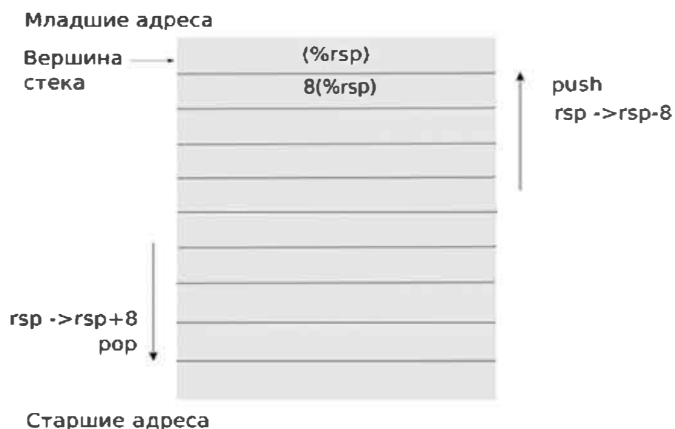


Рис. 21. Схема работы стека при выполнении команд `push` и `pop`

На рисунке 21 представлена схема работы стека при выполнении команд `push` и `pop`. Важно отметить, что стек растет в сторону младших адресов. Адрес вершины стека в `rsp`, его содержимое (`rsp`). По сути, команда `push %rbx` (например) сводится к двум командам: `sub 8, %rsp/mov %rbx, (%rsp)`.

На рисунке 22 показан пример работы с содержимым стека.

```
# prog8.s
.data # сегмент данных
msg:
.ascii "Это стек, детка!\n"
len = . - msg
.text # сегмент кода
```

```

.globl _start
_start:
mov $123, %rax # число, как индикатор
push %rax # положили в стек
mov $msg, %rax # адрес строки
push %rax # положили в стек
cmpq $123, 8(%rsp) # проверяем число в стеке
jnz ex # если наш расчет неверен, то заканчиваем
# напечатать строку
mov $1, %rdi # stdout
mov (%rsp), %rsi # адрес сообщения
mov $len, %rdx # длина сообщения
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# закончить работу программы
ex:
pop %rax # восстанавливаем стек
pop %rax # восстанавливаем стек
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов

```

Рис. 22. Пример разных способов доступа к данным в стеке

Для компиляции программы выполняются команды

```
as prog8.s -o prog8.o
```

```
ld prog8.o -o prog8
```

Прокомментируем программу из рисунка 22.

В программе показано два способа доступа к стеку: 1. Традиционный подход `push/pop`. 2. Непосредственное обращение к элементам стека посредством регистра `rsp` (`cmpq $123, 8(%rsp)`). Обратим внимание, что символ `\n` интерпретируется системной функцией вывода на стандартное устройство, как следовало бы предположить, как символ перевода строки.

2.2.2. Вызов функций

Стек используется для реализации функций на языке ассемблера. Команда вызова функции `call fanc1`, где `fanc1` обычная метка. При вызове

функции в стек кладется адрес следующей за вызовом команды. Чтобы возвратиться из функции используется команда `ret`. Команда извлекает из стека адрес и осуществляет переход на него.

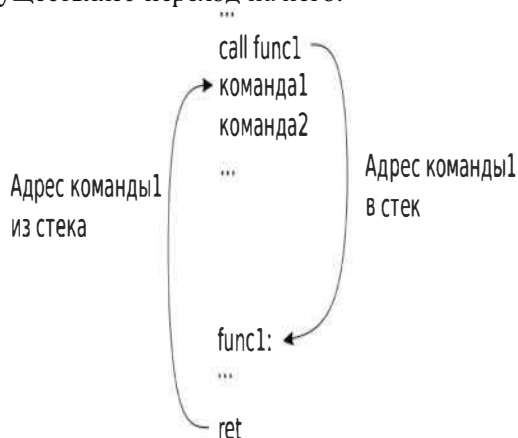


Рис. 23. Схема вызова функции на языке ассемблера

На рисунке 23 представлена схема работы команды `call` и возврата из нее.

```
# prog9.s
.data # секция данных
msg:
.ascii "Строка будет напечатана из функции\n"
len = . - msg
.text # сегмент кода
.globl _start
_start:
call print # вызов функции
# вызов функции
call print # вызов функции
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# процедура печати строки
print:
# печатаем строку
mov $1, %rdi # stdout
```

```

mov $msg, %rsi # адрес сообщения
mov $len, %rdx # длина сообщения
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
ret # возврат из функции

```

Рис. 24. Пример простой функции, выводящей на консоль строку

Для компиляции программы выполняются команды

```

as prog9.s -o prog9.o
ld prog9.o -o prog9

```

Программа на рисунке 24 очень проста по структуре. В ней демонстрируется пример простой функции (метка `print`) на языке ассемблера. Эта функция выводит на консоль строку с помощью с уже неоднократно используемой нами системной функции. Тут следует только отметить одну особенность, которая для ассемблера даже и незаметна. Дело в том, что данные, которые определены в секции `.data`, для функции следовало бы считать глобальной переменной. Передачу параметров будем рассматривать далее. Одним из способов передачи параметров как раз и является использование для этой цели стека.

Поскольку возврат из функции представляет собой два действия: взять значение из стека, перейти по полученному адресу, то соответственно команду `ret` можно заменить двумя командами: `pop/jmp`, тогда функция на рисунке 24 будет выглядеть (см. рисунок 25).

```

print:
# печатаем строку
mov $1, %rdi # stdout
mov $msg, %rsi # адрес сообщения
mov $len, %rdx # длина сообщения
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
pop %rdx
jmp *%rdx

```

Рис. 25. Замена команды `ret` на `pop/jmp`

Аналогично команда `call` может быть заменена последовательностью

```

push $ll
jmp print
ll:

```

Важным элементом использования функций на ассемблере является то, что в функции могут использовать регистры. Если содержимое регистров будет изменено, то это может повлиять на дальнейшее выполнение программы. Обычной схемой при этом является сохранение всех регистров, которые используются в функции, и восстановление их перед выходом из функции. Следует также помнить, что если в функции используются системные вызовы, то при этом изменяются регистры `rcx` и `r11`.

Пример такой функции представлено на рисунке 26.

```
# prog10.s
.data # сегмент данных
msg:
.ascii "Строка печатается в функции"
len = . - msg
.text # сегмент кода
.globl _start
_start:
call print      # вызов функции
# вызов функции
call print # вызов функции
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# процедура печати строки
print:
push %rcx
push %r11
push %rdi
push %rsi
push %rdx
# печатаем строку
mov $1, %rdi # stdout
mov $msg, %rsi # адрес сообщения
mov $len, %rdx # длина сообщения
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
```

```
pop %rdx
pop %rsi
pop %rdi
pop %r11
pop %rcx
mov $0, %rax
ret
```

Рис. 26. Пример простой функции, выводящей на консоль строку, с сохранением регистров

Для компиляции программы выполняются команды

```
as prog10.s -o prog10.o
```

```
ld prog10.o -o prog10
```

В стандарте x86-64 принято возвращать данные из функции через регистры `%rax` (см. рисунок 25). Конечно, если вы используете функции в пределах своей собственной программы, то способ возвращения данных можно выбирать по своему усмотрению.

2.2.3. Передача параметров в функцию и локальные переменные

Ценностью функций является возможность регулировать их выполнение путем использования параметров. Параметризация позволяет заменить вызовом одной функции не только идентичные фрагменты текста программы, но похожие фрагменты. В языках высокого уровня программист в большинстве случаев никак не может регулировать способ, который используется для передачи параметров в функцию. В сущности, обычно его это даже не интересует. При программировании же на ассемблере можно выбирать любой способ, в том числе на основе собственных соображений.

Старый классический способ передачи параметров в функции основывается на использовании стека. Параметры при помощи команд `push` помещаются в стек перед вызовом функции (см. Рисунок 6). После того, как осуществился возврат из функции, стек восстанавливается с помощью команд `pop`, или путем команды `add $N, %rsp`, где `N` количество байтов (кратных 8), используемых в стеке для хранения параметров.

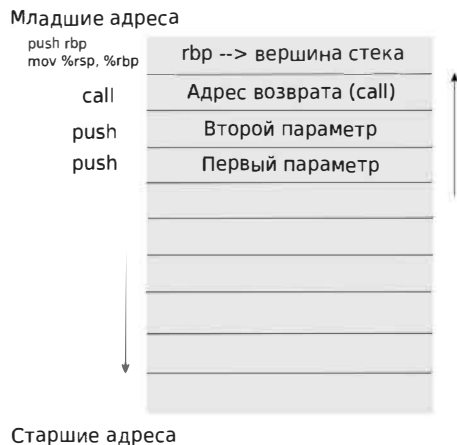


Рис. 27. Вызов функции с передачей параметров

Доступ к параметрам внутри функции осуществляется следующим образом. В начале функции значение, которое лежит в `rsp`, помещается в другой регистр. Обычно для этой цели используется регистр `rbp`. И далее доступ к параметрам осуществляется через регистр `rbp`, при этом изменения регистра `rsp` уже никак не влияет на положение параметров в стеке относительно адреса, который хранится в `rbp`. Примером такого подхода служит функция на рисунке 28.

```
# prog11.s
.data # сектор данных
msg1:
.ascii "Строка печатается в функции\n"
len1 = . - msg1
.text # секция кода
.globl _start
_start:
push $msg1
push $len1
call print # вызов функции
#восстановить стек
pop %rax
pop %rax
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
```



```

syscall # системный вызов
# процедура печати строки
print:
push %rbp # сохранить старое значение регистра
mov %rsp, %rbp # rbp указывает на вершину стека на данный момент
push %rcx
push %r11
push %rdi
push %rsi
push %rdx
# печатаем строку
mov $1, %rdi # stdout
mov 24(%rbp), %rsi # адрес сообщения
mov 16(%rbp), %rdx # длина сообщения
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
pop %rdx
pop %rsi
pop %rdi
pop %r11
pop %rcx
pop %rbp
mov $0, %rax # функция возвращает 0 - функция выполнена успешно
ret

```

Рис. 28. Пример простой функции с передачей параметров через стек

Для компиляции программы выполняются команды

```
as prog11.s -o prog11.o
```

```
ld prog11.o -o prog11
```

В функции `print` на рисунке 28 в начале стоит команда `push %rbp`. Поэтому расположение параметров в стеке относительно значения в `rbp` будет смещено соответственно на 16 и 24 байта (см. Рисунок 6). В чем собственно удобство стека в нашем случае. Стек растет в сторону младших адресов, что никак не влияет на значения параметров, которые лежат в старших адресах. По выходу из функции стек восстанавливается двумя командами `pop`. Но сделать это можно и командой `add $16, %rsp`.

Конечно, порядок передачи параметров может быть разный. Так

кстати и было 32-битовых системах x86. В языках программирования Pascal и C порядок был противоположный, и это приходилось учитывать при интегрировании объектных модулей из другого языка.

В структуре стека функции можно предусмотреть и место для локальных переменных. Для этого достаточно выполнить действие `sub $N, %rsp`, в результате чего будет выделено дополнительное место в стеке, куда можно поместить какие-то локальные данные. Программа из рисунка 29 демонстрирует возможности локальных переменных. В частности, в них можно хранить значения регистров. Кроме того, локальные переменные незаменимы при написания рекурсивных алгоритмов.

```
# prog12.s
.data # сегмент данных
msg1:
.ascii "Строка печатается в функции\n"
len1 = . - msg1
.text # сегмент кода
.globl _start
_start:
# параметры функции
push $msg1
push $len1
call print # вызов функции
#восстановить стек
pop %rax
pop %rax
# выход
mov $60, %rax # номер системного вызова exit
mov %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# процедура печати строки
print:
push %rbp # сохранить старое значение регистра
mov %rsp, %rbp # rbp указывает на вершину стека на данный момент
subq $48, %rsp # выделить для локальных переменных
# сохранить регистры в локальных переменных
mov %rcx, -8(%rbp)
mov %r11, -16(%rbp)
```

```

mov %rdi, -24(%rbp)
mov %rsi, -32(%rbp)
mov %rdx, -40(%rbp)
# печатаем строку
mov $1, %rdi # stdout
mov 24(%rbp), %rsi # адрес сообщения
mov 16(%rbp), %rdx # длина сообщения
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстановить регистры из локальных переменных
mov -8(%rbp), %rcx
mov -16(%rbp), %r11
mov -24(%rbp), %rdi
mov -32(%rbp), %rsi
mov -40(%rbp), %rdx
# восстановить регистр стека
mov %rbp, %rsp
# теперь можно восстановить начальное значение rbp
pop %rbp
mov $0, %rax # функция возвращает 0 - функция выполнена успешно
ret

```

Рис. 29. Пример использования в функции локальных переменных

Для компиляции программы выполняются команды

```
as prog12.s -o prog12.o
```

```
ld prog12.o -o prog12
```

Передача параметров через стек была характерна для x86 систем, но и в них были другие протоколы передачи, например через разные наборы регистров. Такой разноречивой в протоколах передачи создавал серьезные проблемы в использовании библиотек. В 64-битовых системах архитектуры x86-64 произошел переход к соглашению о передаче параметров посредством регистров. Такой тип передачи параметров обычно называют быстрым. Это произошло как в Linux, так и в Windows. Но протоколы передачи параметров оказались разными. Для передачи параметров в 64-х битовых операционных системах Linux последовательно используют шесть регистров: rdi, rsi, rdx, rcx, r8, r9. Если количество параметров превышает шесть, то оставшиеся параметры передаются через

стек, так как мы это делали выше²⁰. Этому подходу придерживаются разработчики операционных систем и компиляторов. И если предполагается интегрировать программу с другими программами и библиотеками, то вам придется придерживаться этого протокола. В противном случае ассемблер разрешает вам использовать любой подход для работы с функциями.



Рис. 30. Структура стека при вызове функции в Linux x86-64

На рисунке 30 показана схема стека при вызове функций в Linux для x86-64. Главной особенностью подобной структуры стека является то, что в случае необходимости (только если необходимо) хранения параметров в стеке (параметров, передаваемых через регистры), стек резервируется непосредственно в вызванной функции (это называется красная зона - red zone). Красная зона располагается в младших по отношению к указателю стека адресах. Размер красной зоны составляет 128 байтов. Смысл красной зоны заключается в том, что эта область не должна затираться какими-либо прерываниями, обрабатываемыми в момент выполнения кода процедуры. Но поскольку при вызове следующей функции эта область фактически затирается, использовать ее есть смысл только в последней функции из

²⁰ Вообще большое количество параметров в функции является, на наш взгляд, своего рода дурновкусием. Большое количество параметров, если это так уж необходимо, следует на наш взгляд заменять указателем на структуру, содержащую всю необходимую информацию.

цепочки вызываемых функций. Красная зона может быть использована для хранения на время выполнения функции.

При этом предлагается следовать следующим принципам:

1. Стек, резервируемый до вызова функции, освобождается после того, как управление передается в вызывающий модуль.
2. Стек, резервируемый в вызывающей функции, освобождается в ней же.

Конечно, и мы это уже подчеркивали, если программу на ассемблере не предполагается интегрировать в другие программные системы, программист на ассемблере волен использовать любой удобный ему способ формирования стека функции, передачу параметров²¹.

На рисунке 31 представлен пример регистровой передачи параметров в функцию. Как и положено первый параметр помещается в регистр `rdi`, второй в регистр `rsi`.

```
# prog13.s
.data # сектор данных
msg1:
.ascii "Строка из функции \n"
len1 = . - msg1
.text # сектор кода
.globl _start
_start:
# поместим параметры согласно соглашению о вызове
mov $msg1, %rdi # адрес строки (первый параметр)
mov $len1, %rsi # длина строки (второй параметр)
call print # вызов процедуры
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# процедура печати строки
print:
# сохраним, используемые в функции регистры
push %rdx
push %rcx
```

²¹ Конечно, некоторые приверженцы строгих правил в программировании могут меня осудить за такие легкомысленные утверждения, но не бывает правил без исключений.

```

push %r11
# печатаем строку
mov %rsi, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес строки
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret

```

Рис. 31. Пример регистровой передачи x86-64 Linux параметров

Для компиляции программы выполняются команды

```
as prog13.s -o prog13.o
```

```
ld prog13.o -o prog13
```

2.2.4. Многомодульное программирование на ассемблере GAS

Современному программисту трудно представить себе программу, представляющую один большой файл, в котором, скажем десятки тысяч строк. С точки зрения разработки, отладки, модернизации – это огромная головная боль даже если текст программы хорошо прокомментирован. Современные программные проекты всегда состоят из большого количества файлов, которые собираются во время трансляции или во время исполнения. Поскольку процесс трансляции с языка ассемблера GAS происходит в два этапа, то разумно предположить возможность двух видов многомодульности программы (см. Рисунок 3 и комментарии к нему). Заметим также, что ассемблер GAS является однократным (в отличие, например, от FASM), так что возможность использовать разные модули, на разных подэтапах (проходах) естественно отсутствует.

Для создания многомодульной программы, собираемой на первом этапе трансляции на понадобится директива `.include`. В сущности, больше ничего и не нужно. В этой директиве указывается имя файла, содержащего часть кода. Когда ассемблер встречает директиву `.include` он подгружает нужный файл, где содержится фрагмент программы и продолжает

обработку с начало вставленного фрагмента, ну а далее переходит к основному тексту программы. Если в добавляемом фрагменте имеются также директивы `.include`, то процесс подгрузки файлов продолжается. Т.е. процесс может идти рекурсивно. В сущности, в результате такой обработки ассемблер компилирует один большой текст программы. Для программистов, однако, это может быть большим подспорьем, поскольку, позволяет, в частности, вести коллективную разработку.

В качестве пример взята программа из рисунка 31 и разбита на модули, которые потом собираются ассемблером.

```
# prog14.s
.include "data.inc"
.text # сектор кода
.globl _start
_start:
.include "main.inc"
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
.include "functions.inc"
Файл data.inc.
.data # сектор данных
msg1:
.ascii "Строка из функции \n"
len1 = . - msg1
Файл main.inc.
# поместим параметры согласно соглашению о вызове
mov $msg1, %rdi # адрес строки (первый параметр)
mov $len1, %rsi # длина строки (второй параметр)
call print # вызов процедуры
Файл functions.inc
# процедура печати строки
print:
# сохраним, используемые в функции регистры
push %rdx
push %rcx
push %r11
```

```

# печатаем строку
mov %rsi, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес строки
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret

```

Рис. 32. Основной модуль программы (prog14.s)

Для компиляции программы используем те же самые команды, что и ранее

```

as prog14.s -o prog14.o
ld prog14.o -o prog14

```

Таким образом многомодульность на первом этапе не вызывает какой-либо сложности. Действительно, мы просто разбиваем большую программу на части, при этом никаких особых механизмов для связывания этих частей нет. Данный подход хорошо для создания программного обеспечения именно на языке ассемблера. Но никаких возможностей для использования такого подхода для интеграции языка ассемблера с другими языками программирования данный подход не дает.

Куда более интересным является многомодульность на втором этапе трансляции (компоновке). Дело в том, что структура объектных файлов, которые собираются на втором этапе трансляции имеют довольно универсальный характер. Следовательно, объектные файлы, полученные из программных моделей на разных языках программирования, в том числе и на языке ассемблера, могут быть объединены в один исполняемый модуль. При этом, конечно, должно быть выполнено ряд условий. Перечислим их.

1. Связывание возможно, если в объектных файлах есть, так называемые публичные имена, по которым это связывание и происходит.
2. Связывание происходит, когда в одном модуле есть обращение к ресурсу (к функции или переменной), которого в этом модуле нет, но обязательно есть в одном и только одном другом связываемом модуле.
3. Имена, используемые на уровне программных модулей и

которые не объявлены публичными, никак не влияют на процесс связывания.

4. Одно из указанных публичных имен должно быть именем функции, с которой начнется выполнения полученного исполняемого модуля.

5. При обращении к функции из другого модуля должен быть соблюден протокол передачи параметров. Если это не соблюдено, то вызов будет некорректен.

6. Кроме связывания объектных модулей могут также подключаться статические объектные библиотеки, а также указываться имена динамических библиотек, которые будут использоваться (подключаться) во время исполнения.

Одна важная особенность ассемблера GAS. Внешние имена, которые используются в данном модуле объявлять не нужно. Ассемблер автоматически считает все имена внешними, если функций и переменных в данном модуле с таким именем нет.

Рассмотрим пример многомодульной программы на языке ассемблера (рисунки 33-35). Программа состоит из трех модулей: prog15 – главный модуль, prog15a.s – функция вывода строки на консоль, prog15b.s – функция получения длины строки.

```
# prog15.s
# главный модуль
.data
msg:
.asciz "Многомодульное программирование\n"
.text # сегмент кода
.globl _start, msg
_start:
# вызов функции печать строки на консоль
mov $msg, %rdi # адрес строки
call print # вызов функции, из другого модуля
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
```

Рис. 33. Главный модуль программы

Пояснение к модулю prog.s. Модуль содержит строку msg, которая

должна выводиться с помощью функции `print`. Несмотря на то, что в функцию в качестве параметра отправляется адрес строки, предполагается, что `msg` также будет использована в другом модуле как внешнее имя.

```
# prog15a.s
# модуль с функцией вывода на консоль строки
.text
.globl print
print: # rdi адрес сообщения
mov $msg, %rdi # адрес строки, находящейся в другом модуле
call len # длина в rax
# сохраним, используемые в функции регистры
push %rdx
push %rcx
push %r11
# печатаем строку
mov %rax, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес сообщения (первый параметр)
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret
```

Рис. 34. Модуль с выводом строки на консоль

Пояснение к модулю `prog15a.s`. Модуль содержит функцию вывода строки на консоль. Для вывода необходима длина строки. Эта строка получается вызовом функции `len`, которая находится в третьем модуле. Команда `mov $msg, %rdi` возможна потому, что в модуле `prog15.s` `msg` объявлена публичным именем.

```

# prog15b.s
# модуль с функцией получения длины строки в байтах
.text
.globl len
# функция получения длины строки в байтах
len: # rdi - адрес строки
xor %rax, %rax # счетчик байтов строки
push %rbx
push %rdi
lo1:
movb (%rdi), %bl # байт из строки в bl
cmpb $0, %bl # конец строки
jz  ex1
inc %rdi
inc %rax # счетчик байтов
jmp lo1
ex1:
pop %rdi
pop %rbx
ret

```

Рис. 35. Модуль с функцией, вычисляющей длину строки

Пояснение к модулю prog15b.s. Функция содержит функцию получения длины строки. Критерием длины строки в байтах является здесь нулевой байт.

Мы разобрали многомодульное программирование на языке ассемблера. Интеграция языка ассемблера с языками высокого уровня будет изложена в следующем разделе.

2.3. Интеграция ассемблера и языков высокого уровня

Продолжением вопроса многомодульного программирования на языке ассемблера GAS является вопрос об интеграции объектных модулей, написанных на разных языках программирования. Вопрос важный с разных точек зрения. При программировании на языке ассемблера часто не хватает возможностей языков высокого уровня по доступу к внешним устройствам. Кроме того, часто необходимы алгоритмы, написания

которых на ассемблере требуют времени, а на языках высокого уровня они уже есть. С другой стороны, программам на языках высокого уровня часто требуется оптимизация, что можно достигнуть, в частности, средствами языка ассемблера.

2.3.1. Использование программы gcc для компилирования ассемблерных модулей

Программу gcc можно с успехом применять для трансляции программ на языке ассемблера. Следует отметить, что, в сущности, gcc не приносит ничего нового, поскольку использует все те же утилиты as и ld. Суть в том, что gcc позволяет легко интегрировать программы на языке ассемблера с программными возможностями языка C. Ниже мы подробно рассмотрим эти вопросы.

Рассмотрим для простоты программу на языке ассемблера с минимальной функциональностью. Она представлена на рисунке 36.

```
# prog16.s
.text # секция кода
.globl _start
_start: # точка входа
# закончить работу программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
```

Рис. 36. Простейшая программа на языке ассемблера

Для трансляции программы в двоичный исполняемый код требуется выполнить команды

```
as prog16.s -o prog16.o
ld prog16.o -o prog16
```

В общем это знакомый подход. Обратим только внимание, на метку `_start`. Как мы неоднократно подчеркивали, по умолчанию для утилиты `ld` это имя используется как имя входа в программу. Другими словами, `_start` присваивается адрес, который потом указывается в исполняемом двоичном модуле, как адрес начала выполнения программы.

Несколько изменим программу из рисунка 36 (см. Рисунок 37).

```
# prog17.s
.text # сеекция кода
```

```
.globl main
main: # точка входа
# закончить работу программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
```

Рис. 37. Программа с входной меткой main

Трансляция программы prog17.s

```
as prog17.s -o prog17.o
```

```
ld -e main prog17.o -o prog17
```

При трансляции мы указали, что точкой входа будет main. Для обычной нашей трансляции это не имеет никакого значения. Для программы по умолчанию точка входа должна иметь имя main. Также как в языке C. На самом деле это не первичная точка входа. По умолчанию gcc подсоединяет к исполняемому файлу библиотеку времени исполнения (run time library). Вот с функций этой библиотеки и начинается работа исполняемого кода. И только потом вызывается код, начинающий саму программу, который соответствует адресу main.

Откомпилировать программу из рисунка 37 можно следующими командами

```
gcc prog17.s -o prog17
```

или

```
as prog17.s -o prog17.o
```

```
gcc prog17.o -o prog17
```

Обратим внимание, что код, полученный с помощью утилиты gcc значительно больше бинарного, полученного без использования этой утилиты. Как я уже сказал, это связано с тем, что к исполняемому коду автоматически добавляется стандартная библиотека времени исполнения языка C. С одной стороны, увеличение кода программы никак не влияет на ее исполнение. С другой стороны, некоторые стандартные функции языка C не будут работать без библиотеки времени исполнения. Позднее мы вернемся к этому вопросу более подробно.

2.3.2. Параметры командной строки

Очень важный вопрос в программировании – доступ к параметрам командной строки. Вообще утилиты командной строки как раз и принимают данные таким образом. Это важная интерфейсная технология.

На рисунке 38 представлена программа, которая берет из командной строки параметры, и выводит их на консоль по одному на строку. Напомним, что параметры командной строки отделяются пробелами. Если в самом параметре есть пробелы, то он должен быть заключен в двойные кавычки.

```
# prog18.s
.data # секция данных
buf:
.space 200
.text # секция кода
.globl _start
_start:
mov %rsp, %rbp # вершина стека
mov (%rbp), %r9 # всего параметров
lo:
add $8, %rbp # следующий элемент стека (параметр командной строки)
mov $buf, %rax # адрес начала буфера
mov (%rbp), %r8 # адрес начала строки – параметра
xor %rdx, %rdx # счетчик байтов для каждой строки-параметра
# здесь копируем параметр в буфер
lo1:
mov (%r8, %rdx), %bl # берем байт
mov %bl, (%rax) # копируем в буфер
inc %rax # следующее место в буфере
inc %rdx # увеличиваем счетчик (необходимо для вывода строки)
cmpb $0, (%r8, %rdx) # конец строки?
jnz lo1 # продолжим копировать
movb $10, (%rax) # перевод строки ('\n')
inc %rdx # увеличить счетчик
# напечатать строку
mov %rdx, %rsi # длина строки
mov $buf, %rdi # адрес начала строки
call print # печать параметра
dec %r9 # проверяем, не закончились ли параметры
jnz lo
# закончить работу программы
mov $60, %rax # номер системного вызова exit
```

```

xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# функция печати строки
print: # rdi - адрес строки, rsi - длина строки
# сохраним, используемые в функции регистры.
push %rdx
push %rcx
push %r11
# печатаем строку
mov %rsi, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес строки
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret

```

Рис. 38. Пример программы читающей и выводящей параметры командной строки

Для трансляции программы выполняются команды:

```
as prog18.s -o prog18.o
```

```
ld prog18.o -o prog18
```

Выполнение программы

```
./prog18 par1 par2 par3
```

```
par1
```

```
par2
```

```
par3
```

Прокомментируем текст программы

1. При запуске исполняемого кода, операционная система подставляет в стек адреса параметров. В вершине стека лежит адрес количества параметров. Надо иметь в виду, что при запуске всегда есть первый параметр: путь к запущенной программе.

2. Программа копирует каждый из параметров в буфер, откуда потом параметр выводится на консоль (функция `print`). В принципе,

программу можно было бы изменить, выводя строку прямо с того, адреса, который мы получаем из стека. Но тогда нужно было бы, во-первых, решать вопрос о длине строки, во-вторых, после вывода каждой строки печатать символ перевода строки.

3. Следует лишний раз подчеркнуть, что структура доступа к параметрам командной строки обусловлена именно тем, что запуск программы осуществляется непосредственно с точки запуска, указанной меткой `_start`.

При компиляции программы с помощью программы `gcc`, как было показано в предыдущем разделе, запуск программы осуществляется с функций библиотеки времени выполнения. Из общих соображений понятно, что доступ к параметрам командной строки должен как-то измениться. Рассмотрим подробнее как получить параметры командной строки если программа компилируется при помощи утилиты `gcc`.

Если запускать программу, откомпилированную `gcc`, то метка `main` будет вызвана уже как функция, с передачей параметров по протоколу Linux x86-64. При этом `rdi` будет содержать количество параметров командной строки, `rsi` будет содержать указатель на массив указателей на строки, представляющие из себя эти самые параметры. Мы постарались провести минимальные изменения в программе из рисунка 38, так чтобы она стала корректно работать, если ее откомпилировать с помощью утилиты `gcc` (см. Рисунок 39). Ниже (рисунок 40) будет дан еще один вариант вывода параметров командной строки, по более упрощенной технологии.


```

# prog19.s
.data # секция данных
_rsi:
.quad 0
buf:
.space 200
.text # секция кода
.globl main
main:
mov %rsi, _rsi # для хранения текущего указателя на адрес параметра
mov %rdi, %r9 # количество параметров
lo:
mov _rsi, %rsi # текущий адрес
mov (%rsi), %rbp # адрес параметра
add $8, %rsi # следующий элемент
mov %rsi, _rsi
mov $buf, %rax # адрес начала буфера
mov %rbp, %r8 # адрес начала строки – параметра
xor %rdx, %rdx # счетчик байтов для каждой строки-параметра
lo1:
mov (%r8, %rdx), %bl # берем байт
mov %bl, (%rax) # копируем в буфер
inc %rax # следующее место в буфере
inc %rdx # увеличиваем счетчик (необходимо для вывода строки)
cmpb $0, (%r8, %rdx) # конец строки?
jnz lo1 # продолжим копировать
movb $10, (%rax) # перевод строки ('\n')
inc %rdx # увеличить счетчик
# напечатать строку
mov %rdx, %rsi # длина строки
mov $buf, %rdi # адрес начала строки
call print # печать параметра
dec %r9 # проверяем, не закончились ли параметры
jnz lo
# закончить работу программы
mov $60, %rax # номер системного вызова exit

```

```

xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# функция печати строки
print: # rdi - адрес строки, rsi - длина строки
# сохраним, используемые в функции регистры.
push %rdx
push %rcx
push %r11
# печатаем строку
mov %rsi, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес строки
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret

```

Рис. 39. Пример программы читающей и выводящей параметры командной строки, при компиляции утилитой gcc

Чтобы транслировать программу из рисунка 39, выполним команды

```
as prog19.s -o prog19.o
```

```
gcc -no-pie prog19.o -o prog19
```

Обратим внимание на параметр `-no-pie`, он необходим, если в программе используются глобальные переменные (в секции `.data`).

На рисунке 40 используется та же технология, что и в программе из рисунка 39, но здесь мы обошлись без дополнительного буфера, куда копировались параметры командной строки. Это сокращает используемый объем памяти и используемое время.

```

# prog20.s
.text # секция кода
.globl main
main:
mov %rsi, %r10 # для хранения текущего указателя на адрес параметра

```

```

mov %rdi, %r9 # количество параметров
lo:
# напечатать строку
mov (%r10), %rdi # адрес начала строки
call print # печать параметра
add $8, %r10
dec %r9 # проверяем, не закончились ли параметры
jnz lo
# закончить работу программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# функция печати строки
print: # rdi - адрес строки
# сохраним, используемые в функции регистры.
push %rdx
push %rcx
push %r11
# получить длину строки
call len
movb $10, (%rdi, %rax)
inc %rax
# печатаем строку
mov %rax, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес строки
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret
# получить длину строки
len: # rdi - адрес строки
xor %rax, %rax # счетчик байтов строки

```

```

push %rbx
push %rdi
lo1:
movb (%rdi), %bl # байт из строки в bl
cmpb $0, %bl # конец строки
jz  ex1
inc %rdi
inc %rax # счетчик байтов
jmp lo1
ex1:
pop %rdi
pop %rbx
ret

```

Рис. 40. Вариант вывода параметров командной строки без использования дополнительного буфера

Чтобы транслировать программу из рисунка 40, выполним команды
as prog20.s -o prog20.o
gcc prog20.o -o prog20

В программе из рисунка 40 добавлена функция по получению длины параметра. В функции print мы добавляем в конец строки код 10, для перевода. Подход не совсем корректен, поскольку мы тем самым портим структуру параметров. Более правильным был бы подход, если печать перевода строки шла отдельно. Теперь становятся ясным смысл использования дополнительного буфера в примерах из рисунков 39 и 38.

2.3.3. Использование ассемблерных модулей на языках высокого уровня

Использование модулей, написанных на языке ассемблера в языках высокого уровня, вполне укладываются в схему компиляции, где на последнем этапе объединяются объектные модули (см. Рисунок 3). В данном разделе рассмотрим пример, когда программа на языке C обращается к функции и переменной, которые определены в модуле на языке ассемблера.

На рисунках 41 и 42 представлены программа на языке C и программный модуль на ассемблере. Программа на языке C использует функцию и переменную из ассемблерного модуля.

```

//prog21.c
#include <stdio.h>
// внешняя функция и переменная из ассемблерного модуля
extern __int64_t add2(__int64_t, __int64_t);
extern __int64_t res;

int main(){
    __int64_t a=5000, b=3500;
    printf("%lu\n", add2(a,b)); // сумма
    printf("%lu\n", res); // разность
    return 0;
}

```

Рис. 41. Программа на языке С, использующая функцию из ассемблерного модуля

Рассмотрим программу на языке С (рисунок 41). Здесь важно отметить, что внешние имена переменных и функций, которые используются в данной программе должны быть объявлены с помощью ключевого слова `extern`. Здесь мы заботимся о том, каков будет протокол передачи параметров, поскольку транслятор автоматически реализует известный уже нам протокол для Linux x86-64.

```

# prog22.s
.data
res:
    .quad 0
.text # сегмент кода
# функция и переменная будут использоваться внешними модулями
.globl add2
.globl res
add2:
# параметры согласно соглашению о вызове
push %rsi
add %rdi, %rsi
mov %rsi, %rax # функция возвращает сумму
pop %rsi
sub %rsi, %rdi
mov %rdi, res # разность

```

```
ret
```

Рис. 42. Модуль на языке ассемблера, содержащий функцию и переменную, которые используются из программы на языке C

Модуль из рисунка 42 содержит функцию, которая вычисляет сумму и разность полученных числовых параметров. Сумма возвращается стандартно через регистр `eax`, как и положено протоколу. Разность помещается в переменную `res`. Заметим, что и функция, и переменная объявляются как `globl`. Только тогда эти имена станут доступны из других модулей при компоновке. Второе, на что следует обратить внимание – протокол передачи параметров. Этот протокол предполагает передачу через параметры через регистры. Первые два параметра передаются через `rdi` и `rsi`.

Трансляция представленных модулей осуществляется командами

```
gcc -c prog21.c
```

```
as prog22.s -o prog22.o
```

```
gcc -no-pie prog21.o prog22.o -o prog22
```

Программа на языке Pascal (Free Pascal), использующая глобальные функцию и переменную из модуля `prog22.s` представлена на рисунке 43. Синтаксис языка позволяет указать подключаемый объектный модуль непосредственно по средством директив в программе. Для указания же внешних переменных и функций используется ключевое слово `external`.

```
Program p1;
{$L prog22.o}
var
  res : longint; external name 'res';
function add2(a, b:longint):longint; external;
begin
  writeln(add2(4000, 1200));
  writeln(res);
end.
```

Рис. 43. Пример программы на Free Pascal (`prog23.pas`), использующий объектный модуль из рисунка 42

Компиляция программы на языке pascal осуществляется, как обычно командой:

```
fpc prog23.pas
```

Предварительно

```
as prog22.s -o prog22.o
```

2.3.4. Статические библиотеки

Статические библиотеки позволяют объединить объектные модули в один файл. Это очень удобно. Такие библиотеки можно использовать в разных проектах облегчая и ускоряя процесс разработки. В качестве примера соберем подобную библиотеку и приведем пример ее использования.

Напишем три модуля на языке ассемблера, содержащие соответственно функции: `print` – вывода строки на консоль (рисунок 44), `len` – получение длины строки в байтах (рисунок 45), `input` – получение строки с консоли (рисунок 46). Ниже представлены тексты всех перечисленных модулей.

```
# prog24.s
# модуль с функцией вывода на консоль
.text
.globl print
print: # rdi адрес сообщения, rsi длина сообщения
# сохраним, используемые в функции регистры.
push %rdx
push %rcx
push %r11
# печатаем строку
mov %rsi, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес сообщения (первый параметр)
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall # системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret
```

Рис. 44. Модуль содержит функцию вывода строки на консоль

```

# prog25.s
# модуль с функцией получения длины строки в байтах
.text
.globl len
# функция получения длины строки в байтах
len: # rdi - адрес строки
xor %rax, %rax
push %rbx
push %rdi
lo1:
movb (%rdi), %bl # получить очередной байт
cmpb $0, %bl # нулевой байт не войдет в длину строки
jz ex1
inc %rdi # счетчик байтов
inc %rax # к следующему байту
jmp lo1
ex1:
pop %rdi
pop %rbx
ret

```

Рис. 45. Модуль содержит функцию получения длины строки в байтах

```

# prog26.s
# модуль с функцией ввода с консоли
.text
.globl input
input: # rdi адрес буфера, rsi длина сообщения
# сохраним, используемые в функции регистры
push %rdx
push %rcx
push %r11
# читаем строку
mov %rsi, %rdx # длина буфера
mov %rdi, %rsi # адрес буфера
mov $0, %rdi # stdin
mov $0, %rax # номер системной функции чтения
syscall # системный в

```



```

pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции.
ret

```

Рис. 46. Модуль ввода строки с консоли

В результате выполнения команд

```

as prog24.s -o prog24.o
as prog25.s -o prog25.o
as prog26.s -o prog26.o

```

мы получим три объектных модуля. Конечно при трансляции можно использовать каждый модуль в отдельности, как это мы до сих пор делали. Но библиотека в этом смысле удобнее. Для создания статических библиотек принято использовать старую Unix-утилиту `ar` (archiver - архиватор). В настоящее время ее основная функция создавать статические библиотеки объектных модулей. Выполним команду

```
ar rc lib_lib1.a prog24.o prog25.o prog26.o
```

Имя библиотеки принято начинать с префикса `lib_`, а расширение библиотек `.a`.

На рисунке 47 представлена программа, ожидающая ввода строки со стандартного устройства и выводящая на стандартное устройства вывода эту строку. При этом программа использует внешние функции `print`, `len`, `input`.

```

# prog27.s
.data
buf:
.space 200, 0 # буфер для ввода
.globl _start
.text
_start:
mov $200, %rsi # адрес буфера
mov $buf, %rdi
call input # вызов библиотечной функции
# получить длину
mov $buf, %rdi
call len # вызов библиотечной функции
mov $buf, %rdi

```

```

mov %rax, %rsi
call print  # вызов библиотечной функции
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall     # системный вызов

```

Рис. 47. Программа, использующая библиотечные функции

Для компиляции программы из рисунка 47 выполним команды:

```
as prog27.s -o prog27.o
```

```
ld prog27.o -L. -l_lib1 -o prog27
```

В результате получим исполняемый модуль prog27, выполняющий описанные выше функции.

Созданная нами библиотека может, конечно, использоваться и в языках высокого уровня. Обратимся к языку программирования Паскаль (Free Pascal). Программа на языке Паскаль на рисунке 48.

```

{prog28.pas}
type
  tm=array[1..200] of char;
var
  a: tm;
  i: integer;
{объявляем внешние функции}
{обязательно указываем протокол передачи параметров cdecl}
Function input(Var x:tm; n:dword):longint; cdecl; external;
Function len(x:tm):longint; cdecl; external;
Function print(x:tm; n:dword):longint; cdecl; external;
{$LINKLIB 'lib_lib1.a'}
begin
  input(a, 200);
  i:=len(a);
  print(a, i);
end.

```

Рис. 48. Программа на языке Паскаль, использующая статическую библиотеку

Трансляция программы из рисунка 48 осуществляется обычной

командой:

```
frs prog28.pas
```

При трансляции статическая библиотека автоматически присоединяется к исполняемому модулю.

2.3.5. Использование языков высокого уровня на языке ассемблера

Для языка ассемблера, который по определению требует много времени для разработки, использование возможностей языков высокого уровня, является важной возможностью ускорить разработку, сократить код, уменьшить вероятность появления ошибок.

В стандартной библиотеке языка С есть довольно много интересных и полезных функций. Использование их в программах на языке ассемблера может значительно сократить время разработке, так как отпадает необходимость написания своих функций на языке ассемблера.

На рисунке 49 представлена программа, использующая функцию из стандартной библиотеки С. Особо отметим, что речь идет о функции, которая не требует для своей правильной работы использования библиотеки времени выполнения языка С.

```
# prog29.s
.data # секция данных
num:
.quad -56
.text # секция кода
.globl _start
_start:
# получить абсолютное значение числа
mov num, %rdi
call abs
# закончить работу программы
mov %rax, %rdi # код возврата, как результат.
mov $60, %rax # номер системного вызова exit
syscall # системный вызов
```

Рис. 49. Пример использования стандартной функции языка С

Для трансляции программы из рисунка 49 используем команды:
as prog29.s -o prog29.o

```
ld prog29.o -static -lc -o prog29
```

Выполняем

```
./prog29
```

```
echo $?
```

Получаем результат 56

Подчеркнем лишний раз, что опции `-static -lc` требуются для правильной компоновки библиотечных функций, которые не требуют библиотеки времени выполнения. `-lc` непосредственно указывает, что при компоновке следует использовать стандартную библиотеку языка C.

Обратимся теперь к остальным функциям стандартной библиотеки C (рисунок 50).

```
# prog30.s
.data
msg:
.ascii "Это стандартная функция puts, товарищ!\n"
.text # сегмент кода
.globl main
main:
# поместим параметры согласно соглашению о вызове
mov $msg, %rdi # адрес строки
call puts      # вызов процедуры, которая в другом модуле
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall      # системный вызов
```

Рис. 50. Пример использования стандартной функции `puts` языка C

Для трансляции используем команды:

```
as prog30.s -o prog30.o
```

```
gcc -no-pie prog30.o -o prog30
```

Особо отметим то, что для компоновки используется программа `gcc`. Она автоматически компоует вместе с программой и стандартную библиотеку времени выполнения языка C, что обеспечивает корректную работу стандартной функции `puts`.

Для некоторых библиотечных функций требуется предварительное резервирование стека перед вызовом. К таким, в частности, относятся функции `scanf()` и `printf()`. Ниже на рисунке 51 представлен пример ввода 64 битового числа и вывод его с помощью стандартной функции `printf`.

```

# prog31.s
.data
num:
.quad 0
format:
.asciz "%llu\n"
format1:
.asciz "%llu"
.text
.global main
main:
# ввод данного
push %rbp # резервирование стека
mov $format1, %rdi # строка формата
mov $num, %rsi # адрес буфера
call scanf # вызов библиотечной функции
pop %rbp
# вывод данного
push %rbx # резервирование стека
mov $format, %rdi # адрес строки - формата
mov num, %rsi # адрес формата
call printf # вызов библиотечной функции
pop %rbx
# выход или просто ret
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов

```

Рис. 51. Пример использования библиотечных функций scanf и printf

Для трансляции программы из рисунка 51 используем последовательность команд:

```

as prog31.s -o prog31.o
gcc -no-pie prog31.o -o prog31

```

Рассмотрим теперь вопрос, когда программа на языке ассемблера использует готовые решения на языке C. Ниже представлено три программных модуля (рисунки 52, 53, 54). На рисунке 52 представлена программа на языке ассемблера, которая использует функции из модулей

на языке C.

```
# prog32.s
.data
num:
.quad 4567823
.text # сегмент кода
# определим глобальные имена
.globl main, num
main:
# вызов функции получения обратного числа
# поместим параметры в регистры согласно соглашению о вызове
mov num, %rdi
call rev
# вызов функции печати, по возвращенному значению
mov %rax, %rdi
call printn
# вызов функции печати, по глобальной переменной
mov $num, %rdi
call printn
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall      # системный вызов
```

Рис. 52. Программа на языке ассемблера, которая использует два модуля на языке C (функции `rev` и `printn`)

Сделаем несколько пояснений к программе из рисунка 52. В программе определена переменная `num`, имя которой объявлено публичной. Переменная инициализирована конкретным числом 4567823. Вызвав функцию `rev` мы получаем перевернутое число. При этом функция меняет содержимое переменной, получив адрес ее в качестве параметра. Одновременно функция возвращает адрес переменной.

```
//prog33.c
#include <stdio.h>
int printn(long long * nm){
    printf("%llu\n",*nm);
    return 0;
}
```

Рис. 53. В модуле представлена функция вывода на стандартное устройство длинного целого (64 битового) числа

В модуле из рисунка 53 выполняется только одно, выводится на стандартное устройство. При этом адрес выводимой переменной передается в качестве параметра.

```
// prog34.c
extern unsigned long long num;
unsigned long long * rev(unsigned long long nm){
    int ar[32], i=0, k=1;
    while(1){
        ar[i] = nm % 10;
        nm = nm / 10;
        if(!nm)break;
        i++;
    }
    num = 0;
    for(int j=i; j>=0; j--){
        num=num+ar[j]*k;
        k=k*10;
    }
    return &num;
}
```

Рис. 54. В модуле представлена функция, которая «переворачивает» число, одновременно присваивая это значение глобальной переменной

На рисунке 54 представлена функция, которая обращает число. При этом алгоритм построен так, что его половина выполняется на основе полученного адреса переменной, а вторая половина обращается к переменной, как к внешней (extern unsigned long long num).

Для трансляции выполним команды:

```
as prog32.s -o prog32.o
gcc -c prog33.c
gcc -c prog34.c
gcc -no-pie prog32.o prog33.o prog34.o -o prog32
```

В результате будет получен двоичный исполняемый модуль prog32, запуском которого ./prog32 можно проверить правильность работы всей программы.

2.3.6. Динамические библиотеки

На последней стадии трансляции объединяются объектные модули и библиотеки. Это, как мы уже подчеркивали называется ранним связыванием²². Все ссылки на внешние функции и переменные должны быть разрешены (связаны). Если какому-либо внешнему имени не найдено соответствие в другом объектном модуле, то трансляция заканчивается с ошибкой. Если объектного кода, который подсоединяется во время трансляции много, то и результирующая исполняемая программа может приобретать большие размеры. Динамические библиотеки содержат в себя исполняемый код и подключаются к программе во время исполнения. Это называется поздним связыванием.

1. Динамическую библиотеку могут одновременно использовать сразу несколько процессов, а это дает экономию и дискового пространства, и памяти.

2. Можно модернизировать сразу несколько программных пакетов изменив код и перекомпилировав только одну динамическую библиотеку и не трогая основной код.

Динамическую библиотеку можно сделать на основе тех же объектных модулей, на основе которых мы делали статическую библиотеку. Обратимся к модулям из рисунков 44, 45 ,46. Как и в случае с динамической библиотекой готовим объектные модули

```
as prog24.s -o prog24.o
as prog25.s -o prog25.o
as prog26.s -o prog26.o
```

Далее следует выполнить команду

```
ld -shared prog24.o prog25.o prog26.o -o lib_my2.so
```

чтобы создать динамическую библиотеку (lib_my2.so), которая будет содержать в себе функции из указанных объектных модулей.

²² Ранним, значит еще на стадии трансляции.

Теперь подготовим главный модуль (Рисунок 55).

```
# prog35.s
.data
buf:
.space 256, 0 # буфер для ввода
.globl main
.text
main:
mov $200, %rsi # адрес буфера
mov $buf, %rdi
call input # вызов библиотечной функции
# получить длину
mov $buf, %rdi
call len # вызов библиотечной функции
mov $buf, %rdi
mov %rax, %rsi
call print # вызов библиотечной функции
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
```

Рис. 55. Программа, использующая динамическую библиотеку

Для создания объектного модуля выполним команду:

```
as prog35.s -o prog35.o
```

Теперь нужно связать объектный модуль с динамической библиотекой. В нашем случае динамическая библиотека находится в текущем каталоге.

```
gcc -no-pie prog35.o ./lib_my2.so -o prog35
```

Запускается программа обычным способом

```
./prog35
```

2.4. Основы системного программирования в операционной системе Linux

Под системным программированием будем понимать программную технологию, позволяющую управлять сервисами операционной системы. Мы выделяем три раздела системного программирования: управление

файловой системой, управление памятью, управление процессами.

2.4.1. Файловая система

Файловая система едва ли не самый важный элемент операционных систем. Операционная система предоставляет доступ к файловому хранилищу посредством системных функций. Можно выделить два вида доступа:

1. Доступ к содержимому конкретного файла: открытие файла, закрытие файла, чтение файла, запись запись, перемещение по файлу, изменение атрибутов открытого файла.
2. Управление каталогами файлов: просмотр списков файлов в каталогах, удаление файлов, создание каталогов, получение данных о файлах.

2.4.2. Доступ к содержимому файлов

В начале перечислим основные системные функции, с помощью которых осуществляется доступ к содержимому файлов (см. Приложение 1).

1. Открыть файл – 2 (sys_open).
2. Закрыть файл – 3 (sys_close).
3. Запись в файл – 1 (sys_write).
4. Чтение из файла – 0 (sys_read)
5. Передвижение по файлу – 8 (sys_lseek).
6. Смена прав доступа – 90 (sys_chmod), 91 – (sys_fchmod).
7. Изменение длины файла – 76 (sys_truncate), 77 – (sys_ftruncate).

Остановимся подробнее на указанных функциях.

Для открытия файла необходимо его имя – короткое, полное или относительное. Вторым параметром являются флаги открытия. Флаги могут объединять командой «ИЛИ». Чаще всего встречаются такие флаги: O_RDONLY = 0 – открыть только для чтения, O_WRONLY = 1 – открыть только записи, O_RDWR = 2 – открыть для чтения и записи, O_CREAT = 64 – создать, если файл не существует, O_TRUNC=512 – обнулить длину файла, если файл существует, O_APPEND=1024 – добавлять в конец файла. Третий параметр – это режим открытия. В случае, если файл открывается только для чтения, этот параметр игнорируется. Если открывать для записи следует продумать сценарий открытия: что делать, если файла нет, что делать если файл есть и т.п. Для этого и нужны перечисленные флаги.

Третий параметр, в сущности, нужен только в том случае, если предполагается возможность создания нового файла. В этом случае предполагается определить флаги доступа, согласно общим положениям операционных систем семейства Unix. При удачном открытии функции открытия файла возвращается дескриптор открытого файла – целое число больше 2. Далее все функции работают с этим дескриптором.

Закрыть файл, значит удалить из памяти все, что было связано с ним. В том числе и сбросить на внешнее устройство содержимое системных буферов. Число – дескриптор становится с этого момента свободным и может быть присвоено другому открытому файлу²³.

Функции чтения и записи из файла были ранее разобраны, поскольку использовались для чтения и записи на стандартные устройства (дескрипторы 0 и 1, дескриптор 2 для вывода на стандартное устройство для вывода ошибок).

Функция передвижения по файлу `sys_lseek`. При открытии файла условный указатель устанавливается на первый (нулевой) байт. При чтении или записи этот указатель сдвигается на количество прочитанных – записанных байтов. Но функция `sys_lseek` позволяет произвольно двигать указатель по файлу. Первый параметр – дескриптор открытого файла. Второй параметр – смещение, на сколько байтов нужно продвинуться. Третий параметр – как двигаться. 0 (`SEEK_SET`) – смещение от начала файла, 1 (`SEEK_CUR`)

- текущее положение плюс смещение, 2 (`SEEK_END`) – смещение плюс длина файла. Отсюда, кстати, вытекает, что можно уйти за конец файл и сделать там запись, тем самым удлинив файл.

Как было указано выше права доступа можно изменить при создании файла. Но есть отдельные функции. Первая изменяет права доступа открытого файла (`sys_chmod`), вторая (`sys_fchmod`) по имени файла. Вторая, обладает несколько большими возможностями, так как не требует открытия файла, который не всегда можно открыть.

Особо остановимся на функциях 76 (`sys_truncate`), 77 (`sys_ftruncate`). Обе функции меняют длину файла. Хотя длина файла также и задается путем записи туда. Например, можно перейти в конец файла (`sys_lseek`), а потом писать туда, при этом длина будет увеличиваться. Но часто необходимо сократить длину файла. Указанные функции вторым аргументом получают длину файла. Первым для первой функции идет имя файла, для второй дескриптор открытого файла. При этом, если указанная

²³ В рамках данного процесса. Поскольку таблицы дескрипторов выделяются каждому запущенному процессу.

больше указанной длины, то файл обрезается, а если меньше, то длина увеличивается. Добавленная часть при этом заполняется нулями.

В качестве иллюстрации использования перечисленных выше функций рассмотрим программу из рисунка 56. Программа открывает указанный файл, читая последние N байтов и записывает эти байты в новый файл. Если длина первого файла меньше N, то берутся все байты файла. Файл для копирования создается заново, т. е. 1. Если файла с таим именем нет, то файл создает. 2. Если файл есть, то в начале его длина обнуляется и затем заполняется скопированными байтами.

```
# prog36.s
.data
num:
.quad 20 # количество байтов, которые нужно считать с конца
fd1:
.quad 0
file1: # указывает на имя первого файла для чтения
.asciz "/ui"
fd2:
.quad 0
file2: # указывает на имя второго файла для записи
.asciz "/temp"
buf:
.space 100, 0
.text
.globl _start
_start:
# открыть файл для чтения
mov $file1, %rdi # адрес строки с именем файла
mov $0, %rsi # открываем для чтения
mov $2, %rax # номер системного вызова
syscall # вызов функции "открыть файл"
cmp $0, %rax # нет ли ошибки при открытии
jl exi # перейти к концу программы
mov %rax, fd1 # дескриптор первого открытого файла
# ==считать num байтов==
# в конец файла
```

```

mov fd1,%rdi # дескриптор
mov $0, %rsi # смещение
mov $2, %rdx # в конец (2)
mov $8, %rax # номер системной функции
syscall
sub num, %rax
jg con # возможный вариант, когда длина меньше указанной длины
add %rax, num
mov $0, %rax
con:
# назад
mov fd1,%rdi # дескриптор
mov %rax, %rsi # смещение
mov $0, %rdx # от начала (0)
mov $8, %rax # номер системной функции
syscall
# читать num байтов в буфер buf
mov fd1, %rdi # номер дескриптора файла для чтения
mov $buf, %rsi # буфер для записи из файла
mov num, %rdx # длина буфера для чтения
mov $0, %rax # номер системной функции чтения
syscall # системный вызов
#==считанные байты в буфере==
# открыть файл для записи
mov $file2,%rdi # адрес строки с именем файла
mov $1, %rsi # открываем для записи (O_WRONLY)
or $64, %rsi # создать, если файла нет (O_CREAT)
or $512, %rsi # обнулить длину, если файл есть (O_TRUNC)
mov $0666, %rdx # права доступа создаваемого файла
mov $2, %rax # номер системного вызова
syscall # вызов функции "открыть файл"
cmp $0, %rax # нет ли ошибки при открытии
jl exi1
mov %rax, fd2
# записать в открытый для записи файл из buf
mov fd2, %rdi # дескриптор
mov $buf, %rsi # буфер с байтами

```

```

mov num, %rdx # количество байтов
mov $1, %rax # номер системной функции для записи
syscall
# закрыть второй файл
mov fd2, %rdi
mov $3, %rax # close
syscall
# закрыть первый файл
exi1:
mov fd1, %rdi
mov $3, %rax # close
syscall
exi:
# выход
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall      # системный вызов

```

Рис. 56. Программа записи части одного файла в другой

Прокомментируем программу из рисунка 56.

1. Чтобы несколько упростить программу имена файлов (file1 и file2) и количество считываемых байтов (num) задается непосредственно в программе. Кроме того, для хранения дескрипторов файлов используются переменные fd1 и fd2. buf – область для временного хранения считанных из файла байтов. Надо иметь в виду, что отведенного объема может не хватить, если увеличить содержимое num. Само собой напрашивается вопрос о динамическом выделении памяти, но этот будет рассмотрен несколько позднее.

2. Обратим внимание, что при открытии файлов обрабатывается возможная ошибка – содержимое rax отрицательно (cmp \$0, %rax/jl exi). В случае ошибки работа программа заканчивается.

3. Особо отметим выполнение функции sys_lseek. При переходе в конец файла в rax помещается его длина. Далее ключевым моментом является следующий фрагмент:

```

sub num, %rax
jg con # возможный вариант, когда длина меньше указанной длины
add %rax, num
mov $0, %rax

```

После выполнения этого фрагмента в `гах` будет число байтов, на которые нужно продвинуться от его начала, чтобы потом считывать количество байтов, которые лежат в `put`. Если длина файла меньше числа в `put`, то указатель перейдет на начало файла. При этом в `put` будет лежать именно длина файла. Если же длина файла больше числа в `put` не изменится, а `гах` будет содержать длину файла минус число `put`, так чтобы находится на расстоянии `put` от конца файла.

4. Дальнейшие действия достаточно очевидны: считывается последние `put` байтов и помещаются в буфер `buf`. Затем эти байты пишутся во второй файл и после оба файла закрываются.

5. Стоит также остановиться при открытии второго файла указываются флаги: открыть для записи, создать если файла нет и обнулить длину файла, если он есть. При этом при создании указывается режим доступа к файлу: `0666` (восьмеричная система) - читать/писать для собственника, читать для группы, читать для других.

Подведем некоторый итог работы с содержимым файлов.

1. Для работы с содержимым файла он должен быть открыт с помощью специальной системной функции `sys_open`. Функция возвращает числовой дескриптор (номер) файла, который потом используется для других операций над файлом.

2. Надо иметь в виду, что для низкоуровневых функций, коими являются системные функции Linux, любой файл является только лишь последовательностью байтов. Для любой другой интерпретации содержимого файла требуется дополнительная программно-алгоритмическая надстройка.

3. Следует иметь в виду, что в многозадачной среде открытие файла для чтения является самой «мягкой» файловой операцией, так как сразу допускает многопользовательский доступ к файлу для чтения из него. Для записи файл нужны дополнительные права, также дополнительные права нужны в том случае, если программа собирается создать файл в каталоге `[]` (право записи в каталог).

4. Еще раз обратим внимание, что для и записи и чтения из файла используются те же функции, что пишут и читают со стандартных устройств. Просто стандартные устройства в Linux рассматриваются как обычные файлы с заранее определенными значениями дескрипторов: 0, 1, 2. Соответственно, при открытии файла ему присваивается дескриптор с номером большим 2.

2.4.3. Управление файловой системой

Как уже было сказано выше, под управлением файловой системой мы понимаем операции в целом над файлами и каталогами, как над целыми объектами²⁴. В сущности, нам нужны следующие операции:

1. Просмотр содержимого каталогов, с возможностью, как минимум различать каталоги и не каталоги.

2. Создание и удаление каталогов.

3. Удаление файлов.

4. Переименование файлов и каталогов.

5. Смена текущего каталога.

Перечисли системные функции, которые позволяют выполнять указанные выше операции.

1. Создание каталога – 83 (`sys_mkdir`).

2. Удаление пустого каталога – 84 (`sys_rmdir`).

3. Удаление файла 87 (`sys_unlink`). Надо иметь в виду, что на один и тот же файл может быть несколько ссылок и данная функция удаляет указанную ссылку. Файл окончательно удаляется, если удалена последняя ссылка на него.

4. Переименовать файл или каталог. Функция 82 – (`sys_rename`). Файл не только можно переименовать, но и переместить в другой каталог. Суть здесь в том, что меняется полное имя файла (путь к файлу).

5. Сменить текущий каталог – 80 (`sys_chdir`). В функции указывается полный путь к каталогу, который становится текущим.

Все перечисленные выше системные функции не столь сложны в использовании. Рассмотрим более сложный вопрос о том, как считывать данные о файлах в каталоге. В силу сложности программы разобьем ее на части на части. Для сборки их используем известную директиву `.include`.

Представленная программа читает указанный в командной строке каталог и выводит на стандартное устройство список файлов, указывая для каждого из файлов его тип. Всего в программе различается семь типов: файл, каталог, файл `fifo`, сокет, символьная ссылка, блочное устройство, символьное устройство.

На рисунке 57 представлена секция данных: буфер для хранения имени файла, строки с названием типов файлов, управляющий символ перевода строки.

²⁴ Конечно, кроме обычных файлов и каталогов, которые также рассматриваются как файлы особого типа, есть объекты другого типа: блочные устройства, символьные устройства, символические ссылки, туннели, сокеты []. Но в нашем изложении мы почти не касаемся этих объектов.


```

buf: # имя каталога
.space 1024,0
# названия типов файло в каталоге
dt_reg:
.ascii " -Файл\0"
dt_dir:
.ascii " -Каталог\0"
dt_fifo:
.ascii " -Файл fifo\0"
dt_sock:
.ascii " -Сокет\0"
dt_lnk:
.ascii " -Символьная ссылка\0"
dt_blk:
.ascii " -Блочное устройство\0"
dt_chr:
.ascii " -Символьное устройство\0"
# символ перевода строки
ent:
.ascii "\n\0"

```

Рис 57. Секция данных программы prog37.s

В модуле на рисунке 58 представлены функции для вывода строки на стандартное устройство: вывод на устройство `print`, получение длины строки `len`, вывод управляющего символа перевода строки `print_ent`. Не будем комментировать более этот модуль, так как выполняемые там фрагменты были ранее используемы, а также прокомментированы.

```

# функция получения длины строки в байтах
len: # rdi - адрес строки
xor %rax, %rax # обнуляем счетчик байтов
push %rbx
push %rdi
loo1:
movb (%rdi), %bl
cmp $0, %bl # проверка конца строки
jz ex1
inc %rdi
inc %rax

```

```

jmp loo1
ex1:
pop %rdi
pop %rbx
ret # в rax длина строки
# функция печати строки
print: # rdi - адрес строки
# определить длину строки
call len
# сохраним, используемые в функции регистры
push %rdx
push %rcx
push %r11
push %rsi
push %rdi
# печатаем строку
mov %rax, %rdx # длина сообщения (второй параметр)
mov %rdi, %rsi # адрес строки
mov $1, %rdi # stdout
mov $1, %rax # номер системного вызова для вывода на консоль
syscall      # системный вызов
# восстанавливаем содержимое всех регистров
pop %rdi
pop %rsi
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax # возвращаем статус выполнения функции
ret
# печать перевода строки
print_ent:
push %rdi
mov $ent, %rdi
call print
pop %rdi
ret

```

Рис. 58. Функция печати `print`, а также получения длины строки `len` и вывод перевода строки `print_ent`

Ранее мы уже рассматривали то, как получить список параметров командной строки. На рисунке 59 представлена функция, которая получает параметр с заданным номером и копирует его в указанный буфер. Номер получаемого параметра должен содержать в `rdi`, адрес буфера в `rsi`. Функция возвращает длину полученного параметра в `rax`. Если параметр не получен (номер параметра превосходит его количество) то по окончании работы функции `rax` будет содержать 0. Важно также отметить, что в функции учитывается смещение стека при вызове функции по отношению адреса, указывающего на блок параметров. Следует, однако, иметь в виду, что по-сути данная функция рассчитана, что ее будут вызывать из основной программы. Если функцию вызвать из другой функции, то следует предварительно сделать корректировку для получения адреса параметров.

```
# получить параметр командной строки с заданным номером в rdi
# адрес, куда копируется параметр в rsi
getpar:
push %rbp
mov %rsp, %rbp  # вершина стека
# rbp указывает на структуру параметров с учетом изменения стека (вызов
+
# команда push) - 16 байтов
add $16, %rbp
# обеспечиваем сохранность всех используемых регистров
push %r8
push %r9
push %rbx
push %rdi
push %rsi
xor %rax, %rax  # в конце или длина параметра или 0, если параметра нет
mov (%rbp), %r9  # всего параметров
lo:
add $8, %rbp  # следующий элемент стека (параметр командной строки)
dec %rdi  # пока счетчик не обнулится
jz lo3
dec %r9  # декремент счетчика общего количества параметров
jz lo2
jmp lo
```

```

lo3:
mov (%rbp), %r8 # адрес начала строки – параметра
lo1:
mov (%r8, %rax), %bl # берем байт
mov %bl, (%rsi) # копируем в буфер
inc %rsi # следующее место в буфере
inc %rax # увеличиваем счетчик байтов (необходимо для вывода
строки)
cmpb $0, (%r8, %rax) # конец строки?
jnz lo1 # продолжим копировать
movb $0, (%rsi) # в конце строки поставим 0 (на всякий случай)
inc %rax # увеличить счетчик
lo2:
# восстановление регистров
pop %rsi
pop %rdi
pop %rbx
pop %r8
pop %r9
pop %rbp
ret

```

Рис. 59. Модуль с функцией получения параметра с заданным номером

На рисунке 60 представлена функция получения списка файлов указанного каталога (getfiles). Функция также выводит имена файлов на стандартное устройство и указывает тип файла. Она основывается на системном вызове с номером 78 (getdents). Особенность данного вызова заключается в том, что в качестве одного из входных параметров берется дескриптор открытого каталога. Каталог, открывается обычным способом, как и любой файл. Единственное отличие от обычного открытия файла заключается в том, что во втором параметре (режим открытия) добавляется константа O_DIRECTORY равная 65536. Добавление осуществляется с помощью операции ИЛИ.

Кроме дескриптора открытого каталога функция 78 имеет вторым параметром адрес буфера, а третьим параметром размер буфера. В данном случае размер берется равным 1024. Функция возвращает количество прочитанных байтов. Это количество меньше или равно 1024. При этом указанный буфера заполняется данными о файлах. При этом размера

буфера может не хватить, так что в этом случае придется вызывать эту функцию несколько раз (внешний цикл). Чтение содержимого каталога заканчивается, когда при очередном чтении системная функция возвратит 0 (каталог закончился) или отрицательно число. Программа, в частности, проверяет не меньше ли результат 1.

Внутренний цикл проходит по результатам чтения (буферу) и выводит данные о файлах. И здесь следует остановиться более подробно. Данные о каждом файле представляются в виде структуры, длина которой может быть разной. Чтобы понять. Как это может быть, рассмотрим структуру в C-нотации.

```
struct DIR {  
    long d_ino;  
    long d_off;  
    unsigned short d_reclen;  
    char d_name[];  
    // char pad;  
    // char d_type;  
};
```

Структура интересна тем, что два последних ее поля закомментированы. На самом деле они есть, просто длина поля d_name не известна. Но общую длину можно получить исходя из поля d_reclen. Оно равно длине всей структуры. Другими словами, байт типа файла (d_type) находится от начала структуры на расстоянии d_reclen-1. Арифметика достаточно проста, но в программе получается несколько длинноватый код, что программистов на языке ассемблера совсем не удивляет.

```
# функция получения списка файлов, rdi - дескриптор открытого каталога  
# имена файлов и их тип выводится на стандартное устройство  
getfiles:  
push %rdi  
push %rsi  
push %rdx  
push %rbx  
push %rdx  
push %rcx  
push %r8  
push %r9  
push %r10  
push %r11
```

```

push %r12
# в r10 будет постоянно храниться дескриптор открытого каталога
mov %rdi, %r10
mov $buf, %rsi
# внешний цикл - чтение порций данных о файлах
loop1:
# получить порцию данных о файлах в каталоге
mov %r10, %rdi
mov $1024, %rdx
mov $78, %rax
syscall
cmp $1, %rax
jl ex # ошибка или данных нет
# запомним, что в rsi всегда начало буфера
# количество прочитанных байтов будет храниться в r9
mov %rax, %r9
# здесь будет копиться сдвиг относительно начала буфера
# указывающем на данные о файле
xor %rcx, %rcx
# внутренний цикл - печатаем полученные данные о файлах
loop2:
# -----
# начало буфера
mov %rsi, %rdx
# сдвиг в буфере, чтобы получить данные о следующем файле
add %rcx, %rdx
# получить длину структуры с данными о файле
xor %rbx, %rbx
mov %rdx, %r8
add $16, %r8
mov (%r8), %bx # в rbx искомая длина
# получить начало имени файла
mov %r8, %rdi
add $2, %rdi
# печать имени
call print
# печать типа файла

```

```

mov %rsi, %r12 # начало буфера
add %rbx, %r12 # длина текущей структуры
add %rcx, %r12 # длина предыдущих прочитанных данных
dec %r12      # байт типа файла
cmpb $8, (%r12)
jnz no1
mov $dt_reg, %rdi
jmp no7
no1:
cmpb $4, (%r12)
jnz no2
mov $dt_dir, %rdi
jmp no7
no2:
cmpb $1, (%r12)
jnz no3
mov $dt_fifo, %rdi
jmp no7
no3:
cmpb $12, (%r12)
jnz no4
mov $dt_sock, %rdi
jmp no7
no4:
cmpb $10, (%r12)
jnz no5
mov $dt_lnk, %rdi
jmp no7
no5:
cmpb $6, (%r12)
jnz no6
mov $dt_blk, %rdi
jmp no7
no6:
cmpb $2, (%r12)
jnz no8
mov $dt_blk, %rdi

```

```

no7:
call print
no8:
# перевод строки
call print_ent
# проверяем, не достигли ли мы, конца объема прочитанных байтов
# количество прочитанных байтов хранится в r9
add %rbx, %rcx # добавляем длину данных о последнем прочитанном
файле
cmp %rcx, %r9
jle ex1
jmp loop2
# -----
ex1:
jmp loop1
# выход
ex:
pop %r12
pop %r11
pop %r10
pop %r9
pop %r8
pop %rcx
pop %rdx
pop %rbx
pop %rdx
pop %rsi
pop %rdi
ret

```

Рис. 60. Функция получения списка файлов, указанного каталогам

На рисунке 61 представлен основной модуль программы. Мы видим, что с помощью директив `.include` в основную программу подключаются фрагменты, которые хранятся в других файлах. Все функции мы поместили в конце программы, но в действительности, можно было их разместить и сразу после указания сегмента `.text`, результат, естественно, (в работе программы) был бы тот же.


```

.data # сегмент данных
.include "data.inc"
.globl _start
.text
_start:
# вызов функции получения параметра с данным номером
# номер 1 - первый в строке параметров (второй если считать имя
программы)
# rsi адрес буфера, куда будет помещен параметр (имя каталога)
mov $2, %rdi
mov $buf, %rsi
call getpar
cmp $0, %rax # если параметра нет, то выходим
jz _exi
# обращаемся к каталогу
# откроем каталог
mov $buf, %rdi
mov $65536, %rsi # флаг, указывающий, что это каталог O_DIRECTORY
mov $2, %rax
syscall
cmp $0, %rax
jl _exi
push %rax
# читаем каталог
pop %rax
mov %rax, %rdi
call getfiles
# закрыть каталог
# mov %r8, %rdi # дескриптор файла
mov $3, %rax # номер системного вызова
syscall
_exi:
# закончить работу программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов
# область функций

```

```
.include "getfiles.inc"  
.include "getpar.inc"  
.include "print.inc"
```

Рис. 61. Основной модуль программы prog 37.s

Для трансляции программы достаточно выполнить команды

```
as prog37.s -o prog37.o  
ld -s prog37.o -o prog37
```

2.4.4. Управление памятью

2.4.4.1. Виды памяти

Для хранения данных в программах на языке ассемблера чаще всего используется глобальная память. Особенность глобальной памяти заключается в том, что мы резервируем определенный ее объем в самой программе. И этот объем резервируется при запуске исполняемого кода. Конечно, мы не можем его увеличить или уменьшить во время выполнения программы, но в большинстве случаев это не мешает писать эффективные программы. Конечно, в языках высокого уровня в настоящее время не принято злоупотреблять использованием глобальных переменных. Но нужно иметь в виду, что мы имеем дело с особым языком – ассемблером. В этом языке, например, условные и безусловные переходы являются одним из основных инструментов программирования. Так что и глобальные переменные будем воспринимать программирования на ассемблере как обычную технологию.

Программа на рисунке 62 демонстрирует пример массива строк. Строки выводятся одна за другой в обычном цикле. При этом сами строки индексируются массивом указателей на них. Конечно, представленная структура имеет существенный недостаток. Размеры строк зафиксированы и не могут быть увеличены. Но всегда можно пойти по пути резервирования достаточного объема для каждой из строк. Во многих случаях этого вполне достаточно.

```
#prog38.s  
.data  
# сегмент данных  
# структура, представляющая массив строк  
ar1:
```

```

.asciz "Строка 1\n"
ar2:
.asciz "Строка 2\n"
ar3:
.asciz "Строка 3\n"
ar4:
.asciz "Строка 4\n"
la:
.quad ar1
.quad ar2
.quad ar3
.quad ar4
.quad 0
.text
# сегмент кода
.globl _start
_start:
mov $la, %r8
loo:
mov (%r8), %rdi
call len
mov %rax, %rsi
mov (%r8), %rdi
call print
add $8, %r8
cmp $0, (%r8)
jnz loo
# выход
ex:
mov $60, %rax
xor %rdi, %rdi
syscall
# номер системного вызова exit
# код возврата (0 - выход без ошибок)
# системный вызов
# функция печати строки
print: # rdi - адрес строки, rsi - длина строки

```

```

# сохраним, используемые в функции регистры
push %rdx
push %rcx
push %r11
# печатаем строку
mov %rsi, %rdx# длина сообщения (второй параметр)
mov %rdi, %rsi# адрес строки
mov $1,%rdi# stdout
mov $1,%rax# номер системного вызова для вывода на консоль
syscall
# системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax
# возвращаем статус выполнения функции
ret
# функция получения длины строки в байтах
len: # rdi - адрес строки
xor %rax, %rax
push %rbx
push %rdi
lo1:
movb (%rdi), %bl
cmpb $0, %bl
jz ex1
inc %rdi
inc %rax
jmp lo1
ex1:
pop %rdi
pop %rbx
ret

```

Рис. 62. Пример массива строк

Несколько комментариев к рисунку 62.

1. Обращаем внимание на директиву `asciz`. Директива аналогична `ascii`, но отличается тем, что в конце строки компилятор автоматически добавляет байт, равный нулю. Это очень удобно, если условиться, что концом строки будет нулевой байт. Как раз наша функция `len` и реагирует на этот нулевой байт.

2. Массив указателей на строки начинается с метки `la`. При этом фиксирована не только длина строк, но и их количество в массиве. Впрочем, в этом нет ничего удивительного. Использование массивов с фиксированным количеством элементов обычная практика во многих языках высокого уровня.

Программа может быть откомпилирована последовательностью команд:

```
as --64 prog38.s -o prog38.o
ld -s prog38.o -o prog38
```

Еще одним типом памяти, которую может использовать программа на ассемблере это стековая память.

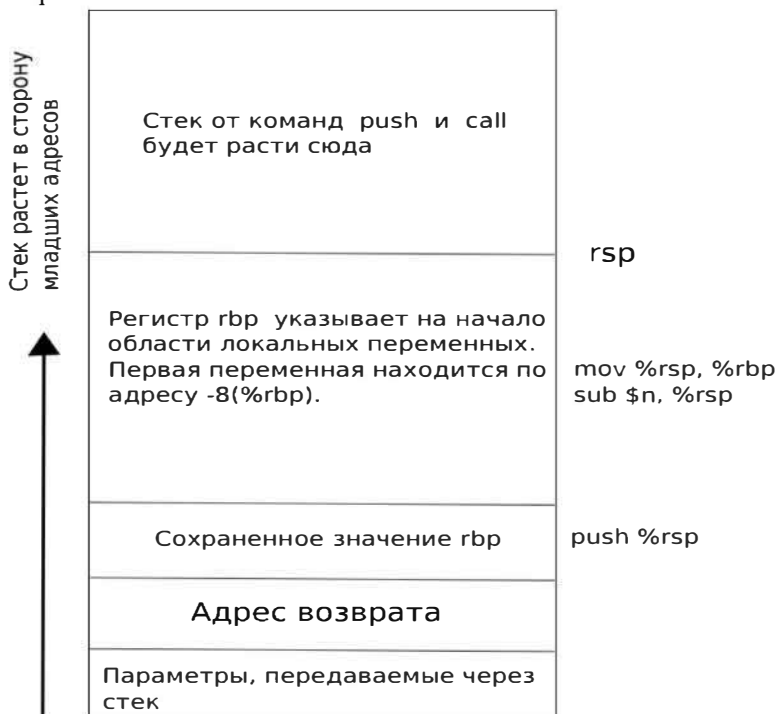


Рис. 63. Стек и локальные переменные в Linux 64

Обратимся к рисунку 63, где представлена схема стековой памяти при вызове функции.

1. Перед тем, как вызвать функцию, при необходимости, в стек отправляются параметры. Напомню, что в x86-64 Linux первые шесть параметров передаются через регистры (в отличие от Linux x86). Это шесть регистров rdi, rsi, rdx, rcx, r8, r9. Но вот если регистров не хватает, то тогда оставшиеся параметры передаются в функцию уже через стек. Так что это довольно редко, да и вообще современные подходы в программировании не рекомендуют большое количество параметров. К тому, можно передавать целые структуры путем передачи указателя на эти структуры.

2. Но в любом случае при вызове функции в стек кладется адрес возврата, который потом извлекается командой ret (см. рисунок 63).

3. Но вот далее действовать можно по-разному. Классический подход заключается в следующем: сохраняется в стеке регистр rbp, в него кладется значение rsp. После этого, значение rsp может продолжаться использоваться, т.е. в стек можно класть данные или вызывать другие функции. При этом rbp не изменяется и его можно использовать как для доступа к параметрам в стеке (если они есть), так и для доступа к локальным переменным. Но последнее возможно, если мы сдвинем в начале значение rsp в сторону младших адресов.

4. И так, если мы сдвинем значение rsp в сторону младших адресов, а уже потом будем использовать rsp для команд push, pop и call это означает, что при вызове функции мы фактически динамически выделили по необходимости некий объем памяти. При этом, если rsp не меняется или на данный момент равно значению верхней границы локальных переменных, то мы можем динамически менять размер этой памяти.

Когда запускается программа, то мы также имеем некую структуру стека²⁵, вершина которого лежит в rsp. В дальнейший рост стека будет происходить от этой вершины. Т.е., по сути, основной текст нашей программы можно считать текстом некоторой функции, которую можно назвать main. А далее для нее будет справедливым все то, что написано выше. Т.е. при запуске программы, мы можем при желании выделить нужный объем памяти в стеке для хранения данных (переменных). При необходимости объем этой памяти можно с осторожностью менять. Такие переменные можно было бы называть стековыми. В программе на рисунке 64 мы демонстрируем пример использования локальных переменных в основной программе. Программа суммирует числа от 1 до 100. При этом в локальных переменных хранится: накапливаемая сумма, количество оставшихся итераций и добавляемые числа.

²⁵ В сущности, это первая функция, с которой начинается выполняться программа

```

#prog39.s
.data
# сегмент данных
# глобальные переменные
buf:
.space 20, 0
.text
# сегмент кода
.globl _start
_start:
push %rbp
mov %rsp, %rbp
sub $24, %rsp
# стек готов к использованию локальных переменных
movq $0, -8(%rbp)
# здесь сумма
movq $100, -16(%rbp) # количество итераций
movq $1, -24(%rbp) # индекс
lo:
mov -8(%rbp), %rdi
mov -24(%rbp), %rsi
call addp
mov %rax, -8(%rbp)
incq -24(%rbp)
decq -16(%rbp)
jnz lo
# преобразуем число в строку и печатаем
mov -8(%rbp), %rdi
mov $buf, %rsi
call tostr
mov $buf, %rdi
call len
mov $buf, %rdi
mov %rax, %rsi
call print
# выход
add $24, %rsp

```

```

pop %rbp
ex:
mov $60, %rax
xor %rdi, %rdi
# номер системного вызова exit
# код возврата (0 - выход без ошибок)
syscall
# системный вызов
# область функций
addp:
mov %rdi, %rax
add %rsi, %rax
ret
# функция печати строки
print: # rdi - адрес строки, rsi - длина строки
# сохраним, используемые в функции регистры.
push %rdx
push %rcx
push %r11
# печатаем строку
mov %rsi, %rdx# длина сообщения (второй параметр)
mov %rdi, %rsi# адрес строки
mov $1,%rdi# stdout
mov $1,%rax# номер системного вызова для вывода на консоль
syscall
# системный вызов
# восстанавливаем содержимое всех регистров
pop %r11
pop %rcx
pop %rdx
xor %rax, %rax
# возвращаем статус выполнения функции
ret
# функция получения длины строки в байтах
len: # rdi - адрес строки
xor %rax, %rax
push %rbx

```



```

push %rdi
lo1:
movb (%rdi), %bl
cmpb $0, %bl
jz ex1
inc %rdi
inc %rax
jmp lo1
ex1:
pop %rdi
pop %rbx
ret
# преобразование числа в строку
tostr: # rdi - число, rsi - начало буфера
push %rdi
push %rsi
push %rdx
push %r8
push %r9
mov $10, %r8
mov %rdi, %rax
mov %rsi, %r9
lo3:
xor %rdx, %rdx
div %r8
add $48, %dl
mov %dl, (%rsi)
inc %rsi
cmp $0, %rax
jnz lo3
# теперь изменим порядок строки на противоположный
dec %rsi
# r9 на первом символе, rsi на последнем
lo4:
mov (%r9), %al
mov (%rsi), %dl
mov %dl, (%r9)

```

```

mov %al, (%rsi)
inc %r9
dec %rsi
cmp %rsi, %r9
jge ex2
jmp lo4
ex2:
pop %r9
pop %r8
pop %rdx
pop %rsi
pop %rdi
mov %rsi, %rax
ret

```

Рис. 64. Пример использования стековых переменных

Сделаем несколько пояснений к рисунку 64:

1. В программе демонстрируется использование трех переменных, память для которых мы зарезервировали в основной программе (sub \$24, %rsp).

2. Доступ ко всем трем переменным осуществляется посредством регистра rbp:

```

movq $0, -8(%rbp)# здесь сумма
movq $100, -16(%rbp)# количество итераций
movq $1, -24(%rbp)# индекс

```

3. Кроме использования стековых переменных, мы используем также стек по прямому назначению: вызов функций и системных функций. Конечно, те. Кто разобрался в том, как работает стек, в общем это нисколько не удивит. Но все-таки мы провели такой эксперимент для чистоты изложения.

Программа может быть откомпилирована с помощью команд:

```

as --64 prog39.s -o prog39.o
ld -s prog39.o -o prog39

```

2.4.4.2. Динамическая память и файлы отображаемые в памяти

В действительности есть и еще одна возможность использовать память. Можно программно в ходе выполнения выделять память для нужд

программы. Та часть памяти, которая позволяет получать ее для программы динамическим способом, называется кучей. При запуске программы куча (динамическая память) располагается между статической памятью и стеком.



Рис.65. Распределение памяти приложения в Linux 64

На рисунке 65 представлена схематическая карта памяти работающей программы. В ней выделены 4 области: исполняемая программа, статическая память, куча, стек. Мы несколько упростили всю схему, в частности статическая память и куча имеют чуть более сложную структуру, но для текущего же изложения это не имеет принципиального значения. Для нас важно вот что. Динамическая память в области кучи выделяется от края статической памяти в сторону больших адресов. Суть в том, что мы можем запрашивать сколько нам памяти нужно, указывая адрес начала не распределенной кучи. Для этого имеется системная функция с номером 12, которая называется `brk`.

На рисунке 66 представлена программа, демонстрирующая динамическое выделение памяти для использования программой.

```
.data
.text
# сегмент данных
# сегмент кода
.globl _start
_start:
# получить адрес начала области для выделения памяти
mov $12, %rax
xor %rdi, %rdi
```

```

syscall
# запомнить адрес начала выделяемой памяти
mov %rax, %r8
# выделить динамическую память
mov $2024, %rdi
add %rax, %rdi
mov $12, %rax
syscall
# обработка ошибки
cmp $-1, %rax
jz ex
# заполним выделенную память
mov $100, %dl
mov $0, %rbx
lo:
movb %dl, (%r8, %rbx)
inc %rbx
cmp $2024, %rbx
jz ex
jmp lo
# выход из программы
ex:
mov $60, %rax
xor %rdi, %rdi
syscall

```

Рис. 66. Динамическое выделение памяти

Поясним программу из рисунка 66:

1. Обратим внимание, что в программе дважды вызывается функция выделения памяти (`brk`). При первом вызове, когда ее аргумент равен нулю, функция возвращает адрес начала динамической памяти, откуда мы будем ее выделять.

2. Далее, программа делает вторичный вызов функции и указываем верхнюю границу запрашиваемой нами памяти. Если выделить не удалось, то функция возвращает `-1`. После этого, мы можем уже использовать полученную память, нижнюю и верхнюю границу мы знаем, в своих целях. Т.е. писать и читать оттуда данные. В нашем случае программа заполняет

выделенную память некоторым значением. Для проверки можно закомментировать второй вызов функции `brk` и убедиться, что будет сгенерирована системная ошибка.

3. В работе системной функции `brk` есть один важный нюанс. Память на самом деле выделяется не байтами, а страницами. По умолчанию страница имеет размер 4096 байтов. Если иметь это в виду, то количество вызовов функции `brk` в программе можно сократить, да и память сэкономить.

Программа может быть откомпилирована с помощью последовательности команд

```
as --64 prog40.s -o prog40.o
```

```
ld -s prog40.o -o prog40
```

Динамическая память очень удобна в плане работы с файлами. Она предоставляет технологию отображения файла в память, позволяющую упростить и ускорить обработку данных. Дело в том, что область памяти, которую мы называем кучей, делится еще на две части. Часть, ближайшая к стеку может быть использована для отображения файлов. Другая часть, как мы писали, может динамически выделяться программе (см. Рисунок 67).

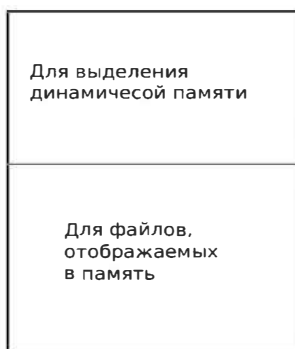


Рис. 67. Структура динамической памяти приложения в Linux 64

Суть технологии отображения файла в память заключается в следующем:

1. Обычным способом открывается файл.
2. Для его отображения (или отображения его части) используется функция `mmap` (номер 9). После чего открытый файл может быть закрыт.
3. После отображения программа получает адрес начала области отображения. Далее мы можем работать с этой областью, как обычной областью памяти. Т.е. читать и писать ее командами процессора.
4. По окончании работы с отображенным файлом, он закрывается с

помощью функции mmap (номер 11).

Работать с файлами, отображенными в память, чрезвычайно удобно и быстро. Описанная выше технология представлена в программе на рисунке 68.

```
#prog41.s
.data
# сегмент данных
fn:
.ascii "text.txt\0"
.text
# сегмент кода
.globl _start
_start:
# открыть файл
mov $fn, %rdi # адрес строки с именем файла
mov $2, %rsi # открываем для чтения и записи (O_RDWR)
mov $2, %rax # номер системного вызова
syscall
# вызов функции "открыть файл"
cmp $0, %rax # нет ли ошибки при открытии
jl ex
# перейти к концу программы
mov %rax, %rbx
# запомнить дескриптор файла
# получить длину файла
mov %rax, %rdi
# дескриптор файла
mov $0, %rsi
# от начала файла
mov $2, %rdx # SEEK_END
mov $8, %rax # номер системной функции
syscall
# вызов функции получить длину
cmp $0, %rax # нет ли ошибки при открытии
jl ex
# перейти к концу программы
mov %rax, %r15
```

```

# запомнить длину файла
# отобразить файл
mov $0, %rdi# с начала
mov %rax, %rsi# длина файла
mov $3, %rdx # PROT_WRITE|PROT_READ
mov $1, %r10 # MAP_SHARED
mov %rbx, %r8 # дескриптор файла
mov $0, %r9 # с начала файла
mov $9, %rax # номер системного вызова
syscall # вызов функции отобразить файл
cmp $0, %rax # нет ли ошибки
jl ex # перейти к концу программы
mov %rax, %r13
# запомнить адрес начала отображенного файла
# закрыть файл
mov %rbx, %rdi # дескриптор файла
mov $3, %rax # номер системного вызова
syscall # вызов функции "закрыть файл"
cmp $0, %rax # нет ли ошибки при открытии
jl ex
# перейти к концу программы
# изменить отображаемый файл
mov %r13, %rbx
# адрес начала области
mov %r15, %rdx
# обрабатываемая длина
lo:
mov (%rbx), %al
# получим байт
cmp $'i', %al # проверим байт
jnz lo1
movb $'1',(%rbx) # заменим байт
lo1:
inc %rbx # к следующему байту
dec %rdx # уменьшим счетчик
jnz lo # если не 0 - продолжим
# закрыть отображение

```

```

mov %r13, %rdi# адрес отображенного файла
mov %r15, %rsi # длина отображенной части файла
mov $11, %rax # номер системного вызова
syscall
cmp $0, %rax # нет ли ошибки
jl ex # перейти к концу программы
# выход из программы
ex:
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов

```

Рис. 68. Файл, отображаемый в память

Пояснение к программе из рисунка 68:

После отображения программа просматривает отображенный файл и заменяет буквы *i* на цифру 1. Поскольку технология работы с файлами, отображенными в память, выше была описана, а программа на рисунке 68 прокомментирована подробно, разберем только используемые функции `mmap` и `munmap`.

Программа может быть откомпилирована с помощью последовательности командами:

```

as --64 prog41.s -o prog41.o
ld -s prog41.o -o prog41

```

Функция `mmap`.

1-й параметр. Начальный адрес, куда предлагается отобразить файл. Обычно полагается 0, что означает, что система сама должна его выбрать.

2-й параметр. Задаёт длину отображаемого файла или его участка. В нашей программе мы взяли длину файла.

3-й параметр. Определяет уровень доступа к отображенному файлу. В нашей программе выбрано `PROT_WRITE|PROT_READ`, что означает, доступ будет и для чтения, и для записи.

4-й параметр. Определяет возможность доступа к участку других процессов. У нас взято `MAP_SHARED`, т.е. доступ разрешен. Впрочем, этот факт нами никак не используется.

5-й параметр. Дескриптор открытого нами файла. Как уже было сказано выше, после отображения, файл закрывается.

6-й параметр. Указывает смещение в файле, откуда будет произведено отображение. В нашем случае, мы отображаем весь файл,

значит этот параметр будет равен 0.

Функция `mmap`.

Функция имеет всего два параметра. Первый параметр адрес начала отображенного файла, второй параметр – длина отображения. Адрес, откуда начинается отображение, получается из функции `mmap`. Длина же отображения нами была задана равной длине открытого файла. В результате область отображения освобождается, а данные сбрасываются в указанный файл.

Использование функции `mmap` для работы с файлами, отображенными в память, не ограничивают ее возможности. Функция может быть использована просто для выделения памяти, как альтернатива функции `brk`.

Рассмотрим возможные подходы в ее использовании.

Если использовать функцию `mmap` с флагом `MAP_ANONYMOUS = 32`, то функция будет игнорировать, во-первых, дескриптор файла, обычно ему присваивают значение 1, во-вторых, последний аргумент – смещение внутри файла. В результате память указанного размера будет выделена, и функция возвратит адрес начала этой памяти, и она может быть использована по усмотрению данной программы.

Функция `mmap` допускает еще один способ, позволяющий выделять память без обращения к файлу, и без использования флага `MAP_ANONYMOUS`. Можно открыть устройство `/dev/zero`, как обычный файл. Полученный дескриптор использовать как дескриптор файла. Данный файл является бесконечным источником нулевых байтов. Туда же можно записывать бесконечно большие цепочки символов. Таким образом данное устройство, в сущности, может использоваться для создания фиктивного файла произвольного размера. Открыв этот файл и используя функцию `mmap` мы получим тот же результат, что и при использовании флага `MAP_ANONYMOUS`. В некоторых `unix`-подобных системах такой способ выделения динамической памяти с помощью функции `mmap` может оказаться единственным.

2.4.5. Управление процессами

Будут рассмотрены элементы многозадачности в операционной системе Linux 64 при программировании на ассемблере GAS.

2.4.5.1. Запуск процессов и создание процессов

При загрузке приложения в память операционная система создает объект под названием процесс. В него входят всевозможные атрибуты и структуры, хранящиеся в ядре, а также виртуальное адресное пространство, куда загружается код программы. Кроме этого объект – процесс хранит еще и окружение, передаваемое при запуске. Можно сказать, процесс – это запущенный экземпляр приложения.

Процесс, однако, это статический объект. Для хранения данных о выполнении кода система создает также еще один объект, называемый потоком. Один поток создается всегда автоматически после создания процесса, он называется главным или первичным потоком. Первичный поток может создавать еще потоки, которые в свою очередь создавать потоки еще. Все потоки существуют в одном виртуальном адресном пространстве, т.е. разделяют одну память – могут пользоваться одними и теми же данными.

Разберем создание процесса путем программного запуска исполняемого модуля. Для запуска исполняемого модуля используется системный вызов с номером 59 (`execve`). Особенность запуска программы таким образом заключается в том, что запущенная программа полностью замещает процесс, который запускает данную программу. Это важный момент, который будем учитывать в дальнейшем. Чтобы управлять запускаемым процессом, нужен механизм, позволяющий запускать параллельный процессы, но это будем обсуждать в следующем параграфе.

Ниже представлена программа, которая запускает указанный исполняемый модуль с возможностью указания параметров, используемых данной программой. Программа состоит из двух модулей. В модуле из рисунка 69 представлены вспомогательные функции: `getpar` – получить параметр командной строки, `_exit` – закончить работу программы. Заметим, что функция `getpar` имеет два входных параметра: номер получаемого параметра и адрес буфера, куда параметр должен быть помещен. Если параметра с данным номером нет, то функция возвращает 0.

```
#getpar.s
.text
.globl getpar, _exit
getpar:
push %rbp
mov %rsp, %rbp # вершина стека
```

```

# rbp указывает на структуру параметров с учетом изменения
# стека (вызов +команда push)
add $16, %rbp
# обеспечиваем сохранность все регистров
push %r8
push %r9
push %rbx
push %rdi
push %rsi
xor %rax, %rax # в конце или длина параметра или 0, если параметра нет
mov (%rbp), %r9 # всего параметров
lo:
add $8, %rbp
# следующий элемент стека (параметр командной строки)
dec %rdi
# пока счетчик ни обнулится
jz lo3
dec %r9
# декремент счетчика общего количества параметров
jz lo2
jmp lo
lo3:
mov (%rbp), %r8 # адрес начала строки – параметра
lo1:
mov (%r8, %rax), %bl # берем байт
mov %bl, (%rsi) # копируем в буфер
inc %rsi
inc %rax
# следующее место в буфере
# увеличиваем счетчик байтов
cmpb $0, (%r8, %rax) # конец строки?
jnz lo1
# продолжим копировать
movb $0, (%rsi) # в конце строки поставим 0 (на всякий случай)
inc %rax
# увеличить счетчик
lo2:

```

```

# восстановление регистров
pop %rsi
pop %rdi
pop %rbx
pop %r8
pop %r9
pop %rbp
ret
# функция выхода из программы
_exit:
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов

```

Рис. 69. Модуль getpar.s

На рисунке 70 основная часть программы запуска исполняемого модуля. Прежде чем разбирать эту программу, обратимся к описанию системной функции `execve`.

Системная функция `execve` имеет три параметра:

1. Полное имя запускаемого файла, например `/bin/ls`, `./prog1` и т. д. Это может быть двоичный исполняемый файл или скрипт, в первой строке которого указано как этот скрипт должен запускаться. Структура первой строки такого файла следующая: `#!запускающая программа [возможный параметр]`.

2. Массив, состоящий из указателей на строки. Последний элемент массива должен быть нулем. По умолчанию первая строка – имя запускаемой программы. Дальше могут идти другие параметры. Но это не обязательно, т.е. первая строка может содержать и другую информацию. Если параметров нет, первая строка должна обязательно присутствовать.

3. Массив, состоящий из указателей на строки окружения. Каждая такая строка имеет структуру `переменная=значение`. Если третий параметр равен нулю (`NULL`), то это означает, что запускаемая программа имеет то же окружение, что и запускающая. Чаще всего выбирается именно этот вариант.

Программа, представленная на рисунке 70, запускает исполняемые модули имена и параметры которых указаны в командной строке как параметры.

```

#prog42.s
.data # сегмент данных
# для строк
nm:
.space 200, 0
nm1:
.space 200, 0
nm2:
.space 200, 0
nm3:
.space 200, 0
nm4:
.space 200, 0
# указатели на строки
nn1:
.quad 0
nn2:
.quad 0
nn3:
.quad 0
nn4:
.quad 0
nn5: # конец массива NULL
.quad 0
.text # сегмент кода
.globl _start
_start:
# заполняем массив строк параметра
# первый параметр
mov $1, %rdi # номер параметра
mov $nm, %rsi # адрес буфера
call getpar
cmp $0, %rax # получен ли параметр, если нет то другие не смотрим
jz exi
# второй параметр
mov $2, %rdi # номер параметра
mov $nm1, %rsi # адрес буфера

```

```

call getpar
cmp $0, %rax # получен ли параметр, если нет то другие не смотрим
jnz l11
mov $nm, %rax
mov %rax, nn1
jmp l1
l11:
mov $nm1, %rax
mov %rax, nn1
# третий параметр
mov $3, %rdi # номер параметра
mov $nm2, %rsi # адрес буфера
call getpar
cmp $0, %rax # получен ли параметр, если нет, то другие не смотрим
jz l1
mov $nm2, %rax
mov %rax, nn2
# четвертый параметр
mov $4, %rdi # номер параметра
mov $nm3, %rsi # адрес буфера
call getpar
cmp $0, %rax # получен ли параметр, если нет, то другие не смотрим
jz l1
mov $nm3, %rax
mov %rax, nn3
# пятый параметр
mov $5, %rdi # номер параметра
mov $nm4, %rsi # адрес буфера
call getpar
cmp $0, %rax # получен ли параметр, если нет, то другие не смотрим
jz l1
mov $nm4, %rax
mov %rax, nn4
l1:
# запуск системной функции exesve
mov $nm, %rdi # строка запуска программы
mov $nn1, %rsi # указатель на массив указателей

```

```

mov $0, %rdx # указатель на массив окружения NULL
mov $59, %rax
syscall
# закончить работу программы
exit:
call _exit

```

Рис. 70. Программа запуска исполняемых модулей

Пояснение к программе из рисунка 70.

1. В программе принята следующая структура данных. `nm` – указывает на первый параметр `execve`. Далее идут метки `nn1-nn4`, где будут размещены параметры для массива, который идет вторым параметром данной системной функции. `nn1-nn5` – указатели на строки, т.е. массив указателей, который и будет передавать в системную функцию. `nn5` – всегда будет содержать 0. Таким образом количество элементов в массиве не может превышать 5, а количество строк не более 4.

2. Далее идет блок получения параметров. Здесь следует отметить, что в отсутствие первого параметра в командной строке приводит сразу к 249окончанию работы программы, так запуск ей не чего. В случае одного параметра в командной строке, второй элемент массива будет показывать на полное имя программы. Т.е. два первых элемента массива показывают на одну и ту же строку. Для всех остальных параметров командной строки используется принцип: если параметра нет, то соответствующий элемент массива будет равен нулю (см. рисунок 69).

3. При запуске `execve` ее параметры, таким образом равны: значение `nm` (адрес полного имени запускаемой программы, `nn1` – начала массива указателей на строки, 0 – последний параметр.

Для трансляции программы выполняется последовательность команд

```

as --64 prog42.s -o prog42.o
as --64 getpar.s -o getpar.o
ld getpar.o prog42.o -o prog42

```

Примеры запуска программы

```

./prog42 /bin/ls -al /etc
./prog42 /bin/ls ls -help
./prog42 /bin/ps ps -A

```

и т.д.

Рассмотренная нами функция `execve` замещает запускающее приложение новым процессом. При этом старое приложение просто теряется. Конечно, хотелось бы иметь механизм, позволяющий не только

запускать другие приложения, но сохранять и старые, более того, иметь возможность управлять запущенными приложениями. Такие механизмы будут изложены ниже.

Для того, чтобы создать параллельный процесс, назовем его дочерним, поскольку его порождает другой процесс, существует системная функция `fork`, номер которой равен 57. Функция не имеет параметров, но не проста для понимания. Она сразу возвращает управление. При этом возникает и дочерний процесс. Дочерний процесс наследует: 1. Определенные до этого переменные. 2. Код, начиная с вызова функции `fork`. При этом в дочернем процессе функция возвращает 0, а в родительском процессе идентификатор дочернего процесса. Надо иметь в виду, что у дочернего процесса своя память. Но она работает с образом, полученным от родительского процесса. Чтобы несколько сократить код программы, для вспомогательных функций вывода на консоль мы используем язык C (см. рисунок 71).

```
#include <stdio.h>
void printn(long long n){
    printf("%lld\n",n);
}
void prints(char* s){
    printf("%s",s);
}
```

Рис. 71. Вспомогательные функции на языке C для программы из рисунка

72

Программа на рисунке 72 запускает дочерний процесс и заканчивает свою работу. При этом дочерний процесс продолжает работать с точки вызова функции `fork`. По сути, тот факт, что родительский процесс заканчивает свою работу никак не влияет на работу дочернего процесса. Просто если родительский процесс заканчивает работу раньше, то для дочернего процесса родительским становится их прародитель в операционной системе Linux – `systemd`. Конечно, родительский процесс может влиять на дочерний, отслеживать его работу, но эти вопросы будут рассмотрены в следующем параграфе. Может возникнуть следующий вопрос: если дочерний процесс унаследует код родительского, то в коде можно различить, выполняется родительский или дочерний. Но вспомним, что в дочернем процессе `fork` возвращает 0, а в родительском – идентификатор дочернего. Именно это мы и учитываем в программе из

рисунка 72.

```
.data # сегмент данных
res:
.quad 0
a:
.quad 0
tx1:
.asciz "До "
tx2:
.asciz "После "
tx3:
.asciz "Дочерний "
tx4:
.asciz "Родительский "
tx5:
.asciz "Ошибка\n"
.text # сегмент кода
.globl main
main:
# выполняется только в родительском процессе
movq $89, a
mov $tx1, %rdi
call prints
mov a, %rdi
call printn
# создаем дочерний процесс
mov $57, %rax
syscall
# системный вызов fork
mov %rax, res
cmpq $-1, res
jnz l1
# сообщение об ошибке
mov $tx5, %rdi
call prints
jmp exi
l1:
```

```

# выполняется и в родительском и дочернем процессе
mov $tx2, %rdi
call prints
mov a, %rdi
call printn
# определяем где выполнять
cmpq $0, res
jnz l3
# -----
# ВЫПОЛНЯЕТСЯ В ДОЧЕРНЕМ ПРОЦЕССЕ
# -----
# получить идентификатор дочернего процесса
mov $39, %rax # getpid текущий процесс
syscall
mov %rax, %r14
mov $tx3, %rdi #дочерний
call prints
mov %r14, %rdi
call printn
# получить идентификатор родительского процесса
mov $110, %rax # getppid родительский процесс
syscall
mov %rax, %r14
mov $tx4, %rdi #родительский
call prints
mov %r14, %rdi
call printn
jmp exi
# эта ветка выполняется в родительском процессе
l3:
# -----
# ВЫПОЛНЯЕТСЯ В РОДИТЕЛЬСКОМ ПРОЦЕССЕ
# -----
# получить идентификатор родительского процесса
mov $39, %rax # getpid
syscall
mov %rax, %r14

```

```

mov $tx4, %rdi # родительский
call prints
mov %r14, %rdi
call printn
mov $tx3, %rdi # дочерний
call prints
mov res, %rdi
call printn
exi:
# выход из программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов

```

Рис. 72. Программа запуска исполняемых модулей

Пояснения к программе из рисунка 72.

Программа на рисунке 72 является своего рода экспериментальной.

По ее работе можно понять, как происходит разделение двух процессов: родительского и дочернего.

1. Обратим внимание на то место в программе, где вызывается системная функция `fork`. Именно с этого момента начинают параллельно работать два идентичного кода. Обратим внимание на переменную `a`. Ее значение задается в родительском коде. Вывод строки До 89 Происходит только один раз, поскольку этот вывод осуществляется только в родительском процессе. С другой стороны, вывод строки После 89 происходит дважды в родительском и дочернем процессе. При этом значение переменной в обоих процессах одинаково.

2. Обращаем внимание на последовательность строк

```

cmpq $0, res
jnz l3

```

Именно в этой точке программы происходит «раздвоение кода» на код, который выполняется в родительском процессе (переход на метку `l3`) и на код, который выполняется в дочернем процессе (после указанных строк). Обратим также внимание на две системные функции `getpid` (номер 39) – получить идентификатор текущего процесса и `getppid` (номер 110) – получить идентификатор родительского процесса. При этом в дочернем 255процессе получаем оба идентификатора с помощью функций `getpid` и `getppid`, а в родительском процессе с помощью функции `getpid` и переменного

res, где хранится значение, возвращенное функцией fork – идентификатор дочернего процесса.

Для трансляции программы используем последовательность команд:

```
as --64 prog44.s -o prog44.o
```

```
gcc -c prog43.c
```

```
gcc -no-pie prog43.o prog44.o -o prog44
```

Пример результата работы данной программы:

До 89

После 89

Родительский 11696

Дочерний 11697

После 89

Дочерний 11697

Родительский 11696

Как видим, идентификаторы, полученные из дочернего и родительского процесса, совпадают. Однако при нескольких запусках с некоторой вероятностью оказывается, что у родительских процессов идентификаторы разные. И это очень важный результат. Происходит следующее. Родительский процесс заканчивает свою работу, а дочерний процесс еще работает. При этом родительским процессом для него становится системный процесс `systemd`. Вот именно идентификатор этого процесса выводится дочерним процессом на консоль.

2.4.5.2. Взаимодействие процессов

Рассмотрим частный вопрос, связанный с взаимодействием родительского и дочернего процесса. В некоторых случаях родительскому процессу важно знать, что дочерний процесс закончил свою работу. Для этого, в частности, можно использовать системную функцию `wait4` (номер 61). Приводим простой пример запуска дочернего процесса, в котором запускается приложение и ожидание родительского процесса, когда дочерний процесс закончит свою работу. Для простоты для запуска была взята системная утилита Linux `ls`, которая рекурсивно осуществляет поиск по каталогам, начиная с данного. Параметра заданы непосредственно в тексте программы (см. Рисунок 73). Чтобы позволить разделить вывод на консоль их дочернего и родительского процессов, родительский процесс будет осуществлять вывод в поток ошибок. На рисунке 74 представлены вспомогательные функции, написанные на языке C, которые используются в основной программе.

```
.data # сегмент данных
res:
.quad 0
er:
.asciz "Ошибка\n"
mess:
.asciz "Окончание работы дочернего процесса\n"
comm:
.asciz "/bin/ls"
par1:
.asciz "-R"
par2:
.asciz "/var"
arr:
.quad comm
.quad par1
.quad par2
.quad 0
status:
.quad 0
.text # сегмент кода
```

```

.globl main
main:
# создаем дочерний процесс
mov $57, %rax
syscall
# системный вызов fork
mov %rax, res
cmpq $-1, res
jnz l1
# сообщение об ошибке
mov $er, %rdi
call prints
jmp exi
l1:
# определяем где выполнять
cmpq $0, res
jnz l3
# -----
# ВЫПОЛНЯЕТСЯ В ДОЧЕРНЕМ ПРОЦЕССЕ
# -----
# запуск системной функции exesve
mov $comm, %rdi # строка запуска программы
mov $arr, %rsi # указатель на массив указателей
mov $0,%rdx # указатель на массив окружения NULL
mov $59,%rax
syscall
jmp exi
# эта ветка выполняется в родительском процессе
l3:
# -----
# ВЫПОЛНЯЕТСЯ В РОДИТЕЛЬСКОМ ПРОЦЕССЕ
# -----
#
call tim
mov $res, %rdi
mov $status, %rsi
mov $1, %rdx # WNOHANG

```

```

mov $0, %r10
mov $61, %rax
syscall
#
mov $mess, %rdi
call fprints
exi:
# выход из программы
mov $60, %rax # номер системного вызова exit
xor %rdi, %rdi # код возврата (0 - выход без ошибок)
syscall # системный вызов

```

Рис. 73. Пример ожидания выполнения дочернего процесса

Пояснения к рисунку 73.

1. Структура программы аналогична программе из предыдущего параграфа. Но параметры запуска здесь зашиты непосредственно в программном тексте (см. блок строк `comt` и массив указателей `arg` на него).

2. В программе есть один важный нюанс. При создании процесса родительский процесс несколько опережает дочерний процесс. Чтобы функция `wait4` при запуске обнаружила запущенный дочерний процесс или его окончание, в родительском процессе запускается функция `tim` которая реализует системное ожидание длиной в одну секунду. Таким образом родительский процесс в любом случае обнаруживает конец дочернего процесса. Если дочерний процесс выполняется, то функция `wait4` ждет окончания его выполнения, а потом возвращает управление. Если дочерний процесс уже закончился, то `wait4` сразу заканчивает свою работу. Таким образом сообщение об окончании работы дочернего процесса покажет действительное его окончание.

3. Системная функция `wait4` имеет параметр. Первым параметром, в частности можно указать идентификатор дочернего процесса. Он у нас хранится в переменной `res`. Вторым параметром является адрес переменной, куда помещается статус дочернего процесса. В нашем случае мы статус не анализируем, поскольку нам важно только, что дочерний процесс работу закончил. Третий параметр определяет условие возвращения из функции `wait4`. В нашей программе принято условие, что возврат осуществляется, если дочерний процесс закончил работу. Четвертый параметр является указателем на структуру, куда будет возвращена информация о дочерних процессах. Указатель может быть

равен NULL (0), этим мы и воспользовались в нашей программе.

```
#include <stdio.h>
#include <time.h>
void prints(char* s){
printf("%s",s);
}
void fprints(char* s){
fprintf(stderr, "%s",s);
}
void tim(){
struct timespec tm;
struct timespec tm1;
tm.tv_sec=1;
tm.tv_nsec=0;
nanosleep(&tm, &tm1);
}
```

Рис. 74. Вспомогательные функции на языке С к программе из рисунка 73

Для трансляции программы следует выполнить последовательность команд.

```
as --64 prog45.s -o prog45.o
gcc -c prog46.c
gcc -no-pie prog46.o prog45.o -o prog45
```

Пример запуска программы

```
./prog45 > lls
```

При этом весь вывод дочернего процесса пойдет в файл lls, а родительский процесс выведет все в поток ошибок, т.е. на консоль.

Существует еще ряд механизмов [18], позволяющих обмениваться данными между процессами

1. Запущенное приложение имеет доступ к окружению²⁶ и может использовать данные, которые в нем хранятся. Если приложение создает другое приложение, например, с помощью системной функции `execve`, то третьим параметром можно указать на свое окружение, необходимое для передачи данных дочернему процессу. В том же случае, если третий параметр функции равен `NULL`, дочернему процессу передается окружение родительского процесса. Дочерний процесс имеет доступ к окружению и может использовать данные, которые в нем хранятся. Если приложение создает другое приложение, например, с помощью системной функции `execve`, то третьим параметром можно указать на свое окружение, необходимое для передачи данных дочернему процессу. В том же случае, если третий параметр функции равен `NULL`, дочернему процессу передается окружение родительского процесса.

2. Ранее мы рассматривали память, которую можно динамически выделить с помощью системной функции `mmap`. В частности, эту память можно сделать разделяемой между процессами. Тогда мы получаем очень удобный механизм для обмена данными между такими процессами. Напоминаю, что при выделении памяти с помощью `mmap` используется опция `MAP_SHARED` равная 1. Родительский процесс создав разделяемую память может передать адрес этой памяти дочерним процессам.

3. Еще одним механизмом обмена данными между процессами являются каналы, по-английски `pipes`. Это объекты ядра, очень похожие на файлы, но обладающий серьезным от них отличием. Суть отличия в том, что при чтении из канала содержимое его удаляется. Таким образом канал очень похож на трубу, через которую «течет» информация. Использовать

²⁶ Окружением в операционных системах Linux называется набор строк, имеющих структуру вида ПЕРЕМЕННАЯ=ЗНАЧЕНИЕ. Приложение получает набор таких строк от родительского процесса, а также может формировать свое окружение для передачи дочернему процессу.

один объект `pipe` для двустороннего обмена проблематично. В любом случае понадобится дополнительное средство синхронизации. Иначе тот, кто написал в `pipe`, может прочесть свое же сообщение, и оно не дойдет до адресата. Есть еще одно важное отличие каналов от файлов. При создании канала программа получает два дескриптора (4-байтового числа): для чтения и записи. Дескрипторы будут располагаться в памяти друг за другом, в начале для чтения, потом для записи. При организации взаимодействия между процессами с помощью объекта `pipe` следует иметь в виду, что данные передаются не большими порциями. Поэтому передавать информацию лучше не большими объемами, известными принимающей стороне. Если объект `pipe` открыт в родительском процессе, то при создании дочернего процесса ему передается и таблица открытых объектов. Таким образом, дочерние процессы наследуют объекты `pipe` и получают возможность обмениваться информацией и с родительским процессом и друг с другом.

4. Синхронизация процессов одна из самых важных проблем многозадачности. Эта задача возникает, когда процессы, в частности, работают с одним и тем же ресурсом. Одним из удобных механизмов синхронизации в операционной системе Linux являются семафоры. Как работает семафор? Семафор – это объект ядра. Представим себе, что после его создания к нему имеют доступ два процесса. Значение счетчика в начале равно 1. Через специальную функцию доступ получает один и значение счетчика уменьшается до 0. Второй процесс, обратившись к этой же функции будет находиться в состоянии ожидания. Когда первый процесс закончит свою работу, он, обратившись к той же функции (с другими параметрами) увеличивает значение счетчика до 1. Тогда с ресурсом начинает работу второй процесс и пока он работает ресурс будет заблокирован для первого. В результате семафор не позволит двум процессам работать одновременно с одним ресурсом.

В заключении главы отметим следующее:

1. Гибкие возможности языка GAS ([86-64]) позволяют писать системные и прикладные программы для операционной системы Linux.

2. Ассемблер легко интегрируется в программы на языках высокого уровня. Интеграция возможна как при использовании языков высокого уровня в программах на языке ассемблера, так и в обратном случае: использование ассемблера в программах на языках высокого уровня. При этом можно использовать как объектные модули (объектные статические библиотеки), так и динамические библиотеки.

3. Мощный интерфейс системных вызовов позволяет создавать программное обеспечение управления операционными системами.

ЗАКЛЮЧЕНИЕ

Нами были рассмотрены ряд вопросов, связанных с низкоуровневым программированием в среде операционной системы Linux на основе архитектуры x86-64. В качестве основного инструмента были взяты стандартные утилиты операционной системы: ассемблер GAS и компоновщик ld. В частности бы ли рассмотрены вопросы:

1. Структура программа на языке ассемблера GAS для операционной системы Linux.
2. Особенности архитектура x86-64 применительно к операционной системе Linux.
3. Основы совместного использования языка ассемблера с языками высокого уровня.
4. Элементы программного управления файловой системой Linux.
5. Управление памятью в программах на языке ассемблера.
6. Основы управления процессами.

Конечно, в издании небольшого объема невозможно охватить многие другие вопросы, связанные с программирование на языке ассемблера в операционной системе Linux: сетевое программирование, отладка программ, графические библиотеки и многое другое. Мы надеемся в будущем расширить рассматриваемые вопросы в следующих книгах по низкоуровневому программированию в среде Linux.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. AMD64 Technology. AMD64 Architecture Programmer's Manual. Vol. 1. Application Programming. – Advanced Micro Devices, 2009. – Text : direct.
2. AMD64 Technology. AMD64 Architecture Programmer's Manual. Vol. 2. System Programming. – Advanced Micro Devices, 2010. – Text : direct.
3. AMD64 Technology. AMD64 Architecture Programmer's Manual. Vol. 3. General-Purpose and System Instructions. – Advanced Micro Devices, 2009. – Text : direct.
4. AMD64 Technology. AMD64 Architecture Programmer's Manual. Vol. 4. 128-Bit and 256-Bit Media Instructions. – Advanced Micro Devices, 2010. – Text : direct.
5. AMD64 Technology. AMD64 Architecture Programmer's Manual. Vol. 5. 64-Bit Media and x87 Floating-Point Instructions. – Advanced Micro Devices, 2009. – Text : direct.
6. AMD64 Technology. AMD64 Architecture Programmer's Manual. Vol. 6. 128-Bit and 256-Bit XOP and FMA4 Instructions. – Advanced Micro Devices, 2009. – Text : direct.
7. Hall, B.R. Assembly Programming and Computer Architecture for Software Engineers / Brian R. Hall, Kevin J. Slonka. – New York : Prospect Press, 2018. – 402 p. – Text : direct.
8. Calling Convention. – Text : electronic // MSDN : [web-site]. – URL: <http://msdn.microsoft.com/en-us/library/9b372w95.aspx> (accessed: 13.03.2021).
9. Eagle, Ch. The Ida Pro book / Eagle Chris. – San Francisco : No Stach Press, 2008. – 676 p. – Text : direct.
10. Ed Jorgensen, Ph.D. x86-64 Assembly Language Programming with Ubuntu/Ed Jorgensen : University of Nevada, 2019. – 343 p. – Text: direct.
11. Exceptional Behavior - x64 Structured Exception Handling. – Text : electronic // The NT Insider. – 2006. – Vol. 13, is. 3 (May-June). – URL: <http://www.osronline.com/article.cfm?article=469> (accessed: 13.03.2021).
12. Fog, A. Calling conventions for different C++ compilers and operating systems / Agner Fog. – Kongens Lyngby : Technical University of Denmark, 2014. – 57 p. – Text: direct.
13. GCC online documentation ; [web-site]. – URL: <http://gcc.gnu.org/onlinedocs> (accessed: 13.03.2021).
14. Intel Corporation. Intel Itanium Processor : manuals. – URL: <http://developer.intel.com/design/itanium/manuals.htm> (accessed: 13.03.2021). – Text : electronic.

15. Intel® 64 and IA-32 Architectures Optimization Reference Manual. – 2010. – September. – Text : direct.
16. Intel® 64 and IA-32 Architectures Software Developer's Manual. Vol. 2B. – 2010. – Text : direct.
17. Intel® 64 and IA-32 Architectures Software Developer's Manual. Vol. 2A. – 2010. – Text : direct.
18. Intel® 64 and IA-32 Architectures Software Developer's Manual. Vol. 1. – 2010. – Text : direct.
19. Intel® 64 and IA-32 Architectures Software Developer's Manual. Vol. 3A. – 2010. – Text : direct.
20. Intel® 64 and IA-32 Architectures Software Developer's Manual. Vol. 3B. – 2010. – Text : direct.
21. Bartlett, J. Learn to Program with Assembly: Foundational Learning for New Programmers / Jonathan Bartlett. – New York : Apress, 2021. – 328 p. – Text : direct.
22. Bartlett, J. Programming from the Ground Up / Jonathan Bartlett. – Indianapolis : Dominick Bruno, Jr., 2003. – 319 p. – Text : direct.
23. Lazarus documentation : [web-site]. – URL: http://wiki.lazarus.freepascal.org/Lazarus_Documentation (accessed: 13.03.2021). – Text : electronic.
24. Mashey, J. R. The Long Road to 64 Bits / Joohn R. Mashey. – Text : direct // Magazine “Queue”. – 2006. – Vol. 4, is. 8 (October). – P. 45-53.
25. Pietrek, M. A Crash Course on the Depths of Win32™ Structured Exception Handling / Matt Pietrek. – Text : electronic // Microsoft system journal. – 1997. – January. – URL: <http://www.microsoft.com/msj/0197/exception/exception.aspx> (accessed: 13.03.2021).
26. Randall, H. The art of assembly language / Hyde Randall. – 2nd edition. – San Francisco : William Pollock, 2010. – 764 p. – Text : direct.
27. Running 32-bit Applications. – Text : electronic // MSDN : [web-site]. – URL: [http://msdn.microsoft.com/en-us/library/aa384249\(VS.85\).aspx](http://msdn.microsoft.com/en-us/library/aa384249(VS.85).aspx) (accessed: 13.03.2021).
28. Scott, W. 64-bit computing in theory and practice / Wasson Scott. – Text : electronic // Techreport : [web-site]. – URL: <https://techreport.com/review/8131/64-bit-computing-in-theory-and-practice/> (accessed: 12.10.2022).
29. Why did the Win64 team choose the LLP64 model? – Text : electronic // MSDN Blogs : [web-site]. – URL: <http://blogs.msdn.com/b/oldnewthing/archive/2005/01/31/363790.aspx>

(accessed: 13.03.2021).

30. Williams, R. N. A Painless Guide to CRC Error Detection Algorithms / Ross N. Williams. – Adelaide : Rocksoft Pty Ltd., 1993. – 36 с. – Текст : direct.

31. Александреску, А. Современное проектирование на C++. Обобщенное программирование и прикладные шаблоны проектирования / Андрей Александреску. – Москва : Вильямс, 2019. – 336 с. – Текст : непосредственный.

32. Библиотека MSDN корпорации Microsoft. – Текст : электронный // MSDN : [веб-сайт]. – URL: <http://msdn.microsoft.com/ru-ru/ms348103> (дата обращения: 13.03.2021).

33. Взгляды Microsoft по поводу 64-битного будущего. – Текст : электронный // Softodrom : [веб-сайт] – URL: <http://news.softodrom.ru/ap/b1803.shtml> (дата обращения: 13.03.2021).

34. Глушаков, С.В. Язык программирования C++. Учебный курс. / С.В. Глушаков, А.В. Коваль, С.В. Смирнов. – Москва : Фолио-Аст, 2003. – 726 с. – Текст : непосредственный.

35. Дейкстра, Э. В. Доводы против оператора goto / Эдсгер В. Дейкстра. – Текст : электронный // Communications of the ACM. – 1968. – Vol. 11, № 3 (March). – P. 147-148. – URL: <http://khp-iip.mipk.kharkiv.edu/library/extent/dijkstra/pp/ewd215.html> (дата обращения: 13.03.2021).

36. Дейтел, Х. Как программировать на C++ / Харви Дейтел, Пол Дейтел. – Москва : Бином, 2021. – 1032 с. – Текст : непосредственный.

37. Зубков, С.В. Assembler для DOS, Windows и Unix / С.В. Зубков. – Москва : БХВ, 2017. – 638 с. – Текст : непосредственный.

38. Из истории корпорации Intel. – Текст : электронный // TimesNet : [веб-сайт]. – URL: <http://timesnet.ru/success/156/> (дата обращения: 13.03.2021).

39. Инициативы Intel в образовании. – Текст : электронный // Intel : офиц. сайт корпорации. – URL: <http://www.intel.ru/content/www/ru/ru/education/elementary/programs.html> (дата обращения: 13.03.2021).

40. Касперски, К. Искусство дизассемблирования / Крис Касперски, Ева Роко. – Санкт-Петербург : БХВ, 2008. – 896 с. – Текст : непосредственный.

41. Касперски, К. Образ мышления – дизассемблер IDA / Крис Касперски. – Москва : СОЛОН-Пресс, 2004. – 425 с. – Текст : непосредственный.

42. Касперски, К. Техника оптимизации под Linux / Крис Касперски.

– Текст : непосредственный // Системный администратор. – 2005. – № 2, 3, 4.

43. Касперски, К. Техника оптимизации программ. Эффективное использование памяти / Крис Касперски. – Санкт-Петербург : БХВ, 2003. – 464 с. – Текст : непосредственный.

44. Касперски, К. Фундаментальные основы хакерства / Крис Касперски. – Москва : СОЛОН-Пресс, 2004. – 447 с. – Текст : непосредственный.

45. Керниген, Б. Язык программирования C / Б. Керниген, Д. Ритчи. – 3-е изд. – Санкт-Петербург : Диалект, 2020. – 288 с. – Текст : непосредственный.

46. Майерс, С. Эффективное использование C++ / Скот Майерс. – Москва : ДМК, 2006. – 300 с. – Текст : непосредственный.

47. Массовая 64-битовая платформа: туманные перспективы. – Текст : электронный // ИТС : [веб-сайт]. – URL: http://itc.ua/articles/massovaya_64-bitovaya_platforma_tumannye_perspektivy_21077 (дата обращения: 13.03.2021).

48. Intel Corporation : офиц. сайт. – URL: http://www.intel.com/?en_US_01 (дата обращения: 13.03.2021). – Текст : электронный.

49. Официальный сайт разработчиков ассемблера fasm : [веб-сайт]. – URL: <http://flatassembler.net/> (дата обращения: 12.01.2023). – Текст : электронный.

50. Пирогов, В.Ю. 64-битовые процессоры: особенности программирования и исследования кода : монография / В.Ю. Пирогов. – Шадринск : ШГПИ, 2013. – 232 с. – Текст : непосредственный.

51. Пирогов, В.Ю. 64-битовые процессоры: особенности программирования для платформ Windows и Unix : монография / В.Ю. Пирогов. – Шадринск : ШГПИ, 2016. – 164 с. – Текст : непосредственный.

52. Пирогов, В.Ю. Ассемблер для Windows / В.Ю. Пирогов. – Изд. 4-е. – Санкт-Петербург : БХВ, 2007. – 896 с. – Текст : непосредственный.

53. Пирогов, В.Ю. Ассемблер и дизассемблирование / В.Ю. Пирогов. – Санкт-Петербург : БХВ, 2006. – 466 с. – Текст : непосредственный.

54. Пирогов, В.Ю. Введение в программирование на языке ассемблера GAS в операционной системе Linux : учеб. пособие для студентов / В.Ю. Пирогов. – Шадринск : ШГПУ, 2022. – 292 с. – Текст : непосредственный.

55. Пирогов, В.Ю. Исследование исполняемого кода для операционной системы Windows и системы команд Intel / В.Ю. Пирогов. –

Шадринск : ШГПИ, 2007. – 243 с. – Текст : непосредственный.

56. Пирогов, В.Ю. Операционные системы на базе набора команд x86–64 в контексте низкоуровневого программирования / В.Ю. Пирогов – Текст : непосредственный // Прикладная информатика. – 2015. – № 6 (60). – С. 70-82.

57. Решение Еврокомиссии в отношении Intel. – Текст : электронный // CNews : [веб-сайт]. – URL: <http://www.cnews.ru/news/line/index.shtml?2009/08/10/357050> (дата обращения: 13.03.2021).

58. Рочкинд, М. Дж. Программирование для Unix / Марк. Дж. Рочкинд. – Санкт-Петербург : БХВ, 2005. – 404 с. – Текст : непосредственный.

59. Сайт компании AMD : [веб-сайт]. – URL: <http://www.amd.com/us> (дата обращения: 13.03.2021). – Текст : электронный.

60. Стивенс, У. UNIX. Взаимодействие процессов / Уильям Стивенс. – Санкт-Петербург : Питер, 2003. – 569 с. – Текст : непосредственный.

61. Элджер, Д. C++ : библиотека программиста / Джефф Элджер. – Санкт-Петербург : Питер, 2000. – 259 с. – Текст : непосредственный.

62. Юров, В.И. Assembler : учебник для вузов / В.И. Юров. – Санкт-Петербург : Питер, 2003. – 637 с. – Текст : непосредственный.

ПРИЛОЖЕНИЕ

Приложение 1

Список системных функций Linux, используемый в работе с кратким описанием в нотации языка C

Таблица 1

Список системных функций Linux, используемый в работе с кратким описанием в нотации языка C

rax	Системная функция	rdi	rsi	rdx	r10	r8	r9
0	sys_read	unsigned int fd	char *buf	size_t count			
1	sys_write	unsigned int fd	const char *buf	size_t count			
2	sys_open	const char *filenam e	int flags	int mode			
3	sys_close	unsigned int fd					
8	sys_lseek	unsigned int fd	off_t offset	unsigned int origin			
9	sys_mmap	unsigned long addr	unsigned long len	unsigned long prot	unsigned long flags	unsigned long fd	unsigned long offm
11	sys_munmap	unsigned long addr	size_t len				
12	sys_brk	unsigned long brk					
39	sys_getpid						
57	sys_fork						
59	sys_execve	const char *filenam e	const char *const argv[]	const char *const envp[]			

rax	Системная функция	rdi	rsi	rdx	r10	r8	r9
60	sys_exit	int error_code					
61	sys_wait4	pid_t upid	int *stat_addr	int options	struct rusage *ru		
78	sys_getdents	unsigned int fd	struct linux_dir ent *dirent	unsigned int count			
80	sys_chdir	const char *filename					
82	sys_rename	const char *oldname	const char *newname				
83	sys_mkdir	const char *pathname	int mode				
84	sys_rmdir	const char *pathname					
87	sys_unlink	const char *pathname					
90	sys_chmod	const char *filename	mode_t mode				

rax	Системная функция	rdi	rsi	rdx	r10	r8	r9
92	sys_chown	const char *filename	uid_t user	gid_t group			

Система команд процессора X86-64

В данном приложении представлена информация об основных группах команд процесса архитектуры x86-64.

Адресация памяти

Во многих командах процессора может присутствовать операнд из памяти. При обращении к памяти важны два аспекта: размер операнда и его адрес. О размере операндов мы подробно писали в параграфе 2.2. Размеры операндов определяется по умолчанию, если это однозначно ясно из команды. Например, `add %ax, %bx` – сложение двух операндов. Очевидно, что речь о сложении двух двухбайтовых чисел. В тех случаях, когда это важно для читаемости программы, используются суффиксы: `b` – байт, `w` – слово, `l` – двойное слово (32-битовое число), `q` – 64-битовое число. Например, `movl %eax, num` выполняет копирование 4-х байтового числа, находящегося в регистре `eax`, в область памяти, адрес которой определяется меткой `num`. Конечно, мы можем написать и так: `movl %eax, num` и для ассемблера все понятно, поскольку есть 32-битовый регистр `eax`. Мы просто лишний раз подчеркиваем, что операция выполняется над 32-битовым числом. Но есть ситуации, когда суффиксы должны однозначно показать ассемблеру, над каким операндом нужно выполнять действие. Например, `dec num` для ассемблера неприемлемая запись, поскольку не понятно над операндом какого размера должна выполняться операция декремента. Мы должны непременно уточнить, например так `decl num`, если речь идет о декремента 4-х байтового числа. Этим ассемблер GAS отличается в положительную сторону от некоторых других ассемблеров. Синтаксис GAS вынуждает программиста непосредственно в тексте программы показывать размер операнда.

Перейдем теперь к вопросу адресации, т.е. способам указания на ту или иную область памяти. Вместо того, чтобы долго объяснять разные виды адресации, мы представляем программу, которая бы все эти виды адресации демонстрирует. Эта программа из рисунка 75. В ней представлено 6 способов адресации. С помощью каждой из них код символа перебрасывается из одной ячейки памяти в другую, а потом выводится на консоль. Для чистоты эксперимента после каждого вывода последняя ячейка памяти чистится (функция `cle`). Вывод символа

осуществляется функцией `pr`. Соответственно, правильность выполненных действий подтверждается выводом на консоль шести букв 'A'. В программе каждый блок с адресацией отделяется комментарием. Ниже остановимся на каждом блоке.

1-й способ. Прямая адресация.

Доступ к памяти происходит непосредственно с помощью адреса (метки) области памяти. В примере это `msg`. Т.е. `mov msg, %al` помещает содержимое однобайтовой ячейки памяти с адресом `msg` в регистр `al`. При чем адресацию можно указывать со смещением, например так `mov msg+1, %al`.

2-й способ. Косвенная адресация.

Для доступа к памяти предварительно получить адрес ячейки. Адрес получаем так: `mov $msg, %rbx` и далее `mov (%rbx), %al` - содержимое ячейки, адрес которой находится в `%rbx` помещается в регистр `al`.

3-й способ. Косвенная адресация.

При использовании регистра, для хранения адреса можно явно указывать смещение: `mov 8(%rbx), %al` - адрес получается путем сложения содержимого регистра `rbx` и числа 8.

4-й способ. Косвенная адресация.

Похоже на 3-й способ, но смещение хранится в другом регистре. Т.е. `mov (%rbx, %rax), %al` - результирующий адрес получается путем сложения содержимого регистров `rbx` и `rax`. Иногда можно встретить такой термин: адресация по базе со сдвигом.

5-й способ. Косвенная адресация.

Данный вариант получается путем комбинации 3-го и 4-го вариантов: `mov 4(%rbx,%rax), %al` - результирующий адрес ячейки получается сложением содержимого регистров `rax`, `rbx` и числа 4.

6-й способ. Косвенная адресация.

Не дает ничего нового в самой адресации. В нем показан другой способ получения адреса с использованием команды `lea`: `lea msg(%rip), %rbx`, что, в сущности, эквивалентно команде `mov $msg, %rbx`.

Компилируем пример из рисунка 29 обычными и много апробированными нами командами:

```
as --64 prog47.s -o prog47.o
```

```
ld -s prog47.o -o prog47
```

```
.data # сегмент данных
```

```
msg:
```

```
.ascii "A"
```

```
char:
```

```

.ascii " "
.globl _start
.text
_start:
# 1-й прямая адресация
mov msg,    %al
mov %al,    char
call pr
call cle
# 2-й косвенная адресация
mov $msg,    %rbx
mov (%rbx),  %al
mov %al,    char
call pr
call cle
# 3-й вариант, косвенная адресация
mov $msg,    %rbx
sub $8,      %rbx
mov 8(%rbx), %al
mov %al,    char
call pr
call cle
# 4-й вариант, косвенная адресация
mov $msg,    %rbx
sub $8,      %rbx
mov $8,      %rax
mov (%rbx,%rax), %al
mov %al,    char
call pr
call cle
# 5-й вариант, косвенная адресация
mov $msg,    %rbx
sub $8,      %rbx
mov $4,      %rax
mov 4(%rbx,%rax), %al
mov %al,    char
call      pr

```

```

call    cle
# 6-й вариант, косвенная адресация
lea msg(%rip), %rbx
mov (%rbx), %al
mov %al, char
call    pr
call    cle
# закончить работу программы
mov $60, %rax
xor %rdi, %rdi
syscall
# очистка
cle:
mov $32, %al
mov %al, char
ret
# печать
pr:
mov $1, %rdi
mov $char, %rsi
mov $1, %rdx
mov $1, %rax
syscall
ret

```

Рис. 75. Пример использования различных способов адресации (prog47.s)

Приведенные примеры адресации, однако еще не исчерпывают все возможности системы x86-64. Речь идет об адресации с масштабированием.

Обращаемся теперь к рисунку 76. На рисунке представлена функция `sum_ar`. Функция получает на входе два параметра: адрес начала числового массива и количество элементов в нем. Функция возвращает сумму элементов массива. `add (%rdi, %rbx, 8), %rax` - вот на эту команду обращаем особое внимание. К содержимому `rax` прибавляется содержимое 8-байтового числа, адрес которого формируется следующим образом: к содержимому `rdi` (начало массива) прибавляется содержимое `rbx` (значение

индекса) умноженное на 8 (масштабирование). Вот это очень важная возможность процессора x86-64 помогает легко обрабатывать числовые массивы. В данном случае мы получаем сумму элементов массива, которая как и полагается возвращается через регистр `rax`.

```
.text
sum_ar:
# параметры согласно соглашению о вызове
# rdi указывает на начало массива
    mov %rsi, %rcx      # количество элементов
    xor %rax, %rax      # в rax будет копиться результат
    xor %rbx, %rbx      # индекс массива (начинается с нуля)
loo:
    add (%rdi, %rbx, 8), %rax  # суммируем i-й элемент массива
    inc %rbx              # к следующему индексу
    loop loo              # повторять пока содержимое rcx больше 0
    ret                  # выход из функции
```

Рис. 76. Пример функции с индексацией с масштабированием (`l2l.s`)

Строковые операции процессора x86-64

Под строковыми операциями¹ будем понимать команды, позволяющие автоматизировать действия над протяженными областями памяти. Перечислим эти команды: `movs`, `lods`, `stos`, `cmps`, `scas`. Каждая из команд выполняет только одну итерацию (одно элементарное действие). Но если использовать префикс `repe` (`repne`) (префикс повторения) можно распространить действие на последовательность данных. Разумеется, данные команды можно эффективно использовать и в обычных циклах.

Команда `movs` - перенос последовательности данных из одной области в другую. Используется с суффиксами `b` (байт), `w` (слово), `d` (двойное слово), `q` (8 байтов). Соответственно для переноса байтов, слов, двойных слов и 8 -байтовую цепочку. Адрес исходных данных определяется содержимым регистра `rsi` (`source`), адрес, куда переносятся данные, лежит в регистре `rdi` (`target`). После переноса регистры `rsi` и `rdi` изменяются соответственно на 1, 2, 4 или 8. Направление изменения определяется значением флага `DF` регистра флагов. Если флаг равен нулю, то над регистрами `rsi` и `rdi` осуществляется инкрементирование, если флаг установлен (равен 1), то содержимое регистров уменьшается на единицу.

Сбросить флаг можно командой `cld`, установить `std`. При этом выполняется еще одна важная операция – содержимое регистра `rcx` уменьшается на 1. При использовании префикса повторения, итерации прекращаются если содержимое `rcx` становится равным нулю.

Рассмотрим фрагмент:

```
mov $lng1, %rsi # откуда
mov $lng, %rdi  # куда
mov $8000, %rcx # длина строки
cld             # установили нужное направление переноса
rep movsb      # выполнили операцию
```

Данный фрагмент кода осуществляет побайтовой копирование данных из одной области памяти в другую. Префикс `rep` не только повторяет команду, но уменьшает на каждом шаге содержимое регистра `rcx`. Повторение прекращается при сбрасывании флага `ZF` в 0 или, когда содержимое `rcx` становится равным нулю. Для данной команды флаг обнуляется только при обнулении `rcx`.

Команда `lods`, используется с суффиксами `b`, `w`, `l`, `q` - команда переносит байт (слово, двойное слово, 8-байтовую цепочку) из памяти, на которую указывает регистр `rsi`. После переноса содержимое `rsi` изменяется в соответствии со значением флага `DF`: если флаг равен 0, то осуществляется инкремент (на байт, слово, двойное слово или на 8 байтов), если флаг равен 1, то значение `rsi` уменьшается. Перенос байтов из памяти осуществляется соответственно в регистры `al`, `ax` или `rax`.

Команда `stos`. Эта команда является командой, обратной команде `lods`. Она переносит байт, слово, двойное слово или 8-байтовую величину из регистра (`al`, `ax`, `eax` или `rax`) в память, которая адресуется регистром `rdi`. После выполнения команды осуществляется инкремент или декремент регистра `rdi`.

Команда `cmps`, команда сравнивает в зависимости от суффикса (`b`, `w`, `l`, `q`) два байта, два слова, два двойных слова или две 8-байтовых величины, расположенных по адресам, расположенным в регистрах `rsi` и `rdi`. Результат сравнения также влияет на флаги, как выполнение команды `str` или команды `sub`. После выполнение команды `cmps` содержимое регистров `rsi` и `rdi` инкрементируется или декрементируется в зависимости от значения флага `DF` и суффикса команды. Команда очень легко может быть приспособлена для сравнения двух областей данных. Рассмотрим один из вариантов такого сравнения.

```
cld # сбросить флаг
mov $lng1, %rsi # указатель на первую байтовую строку
```

```

mov $lng, %rdi    # указатель на вторую байтовую строку
mov $80, %rcx     # количество байтов в строках
repe cmpsb        # осуществляем сравнение
jnz no_eq         # переходим если строки не равны

```

Данный фрагмент сравнивает две цепочки длиной 80 байтов. При этом выход из процесса сравнения происходит в двух случаях: когда цепочки заканчиваются и при очередном сравнении байтов, если байты оказываются разными. Если цепочки не совпадают, то происходит переход на метку `no_eq`.

Команда `scas` – в зависимости от суффикса (b, w, l, q) сравнивает содержимое `al`, `ax`, `eax` или `rax` с байтом, словом, двойным словом, 8-байтовой величиной, адресуемой регистром `rdi`. Команда `scas`, как и команда `cmps` воздействует на соответствующие флаги. После выполнения команды в зависимости от значения флага `DF` изменяется содержимое регистра `rdi`.

Описанные выше команды хороши тем, что одновременно уменьшают объем кода и увеличивают его производительность.

На рисунке 77 представлена функция, которая осуществляет копирование числового массива по указанному адресу. Функция на входе получает три параметра: указатель на массив – куда копируем, указатель на массив – откуда копируем, количество элементов массива (элементов, а не байтов). Предполагается, что массивы состоят из 8-байтовых чисел. Кроме того, функция возвращает указатель на результирующий массив.

```

.text                # сегмент кода
copa:                # метка начала функции
#параметры согласно соглашению о вызове (rdi, rsi, rdx)
    push %rdi        # указывает на массив, куда копировать
    push %rsi        # указывает на массив 64-битовых чисел (откуда
копировать)
    push %rcx        # содержит количество элементов в массиве
    mov %rdx, %rcx   # максимальное количество байтов, которые можно
копировать
    cld              # сбросим флаг на всякий случай
    repe movsq       # копирование содержимого массива в другой
массив
# восстановим содержимое регистров
    pop %rcx
    pop %rsi

```

```
pop %rdi
# возвращает указатель на результирующий массив
mov %rdi, %rax
ret
```

Рис. 77. Пример функции копирования одного числового 8-байтового массива в другой

Команды условных и безусловных переходов

Команды переходов, которые мы многократно использовали в программах делятся на условные и безусловные. Собственно команда безусловного перехода только одна: `jmp`. В этой команду указывается адрес, куда необходимо перейти. Адрес может быть указан несколькими способами:

1. С помощью метки: `jmp ll`, где `ll` метка, которая в выполняющейся программе будет равна некоторому конкретному адресу.
2. С указанием адреса в регистре: `jmp %rax`. Переход осуществляется по адресу, указанному в регистре `rax`.
3. С помощью косвенной адресации: `jmp *(%rbx)`. Переход по адресу, который начинается в ячейке памяти, адрес которой находится в `rbx`.
4. С помощью косвенной адресации с использованием содержимого памяти: `jmp *lo`, переход по адресу, который содержится в памяти, по адресу `lo`.
5. Наконец, для определения адреса перехода можно использовать принятую для текущего адреса в ассемблере GAS точку. Например `jmp .+2` – переход к следующей за инструкцией `jmp` команде.

Как видим у команду безусловного перехода имеется довольно богатый арсенал определения адреса перехода. К сожалению для команд условного перехода, которые мы рассмотрим ниже, в качестве адреса перехода можно указать только метку.

Все команды условного перехода можно разделить на две группы. К первой группе относятся команды, на выполнение которых влияют биты регистра флагов `FLAGS`. На рисунке 78 схематично представлена структура регистра флагов. В схемы указаны только флаги, которые имеют отношения к рассматриваемым нами командам условных переходов.

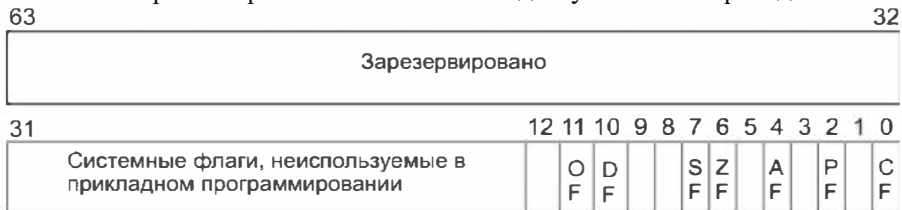


Рис. 78. Биты регистра флагов, используемые в прикладном программировании

В таблице 2 описаны флаги из рисунка 78. Комбинации этих флагов позволяют проверять разные условия и, в конечном итоге, сказываются на

выполнении команд условных переходов.

Таблица 2

Флаги, используемые в прикладном программировании

Бит	Название	Описание	Доступ
0	CF	Флаг переноса (Carry flag). Устанавливается при переносе из старшего бита или заёме в старший бит.	Чтение/запись
2	PF	Флаг четности (Parity flag). Устанавливается, если младший значащий байт результата содержит чётное число ненулевых битов.	Чтение/запись
4	AF	Дополнительный флаг переноса (Auxiliary Carry flag). Аналогичен CF но для бита 3. Изначально ориентировался на BCD арифметику.	Чтение/запись
6	ZF	Флаг нуля (Zero flag). Устанавливается если результат машинной операции равен нулю.	Чтение/запись
7	SF	Флаг знака (Sign flag). Равен старшему значащему биту результата, т.е. биту знака.	Чтение/запись
10	DF	Флаг направления (Direction flag). Значение бита влияет только на строковые команды, которые мы рассмотрим ниже.	Чтение/запись
11	OF	Флаг переполнения (Overflow flag). Устанавливается, если целочисленный результат не помещается в целевом операнде.	Чтение/запись

Как мы уже видели ранее, команды условных переходов имеют вид jxxx. Суффикс xxx может быть разным, в зависимости от того, какое условие мы собираемся проверять. В таблице 3 представлены команды условных переходов, в зависимости от значения флагов и реального

смысла результатов сравнения.

Таблица 3

Команды условных переходов и флаги

Команда	Флаги	Условие
jo	OF=1	Если есть переполнение
jno	OF=0	Если нет переполнения
jc jb jnae	CF=1	Был перенос Ниже Не выше и не равно
jnc jae jnb	CF=0	Без переноса Выше или равно Не ниже
je jz	ZF=1	Равны Ноль (результат)
jne jnz	ZF=0	Не равны Не равен нулю (результат)
jbe jna	(CF= or ZF)=1	Ниже или равно Не выше
ja jnbe	(CF or ZF)=0	Выше Не ниже и не равно
js	SF=1	Отрицательный результат
jns	SF=0	Не отрицательный результат
jp jpe	PF=1	Четность Четное количество единичных битов
jnp jpo	PF=0	Не четность Нечетное количество единичных битов
jl jnge	(SF xor OF)=1	Меньше Не больше и не равно
jg jge jnl	(SF xor OF)=0	Больше Больше или равно Не меньше
jle jng	(SF xor OF) or ZF = 1	Меньше или равно Не больше
jg jnle	(SF xor OF) or ZF = 0	Больше Не меньше и не равно

Обратите внимание (см. Таблица 3), что некоторые команды являются тождественными друг другу. Например: `jc, jnl`; `jc, jbe, jnae` и т.д. Кроме того, есть команды условных переходов, обеспечивающих сравнение беззнаковых и знаковых чисел. Сравнение беззнаковых: `ja`, `jnb`, `jnc`, `jbe`, `jnge`, `jnl`, `jnl`, `jnge`, `jnl`. Сравнение знаковых: `ja`, `jnb`, `jnc`, `jbe`, `jnge`, `jnl`, `jnl`, `jnge`, `jnl`. Например: `sub %rbx, %rax/ja lo` – вычесть содержимое `rbx` из содержимого `rax` и перейти на метку `lo`, если число, которое было в `rax` больше содержимого регистра `rbx`. Вместо `ja` в данном примере можно было бы использовать `jnb`. Еще один пример: `and %rax, %rax/jz zer` – перейти на метку `zer`, если в `rax` содержится 0 (`and` выполняет побитовую операцию И).

Вторая группа команд условного перехода является `jcxz` – переход, если содержимое регистра `ecx` равно нулю и `loop` – переход, если содержимое `ecx` не равно нулю, с одновременным декрементом содержимого `ecx`. Последняя команда `loop` по сути своей предназначена для организации цикла. Схема работы такого цикла очень проста:

```
...
lo:
# тело цикла
...
loop lo
```

При этом надо иметь в виду, что за неизменность содержимого регистра `ecx` при выполнении тела цикла, естественно отвечает программист. Если использовать для хранения значения `ecx` стек, то очень просто организовать и вложенные циклы. Например, для трех вложенных циклов имеем:

```
...
lo3:
push %rcx
...
lo2:
push %rcx
...
lo1 :
push %rcx
...
pop %rcx
loop lo1
pop %rcx
loop lo2
```

```
pop %rcx  
loop lo3
```

Разумеется, не составит никакого труда организовать циклы любой сложности с помощью других команд условного и безусловного переходов. Команда `loop` имеет также ряд суффиксов, что делает ее более совершенной: `loope (loopz)` – переход, если содержимое `rcx` не равно нулю и флаг `ZF=1`, `loopne (loopnz)` - переход, если содержимое `rcx` не равно нулю и флаг `ZF=0`. В следующем параграфе (параграф 3.4) мы рассмотрим организацию циклов в ассемблере 86х-64.

На рисунке 79 представлена функция, которая принимает на входе три числовых параметра и возвращает максимальное значение из трех.


```

.text
.globl cora
cora: # метка начала функции
# параметры в rdi, rsi, rdx соответственно
cmp %rdi, %rsi
jg l1
cmp %rdi, %rdx
jg l1
mov %rdi, %rax
jmp lo
l1:
cmp %rsi, %rdi
jg l2
cmp %rsi, %rdx
jg l2
mov %rsi, %rax
jmp lo
l2:
mov %rdx, %rax
lo:
ret

```

Рис. 79. Пример функции нахождения максимального числа из трех

Базовые арифметические операции

В основном наборе инструкций входят разные вариации четырех арифметических действий: сложение, вычитание, умножение, деление. Важно помнить, что в результате арифметических действий меняются некоторые биты регистра флагов, что позволяет выполнять команду условного перехода, т.е. разветвлять программу на основе результата операции. Замечу, что для команд сложения и вычитания справедливыми являются отмеченное выше для операндов команды `mov`. К командам сложения можно отнести: `add` – обычное сложение, `adc` – сложение с добавлением результату флага переноса в качестве единицы (если флаг равен нулю, то команда эквивалентна команде `add`), `xadd` – сложение, с предварительным обменом данных между операндами, `inc` – прибавление единицы к содержимому операнда. Несколько примеров: `add %rbx, %rax` (или

addq %rbx dt, где четко указано, что складываются 64-битовые величины) – к содержимому области памяти dt добавляется содержимое регистра rbx и результат помещается в dt; adc %rdx, %rdx – удвоение содержимого регистра rdx плюс добавление значения флага переноса; incl ll – увеличение на единицу содержимого памяти по адресу ll. При этом явно указывается, что операнд имеет размер 32 бита (l – dword).

К командам вычитания можно отнести следующие инструкции процессора x86-64: sub – обычное вычитание, sbb – вычитание из результата флага переноса в качестве единицы (если флаг равен нулю, то команда эквивалентна sub), dec – вычитание единицы из результата, neg – вычитание значения операнда из 0. Несколько примеров: sub %rax, ll – из содержимого ll вычитается содержимое регистра rax (или явно subq %rax, ll, где указывается, что операнды имеют 64-размер), и результат помещается в ll; subw go, %ax – вычитание из содержимого ax числа по адресу go, результат помещается в ax; sbb %rdx, %rax – вычитание с дополнительным вычитанием флага переноса (из числа в rax вычитается число в rdx и результат в rax); decb l – вычитание единицы из байта, расположенного по адресу l. Следует отметить еще специальную команду cmp, которая во всем похожа на команду sub, кроме одного – результат вычитания никуда не помещается. Инструкция используется специально, для сравнения операндов.

Две основные команды умножения: mul – умножение беззнаковых чисел, imul – умножение знаковых чисел. Команда содержит один операнд – регистр или адрес памяти. В зависимости от размера операнда данные помещаются: в ax, dx : ax, edx : eax, rdx : rax. Например: mull ll – содержимое памяти с адресом ll будет умножено на содержимое eax (не забываем о суффиксе l), а результат отправлен в пару регистров edx : eax; mul %dl – умножить содержимое регистра dl на содержимое регистра al, а результат положить в ax; mul %r8 – умножить содержимое регистра r8 на содержимое регистра rax, а результат положить в пару регистров rdx : rax.

Для деления (целого) также предусмотрены две команды: div – беззнаковое деление, idiv – знаковое деление. Инструкция также имеет один операнд – делитель. В зависимости от его размера результат помещается: al – результат деления, ah – остаток от деления; ax – результат деления, dx – остаток от деления; eax – результат деления, edx – остаток от деления; rax – результат деления, rdx – остаток от деления. Приведем примеры: divl dv – содержимое edx : eax делится на делитель, находящийся в памяти по адресу dv и результат деления помещается в eax, остаток в edx; div %rsi – содержимое rdx : rax делится на содержимое rsi, результат

помещается в `gaх`, остаток в `rdх`.

Говоря об арифметических операциях, укажем также на инструкции процессора, позволяющие работать с BCD числами. BCD – binary code decimal, т.е. двоично-десятичный код. Есть упакованные и не упакованные BCD -числа. В упакованных BCD – числах каждый полубайт представляет один разряд десятичного числа, в неупакованных BCD – числах на разряд десятичного числа отводится целый байт. Рассматриваемые здесь инструкции позволяют работать с такими числами, точнее они корректируют обычные арифметические операции, которые мы только что разбирали, так, что результат для BCD – числа оказывается верным. Надо сказать, что наличие с современным процессоре `x86-64` инструкций группы FPU (FPU – floating point unit, т.е. устройство чисел с плавающей точкой или математический сопроцессор) делает не нужным использование выше означенных инструкций. Очевидно, по этой причине они были удалены из `64` -битового режима (о котором идет речь в данной книге). Мы только перечислим для справки эти инструкции: `aaa`, `aas`, `aam`, `aad`, `daa`, `das`.

Битовые и логические операции

Я решил объединить в одном параграфе битовые и логические операции. Почему? В сущности ведь логические операции также являются битовыми, поскольку результат логического действия получается на основе операций с отдельными битами, которые рассматриваются в качестве «заменителей» для логических значений «ИСТИНА» и «ЛОЖЬ».

Битовые операции

К битовым инструкциям относятся в основном операции сдвига, а также команды, оперирующие одним битом. Рассмотрим в начале сдвиговые операции. Эти команды содержат два операнда. Сдвиг осуществляется во втором операнде, первый операнд содержит счетчик сдвига (насколько осуществляется сдвиг). Команда `rcl` – сдвиг битов влево, в сторону старших битов. При этом старший бит операнда проходит через флаг переноса, а потом попадает в младший бит операнда. Второй операнд команды может быть регистром или ячейкой памяти от `8` до `64` бит. В качестве первого операнда может быть `8`-битовое число или регистр `cl`. При этом для всех вторых операндов, кроме `64`-битового, процессор маскирует три старших бита первого операнда, так что счетчик сдвига изменяется в диапазоне от `0` до `31`. В случае `64`-битового второго операнда,

процессор маскирует два старших бита, так что значение счетчика сдвига может меняться в промежутке от 0 до 63. Команда `rsc` аналогична команде `rcl`, но процесс сдвига осуществляется от старших к младшим битам (вправо). Например:

```
mov $0x8000, %ax
mov $2, %cl
rcl %cl, %ax
```

– в результате в регистре `ax` будет содержаться 1, команда же `rsc %cl, %ax` – приведет к тому, что `ax` опять будет содержать 0x8000.

Следующая пара команд сдвига это `rol` и `ror`. Соответственно осуществляет сдвиг влево и вправо. Команды очень похожи на `rcl` и `rsc` соответственно, но с одним отличием: старший бит второго операнда сразу переходит в его младший бит. Т.е. последовательность команд

```
mov $0x8000, %ax
mov $2, %cl
rol %cl, %ax
```

приведет к тому, что в `ax` будет содержаться число 2, при этом, флаг переноса, естественно, не меняется.

Команда `sal` (синоним `shl`) сдвиг влево через бит переноса. Очень похожа на команду `rcl` (т.е. старший бит проходит через флаг переноса), но при этом младшие биты заполняются нулями. Команда `shr` осуществляет сдвиг во втором операнде вправо. При этом младший бит проходит через флаг переноса, а старшие биты заполняются нулями. Команда `sar` похожа на `shr`, но при ее выполнении старший бит играет роль заполнителя.

Кроме двухоперандных команд, которые мы только что перечислили, существуют еще две трехоперандные команды `shld` и `shrd`. Команды соответственно похожи на команды `shl` и `shr`. Сдвиг осуществляется в третьем операнде, а в качестве счетчика берется первый (левый операнд). Средний операнд играет роль источника. При выполнении команды `shld` младшие биты считываются из старших битов источника. В случае команды `shrd` старшие биты считываются из младших битов источника. Содержимое источника при этом не меняется. Чтобы уяснить себе работу указанных команд рассмотрим два примера:

```
mov $0, %dx
mov $2, cl
mov $0x8000, %bx
shld %cl, %bx, %dx
```

– в результате выполнения данных команд в `dx` будет содержать число

2;

```
mov $0, %dx
mov $1, %cl
mov $0x8000, %bx
shld %cl, %bx, %dx
shld %cl, %bx, %dx
```

– в результате выполнения этих команда в регистре dx будет число 3.

Обратимся теперь к командам, оперирующим конкретными битами. Четыре команды bt, btc, btr, bts очень похожи друг на друга. Второй операнд – регистр или область памяти (источник), первый операнд регистр или число. Первый операнд индексирует (определяет некоторый) бит в первом операнде. При выполнении команды указанный бит копируется в бит переноса. При этом: bt – меняет исходный бит, btc – инвертирует исходный бит, btr – обнуляет исходный бит, bts – заменяет исходный бит единицей. Особо следует остановиться на правиле индексирования битов. Если первый операнд регистр, то в качестве индекса берется остаток от деления содержимого второго операнда на 16, 32 или 64 соответственно, в зависимости размера первого операнда. Если второй операнд – область памяти, а первый операнд число, то процессор берет только значащие биты, в соответствии с размером первого операнда. Если второй операнд область памяти, а первый операнд регистр, то взяв нулевой бит второго операнда в качестве точки отсчета, смещения нужного бита ищется соответственно в пределах $-2^{63} - 2^{63} - 1$, $-2^{31} - 2^{31} - 1$, $-2^{15} - 2^{15} - 1$ в зависимости от размера регистра.

Команды bsf и bsr осуществляют поиск первого не нулевого бита соответственно слева и справа в операнде. Команды имеют два операнда. Первый операнд (регистр), второй операнд – регистр или область памяти. Поиск не нулевого бита осуществляется во втором операнде, результат поиска – номер найденного бита помещается в первый операнд. Если не нулевой бит существует, то обнуляется флаг нуля, если бит не найден, то флаг нуля устанавливается. Команда popcnt имеет два операнда. Второй операнд – регистр (16 -, 32 -, 64 -разрядный), второй - регистр или переменная. Разрядность второго операнда соответствует разрядности первого операнда. Команда рассчитывает количество единичных битов в первом операнде и помещает результат во второй операнд. Команда поддерживается не всеми процессорами. Команда lzcnt. Команда по формату и назначению аналогична команде popcnt, но подсчитывает количество нулевых битов, идущих подряд, начиная с самого старшего бита в сторону младших бит. Поддерживается не всеми процессорами.

Существует целая группа команд, начинающихся с префикса set.

Окончание у этих команд такое же, как у команд условного перехода. Команда проверяет условие, определяемое окончанием, и помещает в регистр или ячейку памяти 1, если условие выполняется и 0, если условие не выполняется. Размер операнда может составлять только 8 битов. Например, `setzb lg` – проверяет флаг нуля и если он установлен, то помещает по адресу `lg` байт равный 1 и нуль в противном случае. Команду, начинающуюся с приставки `stov` можно отнести к той же группе. Это копирование данных из первого операнда во второй, если выполняется указанное в суффиксе условие. В качестве первого операнда выступает ячейка памяти (16 – , 32 – или 64 - битовая) или регистр (16 – , 32 – или 64 - битовый). Вторым операндом может быть только регистр. Например, `stovz lol, %r8` - копирует данные из 8 байтовой ячейки в регистр `r8`, если установлен флаг нуля.

Логические операции

Существуют набор команд, которые воздействуют на операнд, как на цепочку элементов со значениями истина – ложь. Элементом, как вы понимаете, является бит. А далее все просто. Поддерживаются следующие двухоперандные команды: `and` (логическое «И»), `or` (логическое «ИЛИ»), `xor` (логическое «ИСКЛЮЧАЮЩЕЕ ИЛИ»). Действия осуществляются побитово, результат операции помещается во второй операнд. В качестве второго операнда может выступать регистр или ячейка памяти, в качестве первого регистр, ячейка памяти, числовая константа. Кроме того, есть также команда `test`, которая эквивалентна команде `and` (в воздействии на флаги), но не меняет содержимое операндов. Ее удобно использовать в качестве проверки содержимого регистра на 0, или наличия бита или группы битов в правом операнде. Имеется также однооперандная команда `not`, которая инвертирует каждый бит операнда.

Например, `xor %rax, %rax` – обнуление содержимого регистра `rax` (многие предпочитают такое обнуление команде `mov $0, %rax`).

И для инструкции `test`

```
test $1, %r8
```

```
jnz ll
```

```
# бит отсутствует
```

```
ll:
```

```
...
```

– перейти на метку `ll`, если установлен нулевой бит в регистре `r8`.

Команды для работы с числами с плавающей точкой

Процессор x86-64 имеет довольно богатый набор команд, для вычислений с плавающей точкой (набор FPU). Мы посвятим работе с такими числами два параграфа. В данном параграфе мы познакомимся с технической стороной дела: специальными регистрами и набором команд. Материал параграфа носит в значительной степени справочный характер.

Прежде чем рассматривать команды работы с числами с плавающей точкой (команды арифметического сопроцессора), следует сказать несколько слов о регистрах, используемых при работе с этими командами. Желательно вспомнить материал из параграфа 1.7, где рассматривается структура представления чисел с плавающей точкой.

1. Сопроцессор имеет восемь 80-битных рабочих регистров, представляющих собой стековую кольцевую структуру (стек сопроцессора). Регистры называются R0, R1, ... R7, но доступ к ним напрямую невозможен. Каждый регистр может занимать любое положение в стеке. Название стековых (относительных) регистров - st(0), st(1), st(2), st(3), st(4), st(5), st(6), st(7). Кроме того, имеется еще регистр состояния, по флагам которого можно, в частности, судить о результате выполненной операции. Регистр управления содержит в себе биты, влияющие на выполнение команд сопроцессора.

2. Сопроцессор оперирует следующими вещественными форматами:

- a. Короткое вещественное число. Задействует 32 бита.
- b. Длинное вещественное число. Задействует 64 бита.
- c. Расширенное вещественное число. Задействовано 80 битов.

Все вычисления осуществляются сопроцессором над 80-битовым числом, а 32-х и 64-х битовые представления чисел используются для обмена между арифметическим сопроцессором и памятью, или регистрами основного процессора.

3. Регистр тэгов содержит 16 бит, описывающих содержание регистров сопроцессора: по два бита на каждый рабочий регистр. Тэг говорит о содержимом регистре данных. Вот значения тэгов: 00 – действительное ненулевое число, 01 – истинный нуль, 10 – специальные числа, 11 – отсутствие данных.

4. При вычислении с помощью команд сопроцессора большую роль играют исключения или особые ситуации. Типичной особой ситуацией является деление на 0. Биты особых ситуаций хранятся в регистре состояний. Учет особых ситуаций необходим для получения правильных результатов.

Специальные случаи:

- a. Положительный ноль (все биты нули);
- b. Отрицательный ноль (знаковый бит равен 1);
- c. Положительная бесконечность (знаковый бит 0, все биты мантиссы 0, все биты экспоненты 1);
- d. Отрицательная бесконечность (знаковый бит 1, все биты мантиссы 0, все биты экспоненты 1);
- e. Денормализованное число (все биты экспоненты 0);
- f. Неопределенное число (знаковый бит 1, все биты экспоненты 1, первый бит мантиссы, а для 80-битного числа два бита 1, остальные 0);
- g. Нечисловой экземпляр snan (все биты экспоненты 1, первый бит мантиссы 0, а для 80-битного числа первые два бита 10, а среди остальных битов есть 1);
- h. Нечисловой экземпляр qnan (все биты экспоненты 1, первый бит мантиссы, а для 80-битного числа два первых, равны нулю, среди остальных битов мантиссы есть 1);
- i. Неподдерживаемое число (ситуации, не соответствующие стандартным числам и не описанные в специальных случаях).

5. Список особых ситуаций:

- a. Неточный результат (округление);
- b. Недействительная операция;
- c. Деление на ноль;
- d. Антипереполнение (слишком маленький результат);
- e. Переполнение (слишком большой результат);
- f. Денормализованный операнд.

6. Регистр (слово) состояния:

- a. 0-й бит, флаг недопустимой операции;
- b. 1-й бит, флаг денормализованной операции;
- c. 2-й бит, флаг деления на ноль;
- d. 3-й бит, флаг переполнения;
- e. 4-й бит, флаг антипереполнения;
- f. 5-й бит, флаг неточного результата;
- g. 6-й бит, ошибка стека;
- h. 7-й бит, общий флаг ошибки;
- i. 8,9,10,14-й, флаги условий (c0, c1, c2, c3);
- j. 11,12,13-й, число, показывающее, какой регистр является вершиной;
- k. 15 -й бит, флаг занятости.

7. Регистр (слово) управления:

- a. 0-й бит, маска недействительной операции;
- b. 1-й бит, маска денормализованного операнда;
- c. 2-й бит, маска деления на ноль;
- d. 3-й бит, маска переполнения;
- e. 4-й бит, маска антипереполнения;
- f. 5-й бит, маска неточного результата;
- g. 6-й бит, маска антипереполнения;
- h. 8-9-й биты, управление точностью;
- i. 10,11 -й биты, управление округлением;
- j. 12 -й, управление бесконечностью;
- k. 13,14,15 -й, резерв.

Перейдем теперь к самим инструкциям работы с числами с плавающей точкой. Для удобства разобьем все команды на группы.

Перемещение данных

Команды этой группы работают со своим собственным набором регистров. Следовательно, должны быть инструкции обмена данными между этими регистрами и между ними и регистрами общего назначения. Команда `fblld` – преобразует десяти байтовое число в упакованном формате `bcd` в вещественное число двойной точности и помещает его в вершину стека (`st(0)`) сохраняя знак. Команда `fbstp` – преобразует величину, находящуюся в вершине стека `st(0)` в 18-байтовую целую величину в упакованном `BCD` формате и помещает ее по указанному адресу, произведя выталкивание из стека. Команда `fcmov` – в зависимости от значений флагов в регистре флагов перемещает число из регистра `st(i)` в регистр `st(0)`; используется с суффиксами, указывающими на тип условия: `fcmovb`, `fcmovbe`, `fcmove`, `fcmovnb`, `fcmovne`, `fcmovnbe`, `fcmovu`, `fcmovnu`. Например, `fcmovbe st(0), st(3)` – перемещает содержимое `st(3)` в `st(0)` при условии `CF=1` или `ZF=1`. Команда `ffree` помечает регистр, указанный в качестве операнда как пустой. Команда `fld` – вещественное число из памяти или регистра сопроцессора проталкивает в стек. Инструкции `fst`, `fstp`. Копируют данные из вершины стека (`st(0)`) в указанный регистр `st(i)` или переменную памяти (32 -, 64 – или 80 - битовую). Инструкция `fstp` при этом выталкивает стек после копирования. Команда `fxch` – обменивает содержимое регистра `st(0)` и другого регистра. Например, `fxch` – обменивает содержимое регистра `st(0)` и регистра `st(1)`; `fxch st(2)` – обменивает данные содержимое регистра `st(0)` и регистра `st(2)`.

Преобразование типов

Команда `fild` – преобразует знаковое целое число (16 – , 32 – , 64 – битовое), расположенное в памяти в число с плавающей точкой двойной точности и помещает его в стек. Например, `fild (bp)`. Команды `fist`, `fistp`. Команды преобразуют вещественное число, расположенное в `st(0)` в целое число со знаком и помещают его по указанному адресу. Команда `fist` работает с 16 – и 32 – битовыми целыми числами, команда `fistp` также и с 64-х битовыми целыми числами. Кроме этого `fistp` – после преобразования и переноса числа производит выталкивание стека. Команда `fisttp` – преобразует число с плавающей точкой расположенное в `st(0)`, в целое число, путем отбрасывания дробной части и отправляет его по указанному адресу. После переноса производится выталкивание стека.

Действия

Команды `fadd`, `faddp`, `fiadd`, предназначены для выполнения операции сложения. Команда `fadd` может иметь один и два операнда. В первом случае операндом является адрес числа в памяти. Это число складывается с числом, расположенным в `st(0)` и результат помещается в `st(0)`. В случае с двумя операндами оба операнда являются регистрами, один из которых `st(0)`. Результат сложения помещается в левый регистр. Например, `fadd st(1), st(0)` – результат сложения помещается в `st(1)`. Команда `faddp` – аналогична `fadd`, но после выполнения операции происходит выталкивания стека. Команда `fiadd` аналогична предыдущим, но оперирует целыми знаковыми числами. Команда `fchs` – умножает содержимое `s(0)` на -1, результат остается в `st(0)`. Команды `fdiv`, `fdivp`, `fidiv` – команды деление. Команда `fdiv` имеет два формата. В первом случае имеются два операнда – регистры сопроцессора. Один из регистров – `st(0)`. Осуществляется деление двух вещественных чисел – результат помещается в левый операнд. Во втором случае операнд один – адрес числа в памяти. В этом случае содержимое `st(0)` делится на число из памяти, и результат помещается в `st(0)`. Команда `fdivp` имеет также возможный формат без операнда. В этом случае содержимое `st(1)` делится на содержимое `st(0)` и результат помещается в `st(1)`. Кроме этого, команда `fdivp` осуществляет выталкивание стека сопроцессора. Команда `fidiv` имеет один операнд – адрес 32-битового или 16-битового целого числа. Команда делит содержимое `st(0)` на это число, а результат помещает в `st(0)`. Команды `fdivr`, `fdivrp`, `fidivr` – команды полностью аналогичны командам `fdiv`, `fdivp`, `fidiv` с

одним важным отличием в них делимое меняется с делителем местами. Команды `fmul`, `fmulp`, `fimul` – умножение двух операнд. Команда `fmul` может иметь два или один операнд. В первом случае операндами являются регистры сопроцессора, один из которых `st(0)`. Результат помещается в левый операнд. Во втором случае, когда один операнд – им является адрес числа (32 – или 64- битового вещественного). В этом случае содержимое операнда умножается на содержимое `st(0)`, а результат помещается в `st(0)`. Команда `fmulp` может иметь два операнда – как и в случае с `fmul`. Кроме этого, есть и безоперандный вариант. Тогда перемножается содержимое `st(0)` и `st(1)` и результат помещается в `st(0)`. Кроме того, после выполнения операции происходит выталкивание стека. Команда `fimul` осуществляет умножение целого числа на вещественное число в `st(0)`, результат помещается в `st(0)`. Единственным операндом команды является адрес целого числа (16 – или 32 - битового). Команда `fprem` – вычисляет остаток от деления `st(0)` на `st(1)` и результат помещает в `st(0)`. Команда `fpreml` – вычисляет остаток от деления `st(0)` на `st(1)` и результат помещает в `st(0)`. Отличается от команды `fprem` методом округления частного (вычисление по стандарту IEEE-754). Команда `frndint` – округляет содержимое `st(0)` до целого числа и результат помещается в `st(0)`. Команда `fscale` – умножает содержимое `st(0)` на 2 в степени равной целой части числа, содержащегося в `st(1)`. Результат помещается в `st(0)`. Команда обратна команде `fxtract`. Инструкции `fsub`, `fsubp`, `fisub` - выполняют действие вычитание. Вычитают величину в регистре сопроцессора или значение переменной из значения, которое хранится в другом регистре сопроцессора, и помещают результат в этот регистр. Инструкция `fsubp` после выполнения действия выталкивает стек. Инструкция `fisub` конвертирует знаковое целое в вещественное число двойной точности, прежде чем выполнить действие. Например, `fsub st(0), st(2)` – заменяет содержимое `st(0)` разностью `st(0) – st(2)`; `fsub mem` – заменяет `st(0)` значением `st(0) - mem`. Инструкции `fsubr`, `fsubrp`, `fisubr` – команды вычисления разности аналогичны инструкциям `fsub`, `fsubp`, `fisub`, но с тем лишь отличием, что вычитаемое меняется местами с уменьшаемым. Например, `fsub st(0), st(2)` вычисляет разность `st(2) – st(0)` и помещает результат в `st(0)`. Команда `fxtract` – отделяет дробную часть вещественного числа (мантиссу), находящегося в `st(0)` от степени числа два. Показатель помещается в вершину стека, а затем в стек помещается дробная часть, таким образом, показатель оказывается в регистре `st(1)`. Команда обратна команде `fscale`.

Функции

Здесь представлены всевозможные числовые функции. Инструкция `f2xm1` – возводит число 2 в степень, которая находится в регистре `st(0)`, затем вычитает 1 и результат помещает в `st(0)`. Инструкция `fabs` – вычисляет абсолютное значение числа, находящегося в `st(0)`, отбрасывая знак, помещая результат в `st(0)`. Инструкция `fcos` – вычисляет косинус числа, расположенного в `st(0)` и результат помещает в `st(0)`. Предполагается, что число выражает собой угол в радианах. Команда `fptan` – вычисляет арктангенс. Вычисляется арктангенс числа, лежащего в `st(1)` (Y) и делит результат на число, лежащее в `st(0)` (X), результат помещается в `st(1)`. После выполнения действия происходит выталкивание стека. Команда `fptan` – вычисление тангенса. Команда вычисляет тангенс числа, находящегося в `st(0)`. Результат помещается в `st(0)`. После выполнения действия в стек помещается 1. Другими словами: тангенс в `st(1)`, единица в `st(0)`. Команда `fsin` – вычисляет синус от числа, находящегося в `st(0)` и помещает результат в `st(0)`. Команда `fsincos` – вычисляет синус числа, находящегося в `st(0)` и помещает значение в `st(0)`, затем вычисляет косинус значения `st(0)` (старого) и проталкивает его в стек. Команда `fsqrt` – вычисляет квадратный корень от числа в `st(0)` и помещает результат в `st(0)`. Команда `fyl2x` – вычисляет логарифм по основанию 2 от числа, находящегося в `st(0)` и умножает результат на число, которое находится в `st(1)`. Помещает значение в `st(1)` и производит выталкивание стека. Команда `fyl2xp1` – команда аналогична команде `fyl2x`, но добавляет к результату этой команды еще единицу.

Проверка условий

Команды `fcom`, `fcomr`, `fcomrr` – команды сравнения. Принцип – флаги устанавливаются как при вычитании (воздействуют на флаги `C0`, `C2`, `C3`). Если в команде отсутствует операнд, то сравнение производится `st(0)` с `st(1)`. Если операнд присутствует, то `st(0)` с операндом. Операнд может быть регистром и числом, расположенным в памяти. Команда `fcomr` дополнительно производит выталкивание из стека сопроцессора. Команда `fcomrr` дополнительно производит два выталкивания из стека сопроцессора. Команды `fcomi`, `fcomip` – сравнивают содержимое `st(0)` с другим регистром, например, `fcomi st(0), st(3)`. Особенностью данных команд является то, что они воздействуют на биты регистра флагов, как скажем команда `cmr`. Команда `fcomip` после выполнения также выталкивает стек сопроцессора. Команды `fcom` и `fcomr` Преобразуют целое число (16 – , 32 - битное), расположенное в памяти в число с плавающей точкой с двойной точностью и сравнивают его с числом, которое находится в регистре `s(0)`. Результатом сравнения являются измененные флаги `C0`, `C2`, `C3`. Кроме этого, команда `fcomr` также осуществляет выталкивание верхнего элемента. Например, `fcomrpq (rdx)`. Инструкция `fst` – сравнивает значение, лежащее в `st(0)` с нулем и, по результату сравнения, устанавливает флаги `C0`, `C2`, `C3`. Команды процессора `fucom`, `fucomr`, `fucomrr` – команды неупорядоченного сравнения вещественных чисел. Сравнивает вершину стека (`st(0)`) с содержимым другого регистра. Второй регистр указывается в качестве единственного операнда. Если операнды отсутствуют, то `st(0)` сравнивается с `st(1)`. Кроме этого, после сравнения `fucomr` производит выталкивание из стека, `fucomrr` производит двойное выталкивание из стека. По результатам сравнения устанавливаются флаги `C0`, `C2`, `C3`. Инструкции `fucomi`, `fucomip`. Сравнивают содержимое регистра `st(0)` с содержимым другого регистра. Регистры указываются явно, например, `fucomi st(0), st(2)` и т.п. Действуют на флаги `ZF`, `PF`, `CF`. После сравнения команда `fucomip` выталкивает стек. Команда `fxam`. Команда определяет, какой тип данных находится в вершине стека `st(0)` и в зависимости от этого устанавливает флаги `C0`, `C1`, `C2`, `C3`.

Константы

Команда `fldl` – загружает константу +1.0 в стек. Команда `fldl2t` – загружает в стек `st(0)` значение $\log_2 10$. Команда `fldlg2` – загружает в стек значение $\log_1 02$. Команда `fldln2` – загружает в стек значение $\log_e 2$. Команда

fldpi – загружает в стек значение числа π . Команда fldz – загружает в стек +0.0.

Разные

Команды fclex и fnclex – сбрасывают флаги исключений в слове состояния. При этом fclex проверяет наличие отложенных незамаскированных исключений. Команда fdecstp – уменьшает указатель стека на 1 (вращение стека). Если указатель стека равен 0, то ему присваивается значение 7. Команда fincstp – увеличивает указатель стека на 1 (вращение стека). Если указатель стека равен 7, то ему присваивается значение равное 0. Команды finit и fninit. Команды инициализации устанавливают некоторые регистры сопроцессора в их начальное состояние, независимо от их предыдущих значений. Команда finit перед сбросом проверяет на наличие отложенных незамаскированных исключений. Команда fldcw – загружает двухбайтовую величину из памяти в регистр управления. Команда fldenv – загружает среду из памяти (управляющее слово, слово состояния, слово тэгов). Команда fnop – не выполняет никаких действий подобно обычной команде por. Команда firstor – загрузить контекст сопроцессора, который хранится по адресу, указанному в качестве операнда. Команды fsave, fnsave – сохраняют контекст сопроцессора в память, по указанному в качестве операнда адресу. При этом команда fsave проверяет наличие отложенных незамаскированных исключений. Инструкция fstcv – копирует слово управления в указанную в качестве операнда переменную. Инструкция fstenv – копирует окружение сопроцессора по указанному, в качестве операнда, адресу. Инструкция fstsw – сохраняет содержимое двухбайтового регистра состояния в регистр ax или двухбайтовую переменную памяти. Например, fstsw ax. Инструкция fwait (или wait) – заставляет процессор прежде, чем продолжать выполнять инструкции, проверить наличие отложенных немаскируемых исключений. Основное предназначение команды – это синхронизация арифметического сопроцессора с устройством обработки целочисленной арифметики. Команда fxrstor – восстанавливает состояния регистров FPU из памяти, ранее заполненной с помощью команды fxsave. В качестве аргумента берется адрес памяти, объемом 512 байтов. Команда fxsave – сохраняет состояние регистров сопроцессора в указанной области памяти, размером 512 байтов. Единственным операндом команды является адрес области памяти.

Расширения процессоров x86-64

Рассматривая архитектуру процессоров x86-64, можно говорить о базовых командах и регистрах и расширениях. Уже глядя на рисунок 7 из параграфа 2.1, можно выделить базу: регистры общего назначения и команды, которые с ними оперируют. Ну, а далее регистры для чисел с плавающей точкой, совмещенные с мультимедийными регистрами и 128-битовые регистры `xmm0-xmm15`, и это уже расширения. Но этим расширения не заканчиваются. Ниже кратко я перечислю эти расширения с краткой их характеристикой.

1. Расширения математического сопроцессора. Набор регистров `st0-st7`, где можно хранить числа с плавающей точкой. В двух предыдущих параграфах мы подробно остановились на этом (параграф 3.7 и параграф 3.8).

2. Технология MMX (Multimedia Extensions – т.е. Мультимедийное Расширение) было разработано одним из подразделений фирмы Intel и предназначено для упрощенной обработки двоичных массивов данных. С точки зрения программирования данное расширение представляет набор команд, оперирующих 64-битовыми числами и набор из восьми регистров (обозначаются как `mm0`, `mm1`, `mm2`, `mm3`, `mm4`, `mm5`, `mm6`, `mm7`), которые, однако физически совмещены с регистрами арифметического сопроцессора, точнее, с их младшими 64-битовыми частями. Из последнего, в частности, вытекает, что использовать одновременно команды MMX и команды арифметического сопроцессора нельзя. Если такая необходимость все же появляется, в начале, следует сохранить содержимое регистров, а после выполнения нужного действия, вновь восстановить (см. команды `fxsave`, `fxrstor`). Инструкции MMX могут оперировать упакованными байтами (8 байтов, упакованных в 64-битовой тип данных), упакованными словами (4 слова упакованных 64-битовый тип данных), упакованными двойными словами (два упакованных двойных слова в 64-битовом типе данных) и, наконец, работать с 64-х битовым типом или учетверенным словом.

3. SSE1-расширение увеличивает количество MMX инструкций, а также добавляет операции над упакованными 32-битовыми числами с плавающей точкой. 128-битовый упакованный формат состоит из четырех 32-битовых чисел с плавающей точкой. 128-битовый SSE-регистры как раз предназначены для выполнения действий над такого рода данными. Всего было добавлено восемь 128-битовых регистров: `xmm0 - xmm7`. К слову

сказать в предыдущем параграфе мы использовали регистры `xmm`. В новых 64-битовых процессорах предполагается, что эти регистры будут использоваться для передачи в функции и оттуда числа с плавающей точкой. SSE - Streaming SIMD Extensions, т.е. потоковое SIMD расширение. SIMD (single instruction, multiple data, т.е. одна инструкция, множество данных) – принцип машинных вычислений.

4. Расширение SSE2 вводит операции над упакованными вещественными числами двойной точности (64-битовыми), расширяет синтаксис команд MMX, а также добавляет новые команды. Как и команды из набора SSE оперируют 128-битовыми регистрами `xmm0` – `xmm15`. Команды SSE2 были использованы в предыдущем параграфе. Ниже мы приведем пример использования команд расширения для работы с 64-битовыми вещественными числами.

5. Расширение SSE3. Дальнейшее расширение технологии SSE, предназначенное для дальнейшего увеличения производительности.

6. Расширение SSE4. Дальнейшее расширение SSE, в основном предназначенное для мультимедийных приложений.

7. Расширения 3DNow! Расширяет возможности MMX без добавления новых режимов и новых регистров. Предназначена для операции над упакованными 64-битовыми вещественными числами, состоящими из двух 32-битовых вещественных чисел.

8. AVX расширение. AVX – Advanced Vector Extension, т.е. усовершенствованное векторное расширение. Данный набор расширяет возможности SSE команд. Кроме того, вводится набор 256-битовых регистров (`ymm0-ymm7`, являющихся расширением регистров `xmm0-xmm7`). По сути команды AVX расширения повторяют часть команд SSE, но несколько в ином формате.

Научное издание

Пирогов Владислав Юрьевич

**АССЕМБЛЕР GAS В ОПЕРАЦИОННОЙ
СИСТЕМЕ LINUX НА ПЛАТФОРМЕ X86-64**

Монография

Подписано к выпуску 30.08.2024.

Уч.-изд. л. 6,16. Объем данных 1,72 Мб.

Электронное издание для распространения через Интернет.

ООО «ФЛИНТА», 117342, г. Москва, ул. Бутлерова, д. 17-Б, офис 324.

Тел.: (495) 334-82-65; 336-03-11.

E-mail: flinta@mail.ru; WebSite: www.flinta.ru

ФГБОУ ВО «Шадринский государственный педагогический университет»

641870, г. Шадринск, ул. К. Либкнехта, д. 3